

**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN**

**Nguyễn Duy Lâm - 22120181
Huỳnh Tấn Lộc - 22120186
Nguyễn Vĩnh Lương - 22120197
Huỳnh Công Minh - 22120209
Võ Hoàng Nguyên - 22120241
Nguyễn Thành Nhật - 22120254**

**HỆ THỐNG HỖ TRỢ SINH VIÊN
PHÂN HỆ QUẢN LÝ TÀI CHÍNH**

**ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHUẨN**

Tp. Hồ Chí Minh, tháng 07/2026

**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN**

**Nguyễn Duy Lâm - 22120181
Huỳnh Tấn Lộc - 22120186
Nguyễn Vĩnh Lương - 22120197
Huỳnh Công Minh - 22120209
Võ Hoàng Nguyên - 22120241
Nguyễn Thành Nhật - 22120254**

**HỆ THỐNG HỖ TRỢ SINH VIÊN
PHÂN HỆ QUẢN LÝ TÀI CHÍNH**

**ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHUẨN**

GIẢNG VIÊN HƯỚNG DẪN

**ThS. Phạm Hoàng Hải
ThS. Trương Phước Lộc**

Tp. Hồ Chí Minh, tháng 07/2026

Lời cam đoan

Nhóm chúng em xin cam đoan toàn bộ nội dung được trình bày trong đồ án tốt nghiệp "**Hệ thống hỗ trợ sinh viên - Phân hệ quản lý tài chính**" là kết quả tìm hiểu, thiết kế và xây dựng do chính nhóm thực hiện, dưới sự hướng dẫn trực tiếp của **ThS. Phạm Hoàng Hải** và **ThS. Trương Phước Lộc**, không sao chép nguyên vẹn từ bất kỳ đề tài hay công trình nào khác.

Toàn bộ số liệu, kết quả thực nghiệm, giải pháp thiết kế hệ thống và mã nguồn trình bày trong báo cáo đều do nhóm trực tiếp thu thập, xây dựng và kiểm chứng trong suốt quá trình thực hiện, đảm bảo tính trung thực và chưa từng được công bố ở bất kỳ công trình nào trước đây. Những tư liệu, ý tưởng hoặc đoạn mã nguồn được kế thừa từ nguồn bên ngoài đều đã được nhóm trích dẫn đầy đủ, rõ ràng trong phần Tài liệu tham khảo.

Nhóm xin chịu hoàn toàn trách nhiệm về tính trung thực và chính xác của toàn bộ nội dung trong đồ án này. Nếu phát hiện bất kỳ sai phạm nào, nhóm xin chịu mọi trách nhiệm trước Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên, Đại học Quốc gia Thành phố Hồ Chí Minh cùng các cơ quan liên quan theo đúng quy định hiện hành.

Lời cảm ơn

Lời đầu tiên, chúng em xin được bày tỏ lòng biết ơn chân thành nhất đến Ban Giám hiệu cùng toàn thể quý thầy cô Trường Đại học Khoa học Tự nhiên, Đại học Quốc gia Thành phố Hồ Chí Minh, đặc biệt là các thầy cô thuộc Khoa Công nghệ Thông tin đã tận tình truyền đạt những kiến thức quý báu và tạo mọi điều kiện tốt nhất cho chúng em trong suốt quá trình học tập và rèn luyện tại trường.

Đặc biệt, chúng em xin gửi lời cảm ơn sâu sắc và chân thành nhất đến thầy **ThS. Phạm Hoàng Hải** và thầy **ThS. Trương Phước Lộc**. Trong suốt quá trình thực hiện dự án tốt nghiệp, các thầy đã luôn dành nhiều thời gian, tâm huyết tận tình hướng dẫn, định hướng khoa học và hỗ trợ chúng em vượt qua những khó khăn để hoàn thành đề tài một cách tốt nhất.

Cuối cùng, chúng em xin chân thành cảm ơn gia đình, bạn bè đã luôn ở bên cạnh động viên, hỗ trợ và đồng hành cùng chúng em trong suốt thời gian qua.

Dù đã dành nhiều thời gian và tâm huyết để hoàn thiện dự án, nhưng báo cáo chắc chắn khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những lời góp ý và chỉ dẫn quý báu từ quý thầy cô để đề tài của chúng em được hoàn thiện và phát triển tốt hơn trong tương lai.

Chúng em xin chân thành cảm ơn!

Đề cương chi tiết

Bản đề cương đã thực hiện và nộp cho giáo vụ trước đó (*phải có chữ ký của giảng viên hướng dẫn*).

Mục lục

Lời cảm ơn	ii
Đề cương	iii
Mục lục	iv
Danh sách hình	viii
Danh sách bảng	x
Danh mục từ viết tắt	xi
Tóm tắt	xii
1 Giới thiệu đề tài	1
1.1 Bối cảnh và lý do chọn đề tài	1
1.2 Mục tiêu và phạm vi đề tài	2
1.2.1 Mục tiêu	2
1.2.2 Phạm vi	3
1.3 Mô tả bài toán	3
1.3.1 Quản lý tài chính cá nhân sinh viên	3
1.3.2 Quản lý học bổng trực tuyến	4
1.3.3 Quản lý dữ liệu học thuật	4
1.3.4 Yêu cầu chức năng	5
1.3.5 Yêu cầu phi chức năng	5

1.4	Vấn đề cần giải quyết và định hướng giải pháp	6
1.4.1	Phân tích các khó khăn vướng mắc	6
1.4.2	Giải pháp đề xuất	7
1.5	Các chức năng chính của hệ thống	8
2	Khảo sát và phân tích các giải pháp liên quan	9
2.1	Các hệ thống quản lý tài chính cá nhân hiện có và hạn chế	9
2.2	Các giải pháp kết nối học bổng truyền thống	10
2.3	Các giải pháp quản lý học thuật và bảng điểm hiện có	11
2.4	Ứng dụng AI trong các hệ thống hỗ trợ sinh viên	12
2.4.1	Trong quản lý tài chính cá nhân	13
2.4.2	Trong hệ gợi ý học bổng (Scholarship Recommendation)	14
2.5	Các giải pháp thông báo đẩy thời gian thực trên nền tảng di động	15
3	Kiến trúc hệ thống	16
3.1	Tổng quan kiến trúc hệ thống	16
3.1.1	Mục tiêu kiến trúc	16
3.1.2	Mô hình kiến trúc tổng thể	16
3.1.3	Các thành phần chính và vai trò	17
3.1.4	Luồng hoạt động tổng quát	19
3.2	Kiến trúc Frontend	20
3.2.1	Ứng dụng Mobile	20
3.2.2	Web Admin	21
3.3	Kiến trúc Backend	21
3.3.1	Vai trò	21
3.3.2	Mô hình kiến trúc	22
3.3.3	Các tầng nội bộ	22
3.3.4	Tổ chức theo miền nghiệp vụ	23
3.3.5	Nguyên tắc thiết kế áp dụng	23
3.4	Kiến trúc hệ thống AI	24
3.4.1	Vai trò và vị trí trong kiến trúc tổng thể	24

3.4.2	Luồng xử lý tổng quát	24
3.4.3	Các thành phần chính	25
3.5	Kiến trúc cơ sở dữ liệu	26
3.6	Luồng tương tác giữa các thành phần	30
3.6.1	Luồng ghi nhận và phân loại giao dịch bằng AI	30
3.6.2	Luồng trò chuyện AI thời gian thực	31
3.6.3	Luồng xét duyệt học bổng và thông báo kết quả	31
3.7	Đánh giá kiến trúc hệ thống	31
3.7.1	Ưu điểm	31
3.7.2	Khả năng mở rộng và bảo trì	32
3.7.3	Hướng mở rộng kiến trúc trong tương lai	32
3.7.4	Giới hạn hiện tại	32
4	Phân tích và đặc tả chức năng nghiệp vụ	33
4.1	Tổng quan chức năng hệ thống	33
4.2	Các tác nhân tham gia hệ thống	33
4.3	Sơ đồ phân rã chức năng hệ thống	34
4.4	Đặc tả chức năng nghiệp vụ	35
4.4.1	Quản lý tài chính cá nhân	35
4.4.2	Trợ lý AI tài chính	45
4.4.3	Quản lý học bổng	48
4.4.4	Quản lý dữ liệu học thuật	54
5	Quá trình phát triển và triển khai	61
5.1	Phương pháp thực hiện	61
5.2	Công cụ hỗ trợ phát triển	61
5.3	Tổ chức nhóm và phân công chuyên môn	62
5.4	Các giai đoạn phát triển	63
6	Kiểm thử và đánh giá hệ thống	64
6.1	Môi trường triển khai và công nghệ sử dụng	64
6.2	Hiện thực các chức năng hệ thống	65

6.2.1	Ứng dụng di động dành cho sinh viên	65
6.2.2	Webadmin quản trị hệ thống	71
6.2.3	Cơ chế đồng bộ trạng thái giao diện tức thời (Query Invalidation)	74
6.3	Kiểm thử hệ thống	75
6.3.1	Kiểm thử chức năng	75
6.3.2	Kiểm thử API	75
6.3.3	Kiểm thử AI Assistant	76
6.3.4	Kiểm thử hiệu năng	80
6.4	Đánh giá các chức năng AI	80
6.4.1	Đánh giá phân loại giao dịch	80
6.4.2	Đánh giá gợi ý học bổng	82
6.5	Đánh giá khả năng đáp ứng hệ thống	83
6.6	Nhận xét và đánh giá tổng quan	84
7	Kết luận và Hướng phát triển	86
7.1	Những kết quả đạt được của đề tài	86
7.1.1	Đánh giá mức độ hoàn thiện hệ thống	87
7.2	Hạn chế của hệ thống	89
7.3	Hướng phát triển tiếp theo	90
	Tài liệu tham khảo	92

Danh sách hình

3.1	Sơ đồ ngữ cảnh hệ thống (System Context Diagram) của Student360	18
3.2	Sơ đồ kiến trúc Container của hệ thống Student360	19
3.3	Luồng xử lý và truyền phản hồi thời gian thực của AI Service . .	26
3.4	Sơ đồ thực thể liên kết (ERD) cho mô-đun Tài chính 6 lọ	28
3.5	Sơ đồ thực thể liên kết (ERD) cho mô-đun Học bổng và Học thuật	29
3.6	Sơ đồ ERD cho phân hệ học bổng	30
4.1	Sơ đồ luồng quản trị dữ liệu học thuật trên Webadmin	56
4.2	Sơ đồ luồng thao tác bảng điểm của sinh viên	58
6.1	Giao diện tổng quan ví 6 lọ trên thiết bị di động	66
6.2	Giao diện trò chuyện thời gian thực với trợ lý AI	66
6.3	Giao diện chi tiết hũ tài chính trên di động	66
6.4	Giao diện cấu hình phần trăm phân bổ hũ tài chính	67
6.5	Giao diện biểu mẫu thêm mới giao dịch tài chính	68
6.6	Giao diện danh sách lịch sử giao dịch của sinh viên	68
6.7	Giao diện quản lý lịch chuyển tiền tự động	68
6.8	Giao diện báo cáo phân tích tài chính chi tiêu	69
6.9	Giao diện gợi ý học bổng thông minh	69
6.10	Giao diện chi tiết học bổng và nộp hồ sơ trên thiết bị di động . .	70
6.11	Giao diện bảng điểm và tiến độ tín chỉ của sinh viên	70
6.12	Giao diện nhập và import điểm sinh viên	70
6.13	Giao diện quản lý danh mục trường đại học	71

6.14	Giao diện quản lý danh mục Trường học	71
6.15	Giao diện quản lý môn học và khoa/chương trình đào tạo	72
6.16	Giao diện quản lý cơ sở dữ liệu Môn học	72
6.17	Giao diện quản lý danh sách Học bổng	72
6.18	Giao diện tạo học bổng dành cho nhà tài trợ	73
6.19	Giao diện thiết lập thông tin Học bổng	73
6.20	Giao diện xét duyệt hồ sơ ứng tuyển học bổng	74
6.21	Giao diện xét duyệt hồ sơ ứng tuyển học bổng	74
6.22	Giao diện mẫu cảnh báo gian lận học bổng (prototype, chưa kết nối dữ liệu thật)	74
6.23	Đồ thị biểu diễn thời gian trễ trung bình và phân vị 95% của thông báo đẩy FCM	84

Danh sách bảng

5.1	Phân công chuyên môn theo mảng chức năng	62
6.1	Kết quả kiểm thử tích hợp API sau cải tiến	76
6.2	Kết quả kiểm thử AI trợ lý ảo – Giai đoạn 1	77
6.3	Kết quả kiểm thử AI trợ lý ảo – Giai đoạn 2	78
6.4	Kết quả kiểm thử E2E Chat API (Non-Stream) sau cải tiến tối ưu	79
6.5	Đánh giá độ chính xác phân loại giao dịch của AI theo hồ tài chính	81
6.6	Kết quả đánh giá cơ chế gợi ý học bổng	82
6.7	Thời gian trễ trung bình của hệ thống gửi thông báo FCM	83
7.1	Bảng so sánh tính năng giữa Student360 và các giải pháp hiện có	88

Danh mục từ viết tắt

STT	Từ viết tắt	Giải thích
1	AI	Trí tuệ nhân tạo (Artificial Intelligence)
2	API	Giao diện lập trình ứng dụng (Application Programming Interface)
3	CRUD	Tạo - Đọc - Cập nhật - Xóa (Create, Read, Update, Delete)
4	DB	Cơ sở dữ liệu (Database)
5	ERD	Sơ đồ thực thể liên kết (Entity Relationship Diagram)
6	FCM	Dịch vụ thông báo đám mây của Firebase (Firebase Cloud Messaging)
7	GPA	Điểm trung bình tích lũy (Grade Point Average)
8	HTTP	Giao thức truyền siêu văn bản (HyperText Transfer Protocol)
9	JSON	Ký hiệu đối tượng JavaScript (JavaScript Object Notation)
10	JWT	Mã thông báo JSON Web (JSON Web Token)
11	LLM	Mô hình ngôn ngữ lớn (Large Language Model)

STT	Từ viết tắt	Giải thích
12	NLP	Xử lý ngôn ngữ tự nhiên (Natural Language Processing)
13	RBAC	Kiểm soát truy cập dựa trên vai trò (Role-Based Access Control)
14	REST	Chuyển trạng thái đại diện (Representational State Transfer)
15	SSE	Sự kiện gửi từ máy chủ (Server-Sent Events)
16	TTL	Thời gian sống của bộ nhớ đệm (Time-To-Live)
17	UI	Giao diện người dùng (User Interface)
18	UUID	Định danh duy nhất toàn cầu (Universally Unique Identifier)
19	UX	Trải nghiệm người dùng (User Experience)

Tóm tắt

Trong đời sống giảng đường, sinh viên thường gặp khó khăn trong việc quản lý tài chính cá nhân một cách khoa học cũng như tiếp cận các thông tin học bổng phù hợp để hỗ trợ quá trình học tập. Nhằm giải quyết bài toán này, đề tài tập trung xây dựng và phát triển hệ thống hỗ trợ sinh viên toàn diện tích hợp Trí tuệ nhân tạo (AI) giúp quản lý tài chính cá nhân và tra cứu học bổng thông minh.

Hệ thống được thiết kế theo kiến trúc đa nền tảng bao gồm ứng dụng di động dành cho sinh viên và trang quản trị Web Admin dành cho người quản lý. Phân hệ tài chính cá nhân áp dụng phương pháp quản lý 6 chiếc lọ, kết hợp cùng trợ lý ảo AI để tư vấn chi tiêu và hỗ trợ lập kế hoạch tài chính hiệu quả. Bên cạnh đó, phân hệ học bổng tích hợp trợ lý AI giúp người dùng tìm kiếm, phân tích và gợi ý học bổng cá nhân hóa dựa trên hồ sơ sinh viên. Trang quản trị Web Admin được xây dựng đồng bộ nhằm phục vụ công tác quản lý thông tin học bổng, quản lý cơ cấu tổ chức trường và các khoa.

Kết quả thử nghiệm cho thấy hệ thống vận hành ổn định, giao diện trực quan, các mô hình AI phản hồi chính xác các truy vấn của người dùng với thời gian trễ thấp. Đề tài đóng góp một giải pháp công nghệ thiết thực, hỗ trợ sinh viên rèn luyện tư duy quản lý tài chính bền vững và gia tăng cơ hội tiếp cận học bổng, đồng thời cung cấp công cụ quản trị dữ liệu học tập hiệu quả cho các đơn vị quản lý.

Chương 1

Giới thiệu đề tài

1.1 Bối cảnh và lý do chọn đề tài

Trong kỷ nguyên số hóa hiện nay, đời sống sinh viên tại các trường Đại học đang đối mặt với nhiều biến động và thách thức to lớn. Sự chuyển dịch kinh tế kéo theo chi phí sinh hoạt, học phí và các nhu cầu dịch vụ học tập ngày càng gia tăng. Đối với phần lớn sinh viên, việc tự chủ cuộc sống khi bước vào môi trường đại học là một bước ngoặt lớn, đi kèm với áp lực tự quản lý bản thân trên nhiều phương diện. Trong số đó, hai vấn đề nổi cộm nhất ảnh hưởng trực tiếp đến chất lượng học tập và sự phát triển bền vững của sinh viên là quản lý tài chính cá nhân và cơ hội tiếp cận các nguồn lực hỗ trợ như học bổng học thuật.

Về khía cạnh quản lý tài chính cá nhân, sinh viên thường thiếu kiến thức nền tảng và kỹ năng quản lý dòng tiền. Thói quen chi tiêu tự phát, thiếu kế hoạch, và không kiểm soát được các khoản phát sinh dẫn đến tình trạng mất cân đối tài chính thường xuyên. Điều này không chỉ gây căng thẳng tâm lý mà còn gián tiếp làm suy giảm kết quả học tập khi sinh viên phải dành nhiều thời gian để đi làm thêm trang trải cuộc sống. Phương pháp quản lý tài chính "6 cái lọ" (6 Jars) của T. Harv Eker là một mô hình nổi tiếng thế giới, giúp phân bổ thu nhập một cách khoa học vào các quỹ chi tiêu thiết yếu, đầu tư, giáo dục, tiết kiệm dài hạn, hưởng thụ và từ thiện [1]. Tuy nhiên, việc áp dụng mô hình này một cách thủ công thường gặp nhiều khó khăn do tính phức tạp trong việc ghi chép, phân loại giao dịch hàng ngày và thiếu tính kỷ luật tự giác.

Về khía cạnh tiếp cận nguồn lực hỗ trợ, học bổng là một trong những mục tiêu quan trọng giúp sinh viên giảm bớt gánh nặng tài chính và tạo động lực phấn đấu trong học tập. Hiện nay, các nguồn thông tin về học bổng rất đa dạng nhưng phân tán từ nhiều nhà tài trợ, doanh nghiệp, tổ chức phi chính phủ đến chính sách nội bộ của từng trường. Sinh viên thường bỏ lỡ các cơ hội học bổng phù hợp do không tiếp cận được thông tin kịp thời hoặc hồ sơ năng lực cá nhân chưa được so khớp tối ưu với các tiêu chí xét tuyển phức tạp. Đồng thời, các nhà tài trợ cũng gặp khó khăn trong việc tìm kiếm đúng đối tượng sinh viên đáp ứng đầy đủ yêu cầu học thuật và hoàn cảnh cụ thể.

Xuất phát từ thực trạng trên, đề tài "**Hệ thống hỗ trợ sinh viên - phân hệ quản lý tài chính**" được thiết kế và phát triển. Hệ thống hướng tới việc xây dựng một hệ sinh thái tích hợp đa nền tảng, áp dụng các giải pháp công nghệ hiện đại và trí tuệ nhân tạo (AI) để giải quyết triệt để hai bài toán cốt lõi: tự động hóa quản lý tài chính cá nhân theo mô hình 6 lọ và tối ưu hóa quy trình so khớp học bổng thông minh cho sinh viên.

1.2 Mục tiêu và phạm vi đề tài

1.2.1 Mục tiêu

Đề tài tập trung vào việc thiết kế và phát triển các mục tiêu cụ thể sau:

- Hiện thực hóa mô hình quản lý tài chính cá nhân "6 chiếc lọ" trên ứng dụng di động, giúp sinh viên thiết lập và duy trì thói quen chi tiêu khoa học.
- Ứng dụng trí tuệ nhân tạo (AI Agent và Large Language Models - LLM) để tự động hóa quy trình phân loại giao dịch tài chính từ ngôn ngữ tự nhiên, cũng như tư vấn tài chính cá nhân thời gian thực thông qua Chatbot trợ lý ảo.
- Xây dựng phân hệ học bổng trực tuyến cho phép nhà tài trợ tạo chương trình học bổng, cấu hình điều kiện và tài liệu yêu cầu; đồng thời hỗ trợ

sinh viên lưu nháp, nộp, theo dõi và hủy hồ sơ ứng tuyển theo quy trình xử lý rõ ràng.

- Xây dựng hệ thống đẩy thông báo di động (FCM Push Notification) thông minh, chạy ngầm định kỳ nhằm đưa ra các cảnh báo tài chính chủ động và thông tin học bổng khẩn cấp tới sinh viên.
- Xây dựng phân hệ quản trị dữ liệu học thuật cho phép quản trị viên quản lý danh mục trường đại học, khoa/chương trình đào tạo và môn học; đồng thời hỗ trợ sinh viên lập bảng điểm cá nhân, theo dõi GPA, tổng tín chỉ và mục tiêu học tập.

1.2.2 Phạm vi

- **Về đối tượng áp dụng:** Tập trung vào hành vi tài chính, chỉ tiêu sinh hoạt thường nhật và hồ sơ năng lực học thuật của sinh viên đại học.
- **Về mặt công nghệ:** Phát triển hệ thống đa dịch vụ theo kiến trúc Client-Server phân tán nhiều tầng, tích hợp: Backend (NestJS, TypeScript), AI Service (FastAPI, Python, LangChain, Google Vertex API), Mobile App (Expo React Native), và Web Portal Admin (React JS, Material UI).
- **Về mặt dữ liệu:** Khai thác dữ liệu học thuật mẫu mô phỏng theo cấu trúc đào tạo tín chỉ tại các trường đại học lớn tại Việt Nam (điển hình là mô hình Đại học Quốc gia TP.HCM).

1.3 Mô tả bài toán

1.3.1 Quản lý tài chính cá nhân sinh viên

Bài toán yêu cầu xây dựng một mô hình số hóa 6 quỹ tài chính độc lập cho từng người dùng, đảm bảo các nguyên tắc:

- **Tính phân bổ tự động:** Khi người dùng phát sinh nguồn thu nhập mới (lương, học bổng, trợ cấp), hệ thống phải tự động phân chia dòng tiền vào 6 lọ theo tỷ lệ phần trăm được cấu hình trước.
- **Tính toàn vẹn dữ liệu tài chính:** Số dư của từng hũ tài chính phải là kết quả tính toán động từ dòng tiền thu chi thực tế trong lịch sử giao dịch để tránh sai lệch số liệu. Khi có hành vi xóa hoặc rebalance (điều chỉnh tỷ lệ hũ), hệ thống phải thực hiện các giao dịch bù trừ nội bộ để đảm bảo tổng tài sản của người dùng không bị thay đổi bất thường.
- **Ngưỡng cảnh báo chi tiêu (Budgeting):** Cho phép người dùng thiết lập ngưỡng cảnh báo cho từng lọ (theo số dư tối thiểu hoặc theo phần trăm đã chi) để được cảnh báo khi chi tiêu vượt ngưỡng thiết yếu.

1.3.2 Quản lý học bổng trực tuyến

Bài toán đặt ra là làm thế nào để số hóa toàn bộ quy trình công bố, tiếp nhận và xét duyệt học bổng giữa nhà tài trợ và sinh viên. Mỗi chương trình học bổng có thông tin riêng về giá trị tài trợ, số lượng suất, thời hạn nộp, điều kiện GPA, ngành học, trường mục tiêu và bộ tài liệu yêu cầu. Hệ thống cần đảm bảo sinh viên có thể tiếp cận thông tin rõ ràng, chuẩn bị hồ sơ trực tuyến và theo dõi trạng thái xử lý sau khi nộp.

Đối với nhà tài trợ, hệ thống cần cung cấp cổng quản trị để tạo và cập nhật học bổng, cấu hình điều kiện xét tuyển, định nghĩa các tài liệu bắt buộc và xem danh sách hồ sơ ứng tuyển theo từng chương trình. Đối với sinh viên, hệ thống cần hỗ trợ lưu nháp, nộp hồ sơ, upload tài liệu minh chứng, cập nhật hồ sơ khi được yêu cầu bổ sung và hủy hồ sơ khi còn trong trạng thái cho phép.

1.3.3 Quản lý dữ liệu học thuật

Để dữ liệu học thuật được sử dụng thống nhất giữa Webadmin và ứng dụng di động, hệ thống cần số hóa quy trình quản lý danh mục trường đại học, khoa/chương trình đào tạo và môn học. Mỗi trường có thể có nhiều chương

trình đào tạo và nhiều môn học. Mỗi môn học cần lưu mã môn, tên môn, mã học phần, số tín chỉ, thang điểm và trạng thái hoạt động để phục vụ quá trình nhập điểm của sinh viên.

Đối với quản trị viên, hệ thống cần cung cấp giao diện tạo, cập nhật, xóa, tìm kiếm và import hàng loạt dữ liệu trường, khoa/chương trình đào tạo và môn học. Đối với sinh viên, hệ thống cần cho phép chọn trường và chương trình đào tạo từ danh mục có sẵn, sau đó sử dụng danh mục môn học của trường để nhập điểm, import bảng điểm, theo dõi GPA và tiến độ tín chỉ.

1.3.4 Yêu cầu chức năng

Để đáp ứng nhu cầu thực tế của sinh viên và các bên liên quan, hệ thống cần đáp ứng các yêu cầu chức năng cốt lõi sau:

- **Quản lý ví tài chính 6 loại:** Cho phép thiết lập tỷ lệ phân bổ, ghi nhận thu chi thủ công và tự động hóa việc chia dòng tiền thu nhập mới vào các loại tương ứng.
- **Trợ lý tài chính ảo AI:** Nhận diện ý định, tự động phân loại giao dịch từ câu viết thường và cung cấp hội thoại tư vấn tài chính cá nhân hóa thời gian thực.
- **Quy trình học bổng trực tuyến:** Hỗ trợ nhà tài trợ công bố, thiết lập tiêu chí và xét duyệt hồ sơ; hỗ trợ sinh viên ứng tuyển và nộp minh chứng trực tuyến.
- **Quản lý học thuật và bảng điểm:** Cung cấp Master Data chuẩn hóa và công cụ nhập điểm, tính toán GPA và tiến độ tích lũy tín chỉ cho sinh viên.

1.3.5 Yêu cầu phi chức năng

Hệ thống phải đảm bảo các tiêu chí kỹ thuật và chất lượng vận hành sau:

- **Tính bảo mật (Security):** Xác thực đa người dùng bằng JWT, kiểm soát phân quyền chặt chẽ (RBAC) và bảo vệ an toàn cho API AI chống Prompt Injection/Data Leaks.
- **Hiệu năng và độ trễ (Performance):** Trả lời câu hỏi của trợ lý AI theo cơ chế streaming (SSE) thay vì chờ toàn bộ phản hồi hoàn tất, giúp sinh viên thấy nội dung xuất hiện dần thay vì phải đợi; độ trễ trung bình đầu-cuối một câu trả lời hoàn chỉnh đạt khoảng 4,2 giây (đo chi tiết tại Chương 6). Hệ thống thông báo đẩy FCM được thiết kế xử lý bất đồng bộ qua hàng đợi BullMQ/Redis nhằm không chặn tiến trình API chính khi gửi số lượng lớn thông báo.
- **Tính sẵn sàng (Availability):** Hệ thống hoạt động ổn định trên nền tảng di động và webadmin với cơ chế lưu đệm (Caching) tối ưu, giảm tải cuộc gọi API ngoài. Các thao tác chuyển khoản đơn giữa các hũ được bọc trong transaction để tránh sai lệch số dư; các tác vụ tổng hợp phức tạp hơn (phân bổ thu nhập, xóa hũ) đang được rà soát để bổ sung cơ chế bảo vệ tương đương.

1.4 Vấn đề cần giải quyết và định hướng giải pháp

1.4.1 Phân tích các khó khăn vướng mắc

Trong quá trình xây dựng hệ thống, nhóm phát triển đối mặt với các khó khăn chính sau:

1. **Độ trễ và chi phí tích hợp AI:** Việc gọi trực tiếp các API của mô hình ngôn ngữ lớn (như Gemini) có độ trễ phản hồi tương đối cao và tốn kém tài nguyên.
2. **Tính chính xác trong phân loại giao dịch:** Ngôn ngữ tự nhiên mà người dùng nhập vào khi ghi chép chi tiêu thường mang tính viết tắt, không chuẩn hóa ngữ pháp (ví dụ: "an trua 50k", "cf 30k").

3. **Đồng bộ trạng thái giao diện thời gian thực:** Giao diện điện thoại di động yêu cầu cập nhật tức thì số dư của các hũ và biểu đồ báo cáo ngay sau khi người dùng thực hiện giao dịch mà không làm chậm trải nghiệm ứng dụng.
4. **Bảo mật dữ liệu và phân quyền hệ thống:** Dữ liệu tài chính cá nhân và hồ sơ học thuật/học bổng của sinh viên là các thông tin nhạy cảm. Hệ thống cần đảm bảo an toàn truy cập dữ liệu, chỉ cho phép các đối tượng thích hợp tiếp cận các tập tài nguyên tương ứng (Sinh viên truy cập app cá nhân, Nhà tài trợ quản lý hồ sơ ứng tuyển, Admin quản trị hệ thống).
5. **Đảm bảo tính nhất quán và toàn vẹn của dữ liệu giao dịch tài chính:** Khi thực hiện các tác vụ phức tạp như phân bổ thu nhập (income distribution) vào 6 hũ hoặc cập nhật thông tin hũ, việc phát sinh lỗi giữa chừng hoặc race conditions có thể dẫn đến sai lệch số dư.
6. **Đồng bộ dữ liệu học thuật và bảng điểm cá nhân:** Danh mục trường, khoa/chương trình đào tạo và môn học cần nhất quán giữa Webadmin và Mobile App. Đồng thời, điểm sinh viên nhập phải được kiểm tra theo thang điểm của từng môn, tránh sai lệch GPA và tổng tín chỉ.

1.4.2 Giải pháp đề xuất

Để vượt qua các khó khăn trên, hệ thống đề xuất các giải pháp kỹ thuật ở mức khái niệm sau đây; chi tiết thiết kế và cơ chế hiện thực cụ thể được trình bày đầy đủ tại Chương 3 và Chương 6:

- **Kiến trúc AI Agent kết hợp lưu trữ nhận định tài chính:** FastAPI AI Service đóng vai trò điều phối (orchestration) độc lập, sử dụng các Tool-calling Agent của LangChain để tối ưu hóa việc gọi mô hình, đồng thời lưu bền vững kết quả nhận định tài chính (AI Insights) theo từng chu kỳ báo cáo để tránh phải gọi lại LLM không cần thiết.

- **Phân loại giao dịch bằng LLM kết hợp Rule Engine:** Sử dụng các bộ quy tắc gán cứng (Rule-based) kết hợp kỹ thuật Few-shot Prompting để hướng dẫn mô hình Gemini phân loại các thuật ngữ viết tắt của người dùng một cách chính xác vào 6 lọ tài chính.
- **Đồng bộ giao diện thời gian thực:** Áp dụng cơ chế Invalidation của thư viện quản lý state phía Frontend để tự động làm mới dữ liệu hiển thị ngay khi có thay đổi từ phía máy chủ, không cần tải lại toàn bộ ứng dụng.
- **Xác thực và phân quyền nhiều tầng:** Sử dụng JSON Web Token (JWT) để xác thực người dùng trên các API, kết hợp cơ chế phân quyền theo vai trò (RBAC) để tách bạch phạm vi truy cập dữ liệu giữa sinh viên, nhà tài trợ và quản trị viên.
- **Đảm bảo toàn vẹn dữ liệu giao dịch tài chính:** Áp dụng Database Transaction cho các thao tác chuyển khoản giữa các hũ để đảm bảo tính nguyên tử (Atomicity), đồng thời tính toán lại số dư từ tổng hợp lịch sử giao dịch thực tế thay vì lưu số dư tĩnh, giúp hạn chế sai lệch dữ liệu khi có thao tác điều chỉnh.
- **Thiết kế cơ sở dữ liệu quan hệ chuẩn hóa cao:** Xây dựng cấu trúc dữ liệu với các ràng buộc khóa ngoại (Foreign Keys) chặt chẽ để đồng bộ dữ liệu học thuật phân cấp từ trường học tới sinh viên, đảm bảo tính nhất quán toàn vẹn dữ liệu.

1.5 Các chức năng chính của hệ thống

Bốn nhóm chức năng cốt lõi đã nêu tại Mục tiêu và Yêu cầu chức năng ở trên được hiện thực hóa thành bốn phân hệ tương ứng của hệ thống: Quản lý tài chính cá nhân, Trợ lý tài chính ảo AI, Quản lý học bổng trực tuyến và Quản lý học thuật/Bảng điểm. Danh sách tác nhân tham gia, sơ đồ phân rã chức năng và đặc tả chi tiết từng nghiệp vụ của bốn phân hệ này được trình bày tại Chương 4.

Chương 2

Khảo sát và phân tích các giải pháp liên quan

2.1 Các hệ thống quản lý tài chính cá nhân hiện có và hạn chế

Trong quá trình xây dựng giải pháp quản lý tài chính cá nhân hiện nay, chúng tôi tiến hành khảo sát các ứng dụng phổ biến trên thị trường toàn cầu và Việt Nam, tiêu biểu như Money Lover, Spendee, và Wallet by BudgetBakers. Các phần mềm này đã giải quyết tốt một số nhu cầu cơ bản của người dùng, cụ thể:

- **Money Lover:** Đây là ứng dụng quản lý tài chính nổi tiếng tại Việt Nam, hỗ trợ đắc lực việc ghi chép các khoản chi tiêu thường ngày, lập ngân sách định kỳ và quét hóa đơn thông qua ảnh chụp. Ứng dụng cung cấp các biểu đồ thống kê trực quan và hỗ trợ liên kết tài khoản ngân hàng để tự động cập nhật số dư.
- **Spendee:** Ứng dụng sở hữu thiết kế giao diện hiện đại, tối giản và tập trung mạnh vào các ví dùng chung (Shared Wallets), hỗ trợ gia đình hoặc các nhóm bạn bè cùng quản lý ngân sách chi tiêu.
- **Wallet by BudgetBakers:** Cung cấp giải pháp báo cáo chuyên sâu, hỗ trợ nhiều loại tiền tệ và tích hợp tính năng đồng bộ hóa tự động với hơn 4000

ngân hàng trên thế giới.

Tuy nhiên, các hệ thống này vẫn tồn tại những hạn chế cốt lõi sau đối với đối tượng sinh viên:

1. **Thiếu tích hợp sâu mô hình 6 lọ:** Mặc dù người dùng có thể tự tạo các danh mục chi tiêu, các ứng dụng trên chưa số hóa quy chuẩn mô hình "6 cái lọ" của T. Harv Eker [1]. Người dùng phải tự thực hiện việc tính toán phần trăm phân bổ và chia nhỏ dòng tiền một cách thủ công mỗi khi có nguồn thu nhập mới.
2. **Mức độ tự động hóa bằng AI còn thấp:** Đa số các ứng dụng chỉ sử dụng các thuật toán so khớp chuỗi (string matching) hoặc biểu thức chính quy (regular expressions) đơn giản để phân loại giao dịch. Khi người dùng nhập mô tả bằng ngôn ngữ tự nhiên viết tắt hoặc sai chính tả, hệ thống thường không nhận diện được và bắt buộc người dùng chọn danh mục thủ công, gây nản lòng và bỏ dở thói quen ghi chép.
3. **Thiếu tính chủ động và tính cá nhân hóa (Proactivity):** Các ứng dụng hiện nay hoạt động theo mô hình thụ động (Passive Model), nghĩa là chỉ ghi nhận và thống kê số liệu sau khi giao dịch đã xảy ra. Hệ thống thiếu các cơ chế phân tích hành vi ngầm để đưa ra các gợi ý tái cấu trúc dòng tiền (Rebalancing) hoặc các cảnh báo chi tiêu vượt hạn mức một cách thông minh trước khi sự cố tài chính xảy ra.

2.2 Các giải pháp kết nối học bổng truyền thống

Đối với sinh viên Việt Nam, các nguồn thông tin học bổng thường được công bố qua các kênh truyền thống như:

- Trang web chính thức của Phòng Công tác Sinh viên (CTSV) hoặc Giáo vụ khoa của các trường Đại học.

- Các diễn đàn trực tuyến của sinh viên, các trang mạng xã hội (Facebook, LinkedIn).
- Các trang tin học bổng chuyên biệt (ví dụ: Scholarship Planet, YBOX).

Các phương thức truyền thông này bộc lộ nhiều hạn chế rõ rệt:

1. **Thông tin bị phân mảnh và chậm trễ:** Sinh viên phải truy cập thủ công nhiều trang web khác nhau để tìm kiếm cơ hội. Nhiều học bổng có thời gian ứng tuyển ngắn dẫn đến việc sinh viên chỉ phát hiện ra khi đã sát hạn chót nộp hồ sơ.
2. **Cơ chế lọc thông tin thô sơ:** Các trang tin học bổng hiện tại chỉ hỗ trợ lọc theo các tiêu chí cứng cơ bản (như Quốc gia, Ngành học chung). Sinh viên không thể biết chính xác hồ sơ năng lực hiện tại của mình (GPA, điểm rèn luyện, chứng chỉ tiếng Anh, hoạt động ngoại khóa) đáp ứng được bao nhiêu phần trăm điều kiện của học bổng đó.
3. **Gánh nặng cho nhà tài trợ và ban quản trị:** Quy trình xét duyệt hồ sơ ứng tuyển của sinh viên đa phần vẫn thực hiện thủ công qua email hoặc Google Forms. Việc này gây khó khăn trong việc quản lý tập trung hồ sơ, dễ xảy ra sai sót khi nhập liệu và khó phát hiện các hành vi gian lận thông tin giữa các ứng viên.

2.3 Các giải pháp quản lý học thuật và bảng điểm hiện có

Song song với nhu cầu quản lý tài chính, sinh viên còn cần theo dõi kết quả học tập và tiến độ tích lũy tín chỉ của bản thân. Các giải pháp hiện có cho nhu cầu này thường thuộc một trong hai nhóm:

- **Cổng thông tin sinh viên (Student Portal) của từng trường đại học:** Đây là kênh chính thức để sinh viên tra cứu điểm số sau khi trường đã

hoàn tất nhập liệu và công bố kết quả. Cổng thông tin đảm bảo tính chính xác tuyệt đối của dữ liệu điểm nhưng thường chỉ hiển thị kết quả cuối cùng, không hỗ trợ sinh viên tự nhập điểm dự kiến để mô phỏng (what-if) các kịch bản GPA trước khi có kết quả chính thức, và hoàn toàn tách biệt với các nhu cầu quản lý khác của sinh viên như tài chính hay học bổng.

- **Ứng dụng tính GPA độc lập:** Nhiều ứng dụng di động hoặc bảng tính (Excel/Google Sheets) cho phép sinh viên tự nhập điểm và số tín chỉ để tính GPA tích lũy. Tuy nhiên, các công cụ này không đồng bộ với danh mục môn học và thang điểm chuẩn hóa của từng trường, khiến sinh viên phải tự tra cứu và nhập thủ công số tín chỉ, dễ dẫn đến sai lệch khi trường có môn học trùng tên nhưng khác số tín chỉ giữa các khóa, hoặc thay đổi thang điểm quy đổi.

Cả hai nhóm giải pháp trên đều tồn tại những hạn chế chung sau đối với sinh viên:

1. **Thiếu liên kết giữa dữ liệu học thuật và các nhu cầu khác của sinh viên:** Kết quả học tập (GPA, tiến độ tín chỉ) là một tiêu chí xét duyệt quan trọng của nhiều chương trình học bổng, nhưng sinh viên phải tự tra cứu, tổng hợp thủ công từ cổng thông tin trường để điền vào hồ sơ ứng tuyển học bổng ở một hệ thống khác, không có cơ chế đối chiếu tự động.
2. **Không chuẩn hóa dữ liệu môn học giữa các trường:** Do mỗi trường có mã môn, số tín chỉ và thang điểm quy đổi riêng, các ứng dụng tính GPA độc lập không thể tự động xác thực dữ liệu sinh viên nhập vào, dẫn đến sai số tích lũy nếu sinh viên nhầm lẫn thông tin môn học.

2.4 Ứng dụng AI trong các hệ thống hỗ trợ sinh viên

Sự bùng nổ của các Mô hình ngôn ngữ lớn (Large Language Models - LLM) và các hệ thống đại lý thông minh (AI Agents) đã mở ra hướng giải quyết mới

cho các bài toán quản lý và gợi ý thông tin.

2.4.1 Trong quản lý tài chính cá nhân

Việc ứng dụng AI giúp chuyển dịch mô hình quản lý tài chính từ thụ động sang chủ động:

- **Xử lý ngôn ngữ tự nhiên (NLP [2]):** Sử dụng LLM để hiểu ý định của người dùng qua tin nhắn thoại hoặc văn bản mô tả ngắn gọn. AI có khả năng phân tích ngữ cảnh để trích xuất các thực thể tài chính như số tiền, mục đích chi tiêu và tự động quy về hồ tài chính tương ứng.
- **Kỹ thuật gợi nhắc few-shot và suy luận từng bước:** Các mô hình ngôn ngữ lớn hiện đại có khả năng học nhanh từ một vài ví dụ minh họa được đưa vào lời nhắc (Few-shot Prompting) mà không cần huấn luyện lại [4], đồng thời có thể được dẫn dắt suy luận qua từng bước trung gian (Chain-of-Thought Prompting) để tăng độ chính xác cho các tác vụ đòi hỏi lập luận nhiều bước [5]. Đây là hai kỹ thuật nền tảng cho cơ chế phân loại giao dịch bằng Few-shot Prompting được trình bày ở Chương 4.
- **Kiến trúc tác nhân suy luận và hành động (ReAct):** Thay vì chỉ trả lời trực tiếp từ kiến thức đã huấn luyện, nhiều hệ thống trợ lý ảo hiện đại kết hợp bước suy luận (reasoning) với bước gọi công cụ bên ngoài (acting) trong một vòng lặp lặp lại, được gọi là mô hình ReAct [6]. Cách tiếp cận này cho phép AI Agent tự quyết định khi nào cần truy vấn dữ liệu thực tế (ví dụ lịch sử chi tiêu) trước khi tổng hợp câu trả lời, là nền tảng cho kiến trúc tác nhân AI được trình bày ở Chương 3.
- **Hệ chuyên gia tư vấn:** AI Agent có thể gọi các công cụ truy vấn cơ sở dữ liệu (Tool-calling) để phân tích lịch sử chi tiêu, từ đó đưa ra các khuyến nghị tài chính cá nhân hóa như cắt giảm chi phí hưởng thụ hoặc tối ưu quỹ tiết kiệm.

2.4.2 Trong hệ gợi ý học bổng (Scholarship Recommendation)

Thay vì lọc dữ liệu theo điều kiện cứng, các nghiên cứu hiện đại áp dụng thuật toán so khớp thực thể và độ đo tương đồng văn bản:

- **Fuzzy Matching và Tokenization:** Áp dụng cho các trường dữ liệu tên trường, tên ngành hoặc kỹ năng, giúp khắc phục các lỗi gõ sai, viết tắt hoặc khác biệt về cách dịch thuật ngữ (ví dụ: "IT", "CNTT", "Information Technology"). Chỉ số độ tương đồng Jaccard [7] và phương pháp N-gram [8] là hai độ đo nền tảng được ứng dụng rộng rãi trong tìm kiếm thông tin không chính xác. Bên cạnh đó, khoảng cách chỉnh sửa Levenshtein [9] là một độ đo kinh điển khác định lượng số phép biến đổi tối thiểu để chuyển một chuỗi ký tự thành chuỗi còn lại, là nền tảng lý thuyết cho các thuật toán so khớp chuỗi xấp xỉ như SequenceMatcher mà hệ thống sử dụng (xem Chương 4).
- **Kết hợp đa độ đo:** Các hệ thống so khớp hiện đại thường không dừng lại ở một độ đo duy nhất mà tổ hợp có trọng số giữa nhiều độ đo tương đồng (ví dụ Jaccard cấp độ token, Jaccard cấp độ bigram ký tự, tỷ lệ khớp chuỗi), nhằm cân bằng giữa độ nhạy cảm từ vựng cục bộ và khả năng chịu lỗi ở mức ký tự, giúp giảm thiểu sai sót với các thuật ngữ viết tắt phổ biến trong môi trường học thuật Việt Nam. Công thức trọng số cụ thể mà hệ thống Student360 áp dụng được trình bày chi tiết tại Chương 4.
- **Nhận diện ý định và phân loại (Intent Classification):** Sử dụng mô hình ngôn ngữ lớn Gemini [10] thông qua thư viện LangChain [11] để xây dựng AI Agent phân loại ý định học bổng của người dùng. Kết quả phân loại ý định giúp Agent điều phối đúng các Tool truy vấn, nâng cao chất lượng câu trả lời tư vấn về học bổng trong hộp thoại AI.

2.5 Các giải pháp thông báo đẩy thời gian thực trên nền tảng di động

Hệ thống thông báo đẩy (Push Notification) đóng vai trò quan trọng trong việc tăng tỷ lệ giữ chân người dùng (Retention Rate) và đảm bảo thông tin khẩn cấp được truyền tải kịp thời. Các công nghệ phổ biến bao gồm:

- **Firestore Cloud Messaging (FCM) [12]:** Dịch vụ đám mây trung gian của Google, hỗ trợ gửi thông báo đẩy đến các thiết bị di động (Android và iOS) ngay cả khi ứng dụng đã bị đóng (Background/Killed state). FCM tối ưu hóa thời lượng pin của thiết bị bằng cách gom nhóm các kết nối mạng ngầm của hệ điều hành. Đây là lựa chọn chính được hệ thống sử dụng để gửi thông báo học bổng và cảnh báo tài chính.
- **BullMQ và Redis Queue [13]:** Để xử lý gửi thông báo đến hàng nghìn thiết bị mà không làm nghẽn tiến trình chính của máy chủ, các hệ thống áp dụng kiến trúc hàng đợi thông điệp (Message Queue). BullMQ kết hợp với Redis [14] giúp quản lý thứ tự gửi, xử lý lỗi tự động (Automatic retries) và phân bổ tài nguyên gửi tin một cách tối ưu. Hệ thống áp dụng kiến trúc BullMQ cho hàng đợi thông báo đẩy học bổng và cảnh báo bất thường chi tiêu; riêng việc quét phát hiện bất thường được thực hiện bởi một Cron Job định kỳ chạy trực tiếp tại Backend NestJS, sau khi phát hiện bất thường mới đẩy cảnh báo vào hàng đợi BullMQ để gửi đi qua FCM.

Chương 3

Kiến trúc hệ thống

3.1 Tổng quan kiến trúc hệ thống

3.1.1 Mục tiêu kiến trúc

Kiến trúc của hệ thống Student360 được thiết kế nhằm đạt được bốn mục tiêu chính: (1) phân tách rõ ràng trách nhiệm giữa các thành phần (Separation of Concerns), giúp mỗi phần chỉ tập trung xử lý đúng vai trò của mình; (2) cho phép các thành phần được phát triển, kiểm thử và vận hành độc lập với nhau; (3) hỗ trợ nhiều thành viên trong nhóm phát triển song song trên các phần khác nhau của hệ thống mà không xung đột mã nguồn với nhau, phù hợp với cách phân công theo cặp đã trình bày ở Chương 5; (4) cô lập được rủi ro và chi phí vận hành của thành phần Trí tuệ nhân tạo (độ trễ, chi phí gọi mô hình ngôn ngữ lớn) khỏi luồng nghiệp vụ lõi của hệ thống.

3.1.2 Mô hình kiến trúc tổng thể

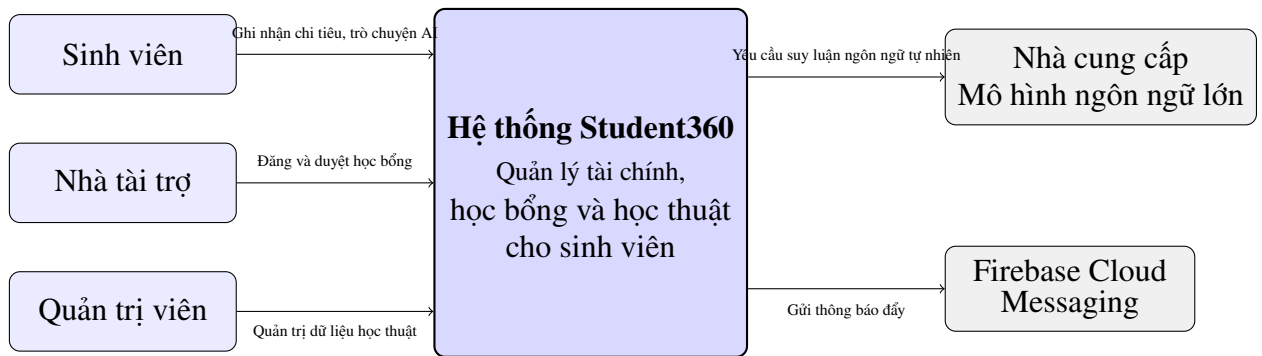
Hệ thống được tổ chức theo mô hình **Kiến trúc Client–Server phân tán nhiều tầng**, gồm ba tầng logic: tầng trình bày (Presentation Tier), tầng ứng dụng/xử lý nghiệp vụ (Application Tier) và tầng dữ liệu (Data Tier). Về mặt triển khai mã nguồn, mỗi thành phần thuộc tầng trình bày và tầng ứng dụng được hiện thực trong một repository Git độc lập (đa-repository), không dùng

chung một repository duy nhất. Các thành phần giao tiếp với nhau hoàn toàn qua mạng, thông qua các giao thức kết nối tiêu chuẩn như HTTP REST, Server-Sent Events (SSE) và hàng đợi thông điệp (Message Queue), chứ không chia sẻ trực tiếp mã nguồn hay tiến trình với nhau. Cách tổ chức này cho phép mỗi dịch vụ được phát triển, triển khai và mở rộng độc lập, đồng thời phù hợp với quy mô một nhóm nhỏ (6 thành viên) khi mỗi cặp thành viên có thể làm việc trọn vẹn trên repository của mình mà ít va chạm với phần việc của cặp khác.

3.1.3 Các thành phần chính và vai trò

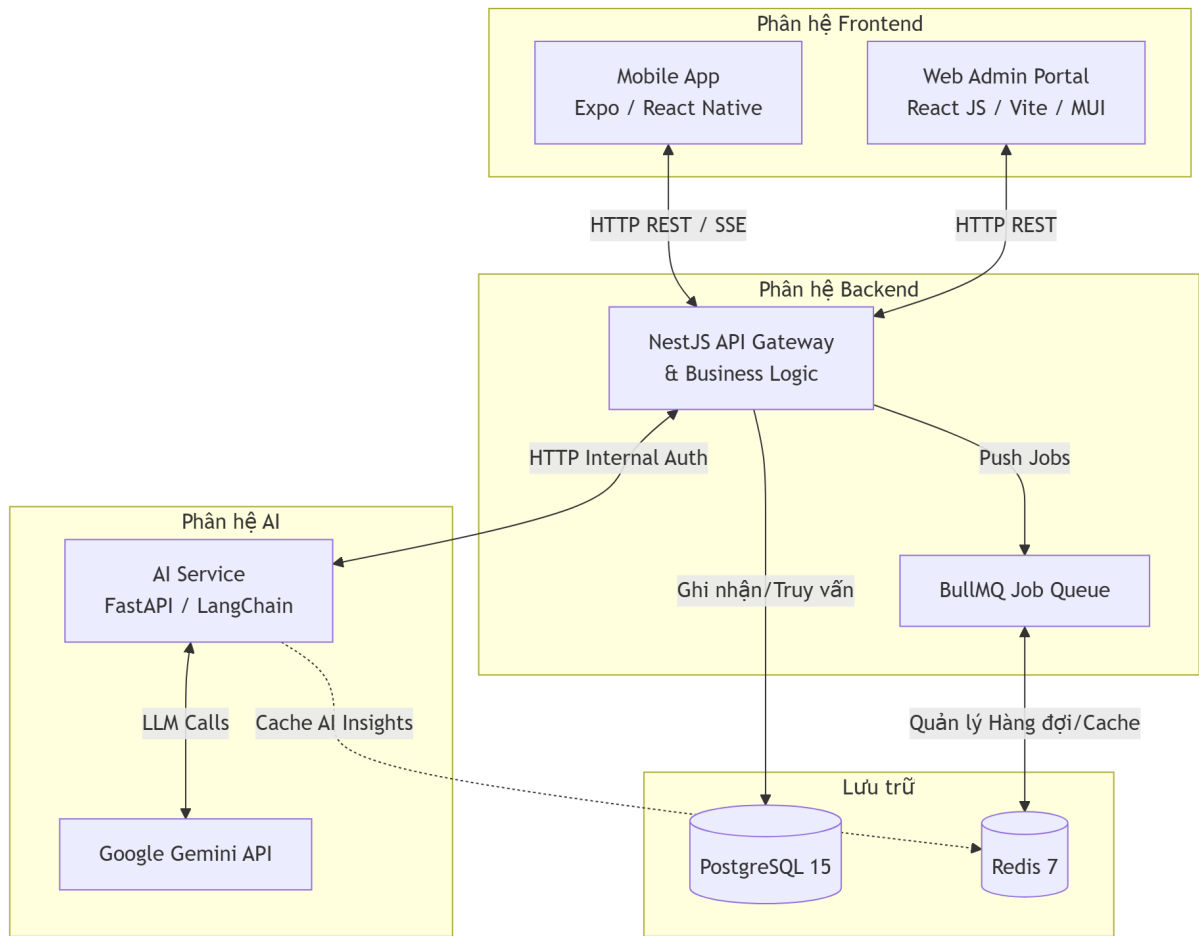
- **Mobile App:** Kênh tương tác chính của sinh viên với hệ thống, thuộc tầng trình bày.
- **Web Admin Portal:** Công cụ quản trị dành cho nhà tài trợ và quản trị viên, thuộc tầng trình bày.
- **Backend:** Trung tâm xử lý nghiệp vụ, xác thực, phân quyền và điều phối truy cập dữ liệu, thuộc tầng ứng dụng.
- **AI Service:** Dịch vụ chuyên trách xử lý ngôn ngữ tự nhiên và các tác vụ liên quan tới Trí tuệ nhân tạo, thuộc tầng ứng dụng.
- **Tầng lưu trữ (PostgreSQL và Redis):** Nơi lưu trữ dữ liệu nghiệp vụ và quản lý hàng đợi xử lý bất đồng bộ, thuộc tầng dữ liệu.

Hình 3.1 mô tả sơ đồ ngữ cảnh hệ thống (System Context Diagram), thể hiện các tác nhân tương tác với hệ thống và các hệ thống bên ngoài mà Student360 phụ thuộc vào.



Hình 3.1: Sơ đồ ngữ cảnh hệ thống (System Context Diagram) của Student360

Ở mức chi tiết hơn, Hình 3.2 trình bày sơ đồ kiến trúc Container, thể hiện các thành phần triển khai cụ thể (Mobile App, Web Admin Portal, Backend, AI Service, hàng đợi và tầng lưu trữ) cùng giao thức giao tiếp giữa chúng.



Hình 3.2: Sơ đồ kiến trúc Container của hệ thống Student360

3.1.4 Luồng hoạt động tổng quát

Người dùng thao tác trên Mobile App hoặc Web Admin Portal, gửi yêu cầu tới Backend qua giao thức HTTP REST thông thường, hoặc qua kênh SSE riêng đối với chức năng trò chuyện AI cần phản hồi theo thời gian thực. Backend xác thực danh tính và phân quyền truy cập, sau đó xử lý nghiệp vụ và đọc/ghi dữ liệu trực tiếp trên PostgreSQL. Đối với các yêu cầu cần năng lực xử lý ngôn ngữ tự nhiên (tư vấn tài chính, phân loại giao dịch, gợi ý học bổng), Backend đóng vai trò cổng trung gian, chuyển tiếp yêu cầu sang AI Service; AI Service không truy cập trực tiếp vào cơ sở dữ liệu mà luôn gọi ngược lại Backend để lấy dữ liệu thật khi cần suy luận. Kết quả xử lý cuối cùng được trả ngược qua Backend

về lại giao diện người dùng. Đối với các tác vụ không cần phản hồi tức thời (gửi thông báo hàng loạt, xử lý nền định kỳ), Backend đẩy tác vụ vào hàng đợi Redis để xử lý bất đồng bộ, tránh làm chậm các yêu cầu API chính.

3.2 Kiến trúc Frontend

3.2.1 Ứng dụng Mobile

Ứng dụng Mobile (Expo/React Native) là kênh tương tác chính của sinh viên với hệ thống, đảm nhận việc ghi nhận thu chi tài chính, trò chuyện với trợ lý AI, ứng tuyển học bổng, quản lý bảng điểm cá nhân và nhận thông báo đẩy.

Về mặt tổ chức, ứng dụng được chia thành ba tầng logic tách biệt: tầng giao diện và định tuyến, tổ chức theo mô hình định tuyến dựa trên tệp tin (file-based routing) giúp cấu trúc màn hình phản ánh trực tiếp cấu trúc điều hướng của ứng dụng; tầng trạng thái, tách biệt rõ giữa trạng thái phiên đăng nhập/giao diện cục bộ và trạng thái dữ liệu nghiệp vụ được đồng bộ từ máy chủ; và tầng giao tiếp API, tập trung toàn bộ việc gọi Backend qua một lớp client duy nhất.

Về xử lý dữ liệu, trạng thái phiên đăng nhập và các trạng thái giao diện cục bộ (như phiên trò chuyện AI, bộ lọc hiển thị) được quản lý độc lập với trạng thái dữ liệu nghiệp vụ lấy từ Backend. Dữ liệu nghiệp vụ được đồng bộ hai chiều với Backend thông qua cơ chế làm mới bộ đệm tự động mỗi khi có thay đổi (đã trình bày cơ chế cụ thể ở Chương 6, mục Cơ chế đồng bộ trạng thái giao diện tức thời), giúp số dư và lịch sử giao dịch trên màn hình luôn phản ánh đúng dữ liệu mới nhất mà không cần người dùng tải lại thủ công. Lớp giao tiếp API tự động đính kèm token xác thực vào mỗi yêu cầu và tự động làm mới token khi hết hạn, giúp phiên đăng nhập của người dùng được duy trì liên mạch.

Việc lựa chọn nền tảng Expo/React Native cho phép phát triển đồng thời cho cả iOS và Android từ một mã nguồn duy nhất, phù hợp với quy mô và thời gian phát triển hạn chế của một đồ án tốt nghiệp. Thư viện thiết kế giao diện Tamagui được sử dụng nhất quán trong toàn bộ ứng dụng, đảm bảo giao diện đồng bộ về phong cách và tối ưu về hiệu năng hiển thị trên thiết bị di động.

3.2.2 Web Admin

Web Admin Portal (React JS/Vite) là công cụ dành cho quản trị viên và nhà tài trợ, phục vụ việc quản lý dữ liệu học thuật chuẩn hóa (trường, chương trình đào tạo, môn học) và toàn bộ vòng đời của một chương trình học bổng, từ khởi tạo, cấu hình điều kiện cho tới xét duyệt hồ sơ ứng tuyển của sinh viên.

Về mặt tổ chức, Web Admin Portal được chia theo miền nghiệp vụ, mỗi miền là một khối chức năng độc lập có luồng dữ liệu và giao diện riêng. Việc kiểm soát truy cập theo vai trò (quản trị viên hay nhà tài trợ) được thực hiện ngay ở tầng định tuyến, đảm bảo mỗi vai trò chỉ nhìn thấy và thao tác được trên đúng phần giao diện được phép truy cập.

Trạng thái đăng nhập và thông tin phân quyền của người dùng được quản lý tập trung trong toàn ứng dụng, trong khi dữ liệu nghiệp vụ của từng miền chức năng (danh sách trường, danh sách học bổng, hồ sơ ứng tuyển...) được đồng bộ riêng theo từng khu vực giao diện tương ứng. Giống như ứng dụng Mobile, Web Admin Portal giao tiếp với Backend qua một lớp client tập trung, tự động đính kèm token xác thực vào mỗi yêu cầu.

Việc lựa chọn React JS kết hợp Vite giúp tối ưu tốc độ phát triển và build ứng dụng, trong khi thư viện Material UI cho phép dựng nhanh các giao diện quản trị dữ liệu dạng bảng với khối lượng bản ghi lớn, vốn là nhu cầu chính của một cổng quản trị.

3.3 Kiến trúc Backend

3.3.1 Vai trò

Backend, hiện thực trên nền tảng NestJS [15], là trung tâm điều phối của toàn hệ thống: xác thực người dùng, phân quyền truy cập, xử lý toàn bộ logic nghiệp vụ, là điểm truy cập duy nhất tới cơ sở dữ liệu quan hệ, đồng thời đóng vai trò cổng trung gian (gateway) an toàn cho các yêu cầu cần tới năng lực xử lý của AI Service. Việc lựa chọn NestJS xuất phát từ kiến trúc module hóa sẵn

có của framework (hỗ trợ tách rời Controller, Service và Guard một cách tường minh, phù hợp trực tiếp với mô hình phân lớp trình bày ở Mục 3.3.2) và hệ sinh thái TypeScript giúp giảm thiểu lỗi thời gian chạy khi nhiều thành viên cùng phát triển song song trên các module khác nhau.

3.3.2 Mô hình kiến trúc

Backend được tổ chức theo **Kiến trúc phân lớp (Layered Architecture)**: một dịch vụ duy nhất, được module hóa rõ ràng theo từng miền nghiệp vụ, chứ không tách thành nhiều dịch vụ vận hành độc lập theo đúng nghĩa Microservices, cũng không áp dụng các mô hình như CQRS (tách biệt luồng xử lý lệnh ghi và truy vấn đọc) hay kiến trúc hướng sự kiện (Event-driven Architecture) với cơ chế phát/nhận sự kiện nghiệp vụ. Lựa chọn này xuất phát từ quy mô của nhóm phát triển (6 thành viên) và thời gian thực hiện đồ án có giới hạn: việc module hóa theo miền nghiệp vụ trong một dịch vụ duy nhất đã đủ để tách biệt rõ trách nhiệm giữa các phần chức năng, trong khi tránh được chi phí vận hành và điều phối phát sinh khi phải triển khai, giám sát và đồng bộ dữ liệu giữa nhiều dịch vụ độc lập như kiến trúc Microservices thực thụ đòi hỏi.

3.3.3 Các tầng nội bộ

Một yêu cầu đi qua Backend lần lượt qua các tầng sau:

- **Tầng tiếp nhận và xác thực yêu cầu:** tiếp nhận yêu cầu từ client, xác thực danh tính người dùng qua JSON Web Token (JWT) và kiểm tra quyền truy cập theo vai trò trước khi cho phép yêu cầu đi tiếp.
- **Tầng chuẩn hóa và giám sát:** chuẩn hóa cấu trúc phản hồi trả về và ghi log của mỗi yêu cầu, phục vụ việc theo dõi và gỡ lỗi hệ thống.
- **Tầng xử lý nghiệp vụ:** nơi hiện thực toàn bộ logic nghiệp vụ cốt lõi của hệ thống, độc lập với cách yêu cầu được tiếp nhận hay dữ liệu được lưu trữ.

- **Tầng truy xuất dữ liệu:** chịu trách nhiệm đọc/ghi dữ liệu trên PostgreSQL, cô lập tầng xử lý nghiệp vụ khỏi chi tiết truy vấn cơ sở dữ liệu.
- **Tầng đẩy tác vụ nền:** đối với các tác vụ không cần phản hồi tức thời (gửi thông báo hàng loạt, lên lịch giao dịch định kỳ), Backend đẩy tác vụ vào hàng đợi Redis/BullMQ thay vì xử lý đồng bộ trong cùng một yêu cầu, giúp không làm chậm tiến trình xử lý API chính.

3.3.4 Tổ chức theo miền nghiệp vụ

Trong phạm vi đề tài này, Backend được tổ chức thành các miền nghiệp vụ chính: Xác thực và phân quyền, Tài chính 6 lọ, Học bổng, Học thuật/Bảng điểm và Thông báo. Cần lưu ý rằng Backend là nền tảng dùng chung cho toàn bộ hệ sinh thái Student360, phục vụ cả các phân hệ khác ngoài phạm vi đề án này; nội dung trình bày trong báo cáo chỉ tập trung vào các miền nghiệp vụ liên quan trực tiếp tới tài chính, học bổng và học thuật.

3.3.5 Nguyên tắc thiết kế áp dụng

- **Phân tách trách nhiệm:** tầng xác thực/phân quyền và tầng chuẩn hóa phản hồi được tách hoàn toàn khỏi tầng xử lý nghiệp vụ, mỗi thay đổi về quy tắc xác thực hay định dạng phản hồi không ảnh hưởng tới logic nghiệp vụ.
- **Khả năng mở rộng:** việc module hóa theo miền nghiệp vụ cho phép bổ sung miền chức năng mới mà không ảnh hưởng tới các miền đã có; xử lý bất đồng bộ qua hàng đợi giúp hệ thống chịu tải tốt hơn khi cần gửi thông báo tới số lượng lớn người dùng cùng lúc.
- **Khả năng bảo trì:** cấu trúc module hóa rõ ràng theo miền nghiệp vụ giúp việc định vị và sửa lỗi trong một miền cụ thể không đòi hỏi phải hiểu toàn bộ hệ thống.

- **Khả năng tích hợp hệ thống ngoài:** việc gọi sang AI Service được cô lập qua một lớp client HTTP trung gian riêng biệt, giúp phần lõi nghiệp vụ của Backend không phụ thuộc trực tiếp vào chi tiết triển khai của AI Service, dễ dàng thay đổi hoặc nâng cấp AI Service mà không ảnh hưởng Backend.

3.4 Kiến trúc hệ thống AI

3.4.1 Vai trò và vị trí trong kiến trúc tổng thể

AI Service, hiện thực bằng FastAPI [16] kết hợp thư viện LangChain [11] để xây dựng tác nhân AI và kết nối tới mô hình ngôn ngữ lớn Gemini [10] của Google, cung cấp năng lực xử lý ngôn ngữ tự nhiên cho hệ thống: tư vấn tài chính qua hội thoại, phân loại giao dịch từ mô tả viết tay của người dùng, và gợi ý học bổng phù hợp với hồ sơ sinh viên. Dịch vụ này được tách thành một thành phần độc lập trong kiến trúc tổng thể nhằm cô lập độ trễ và chi phí gọi mô hình ngôn ngữ lớn khỏi luồng xử lý nghiệp vụ lõi của Backend. AI Service chỉ tiếp nhận yêu cầu từ Backend, không có client nào được phép gọi trực tiếp; mọi yêu cầu giữa hai dịch vụ đều được xác thực nội bộ bằng một token dùng chung. Việc lựa chọn Python/FastAPI thay vì cùng nền tảng NestJS với Backend xuất phát từ hệ sinh thái thư viện AI/LLM phong phú của Python, trong khi LangChain cung cấp sẵn các thành phần trừu tượng hóa cho tác nhân tool-calling và khả năng tích hợp đa nhà cung cấp mô hình ngôn ngữ, phù hợp trực tiếp với tầng trừu tượng hóa LLM mô tả ở Mục 3.4.3.

3.4.2 Luồng xử lý tổng quát

Khi sinh viên đặt một câu hỏi hoặc nhập một mô tả giao dịch trên Mobile App, yêu cầu được Backend xác thực rồi chuyển tiếp sang AI Service. Tại đây, hệ thống trước tiên phân loại ý định của câu hỏi để xác định ngữ cảnh và phạm vi xử lý phù hợp, sau đó một tác nhân AI (Agent) tiến hành suy luận theo mô

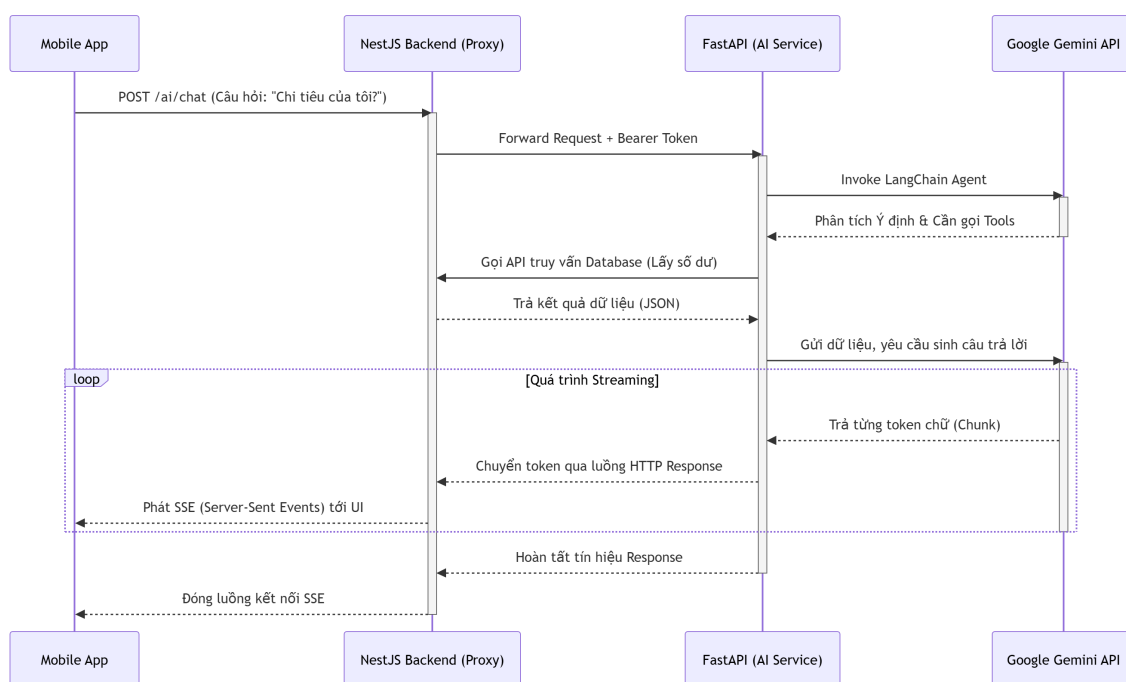
hình lập truy vấn–hành động (ReAct: reasoning + acting) [6], gọi tới các công cụ nghiệp vụ tương ứng khi cần dữ liệu thực tế (thông qua việc gọi ngược lại Backend). Kết quả sau khi được tổng hợp từ mô hình ngôn ngữ lớn được trả về dưới dạng luồng token liên tục cho Backend, và Backend tiếp tục đẩy luồng này về ứng dụng di động để hiển thị theo thời gian thực, tương tự trải nghiệm của các trợ lý ảo thương mại.

3.4.3 Các thành phần chính

- **Tầng tiếp nhận yêu cầu:** nhận yêu cầu từ Backend, xác thực token nội bộ trước khi chuyển tiếp cho tầng xử lý.
- **Tầng phân loại ý định:** xác định loại câu hỏi (truy vấn tài chính, phân loại giao dịch, tư vấn học bổng...) để giới hạn đúng phạm vi ngữ cảnh và tập công cụ được phép sử dụng cho từng loại câu hỏi, thay vì luôn cho phép toàn bộ công cụ có sẵn – giúp giảm chi phí gọi mô hình và tăng độ chính xác của kết quả.
- **Tầng suy luận và gọi công cụ:** một tác nhân AI duy nhất theo mô hình tool-calling (ReAct), được "thu hẹp phạm vi" linh hoạt theo kết quả phân loại ý định ở bước trước, thay vì cần nhiều tác nhân chuyên biệt tách rời phối hợp với nhau.
- **Tầng công cụ nghiệp vụ:** tập hợp các hàm truy vấn và tính toán cụ thể mà tác nhân AI có thể gọi khi cần dữ liệu thực tế, ví dụ truy vấn sổ dư hũ tài chính, lịch sử giao dịch, hoặc tính điểm phù hợp giữa hồ sơ sinh viên và một chương trình học bổng.
- **Tầng trừu tượng hóa nhà cung cấp mô hình ngôn ngữ lớn:** cho phép hệ thống chuyển đổi hoặc dự phòng linh hoạt giữa nhiều nhà cung cấp mô hình ngôn ngữ khi gặp lỗi hạn mức hoặc gián đoạn kết nối, mà không ảnh hưởng tới logic nghiệp vụ ở các tầng phía trên.

- **Tầng xử lý nền:** một tiến trình hoàn toàn tách biệt khỏi tiến trình xử lý yêu cầu chính, đảm nhận các tác vụ định kỳ không cần phản hồi tức thời, như tổng hợp nhận định tài chính theo chu kỳ hoặc dọn dẹp các phiên hội thoại không còn hoạt động.

Hình 3.3 minh họa luồng xử lý và truyền phản hồi theo thời gian thực nói trên.



Hình 3.3: Luồng xử lý và truyền phản hồi thời gian thực của AI Service

3.5 Kiến trúc cơ sở dữ liệu

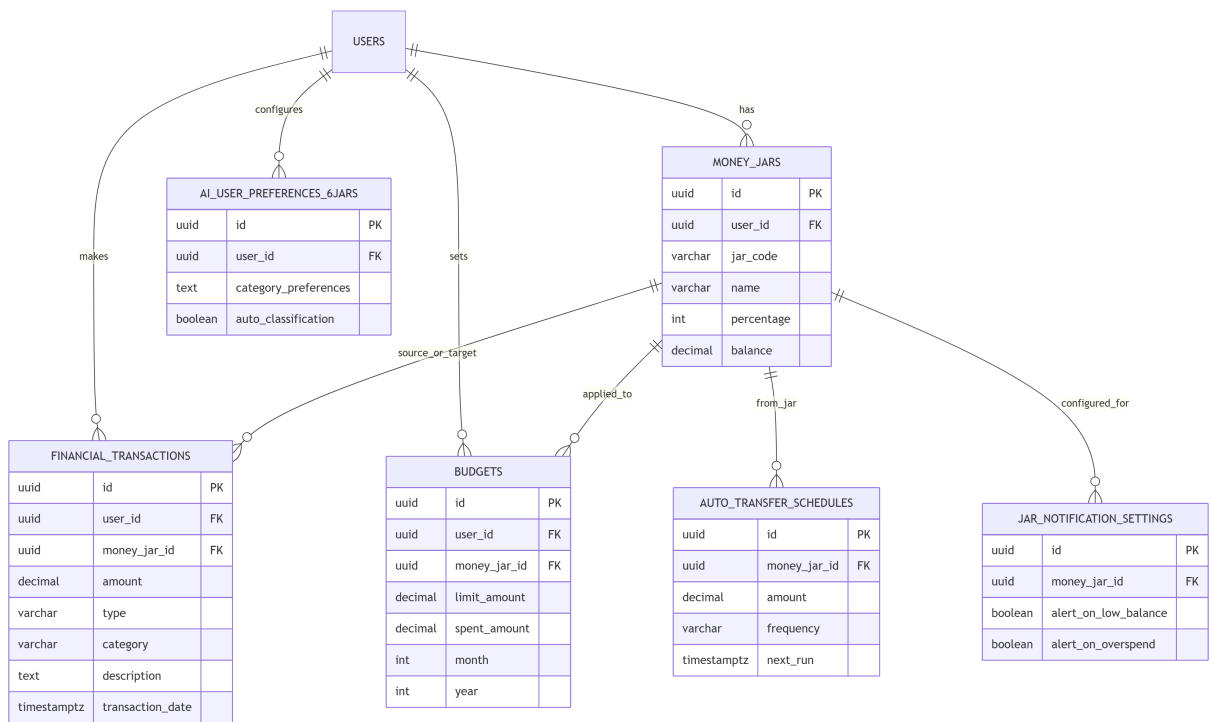
Hệ thống sử dụng cơ sở dữ liệu quan hệ **PostgreSQL 15** [17] làm nguồn dữ liệu tin cậy duy nhất (single source of truth) cho toàn bộ dữ liệu nghiệp vụ, được chuẩn hóa theo mô hình quan hệ với các ràng buộc khóa ngoại chặt chẽ để đảm bảo tính toàn vẹn dữ liệu. Bên cạnh đó, **Redis 7** [14] đóng vai trò message broker phục vụ hàng đợi xử lý bất đồng bộ, không lưu trữ dữ liệu nghiệp vụ lâu dài.

Dữ liệu của hệ thống được tổ chức thành ba nhóm chính, thể hiện trong sơ đồ ERD ở Hình 3.4, Hình 3.5 và Hình 3.6:

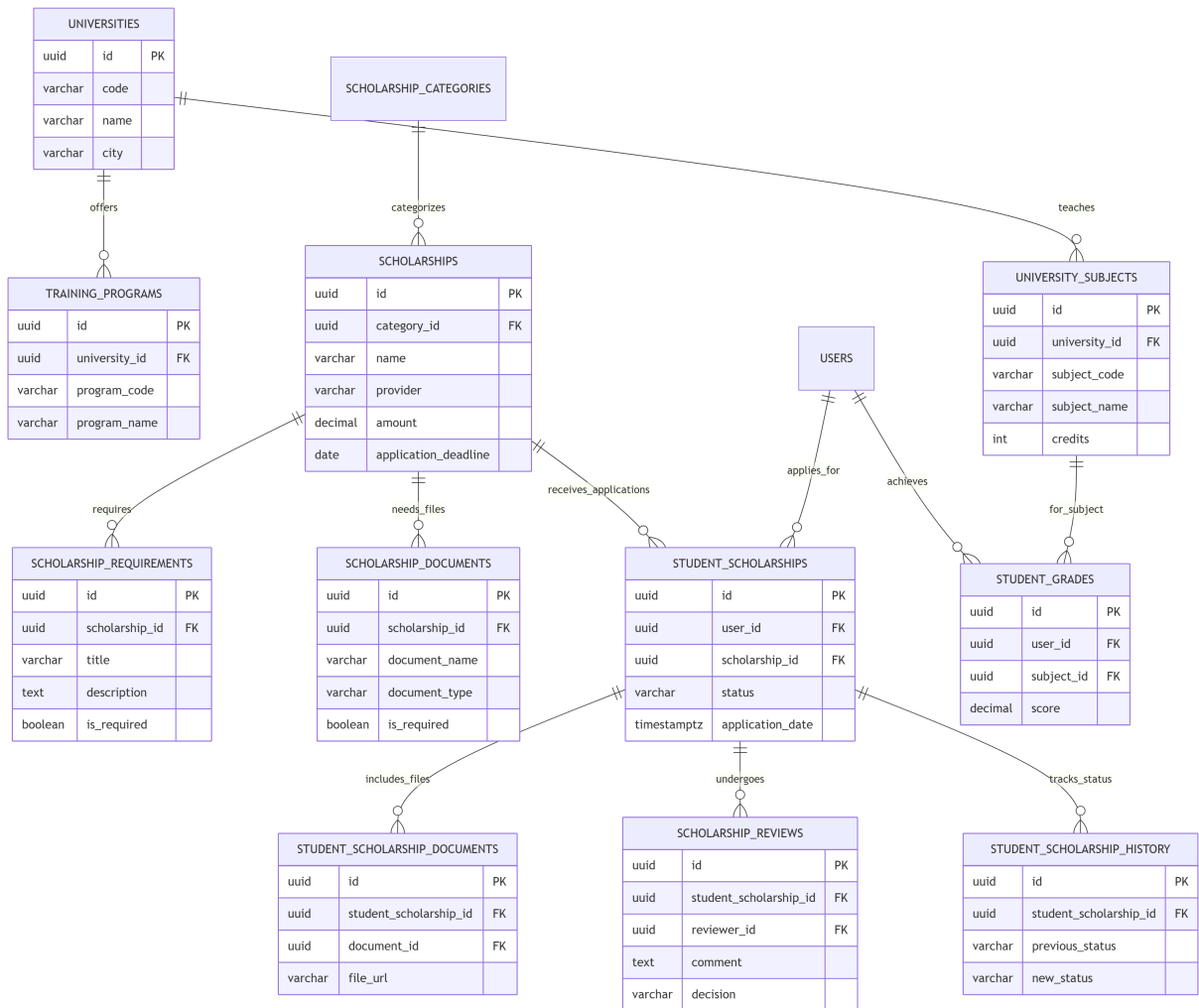
- **Nhóm Tài chính:** quản lý các hũ tài chính, lịch sử giao dịch thu chi, lịch trình chuyển tiền tự động và cấu hình ngưỡng cảnh báo chi tiêu theo từng hũ.
- **Nhóm Học thuật:** quản lý danh mục trường đại học, chương trình đào tạo, môn học, hồ sơ học thuật và bảng điểm cá nhân của sinh viên.
- **Nhóm Học bổng:** quản lý danh mục và thông tin chương trình học bổng, điều kiện xét tuyển, tài liệu yêu cầu, hồ sơ ứng tuyển của sinh viên, lịch sử chuyển trạng thái và quá trình đánh giá hồ sơ.

Về quan hệ giữa các nhóm, nhóm Học bổng tham chiếu tới nhóm Học thuật để đối chiếu điều kiện xét duyệt (ví dụ điểm GPA tối thiểu, ngành học phù hợp) với hồ sơ học thuật thực tế của sinh viên. Nhóm Tài chính được thiết kế độc lập hoàn toàn với hai nhóm còn lại, đúng theo ranh giới nghiệp vụ giữa quản lý dòng tiền cá nhân và quản lý học tập/học bổng.

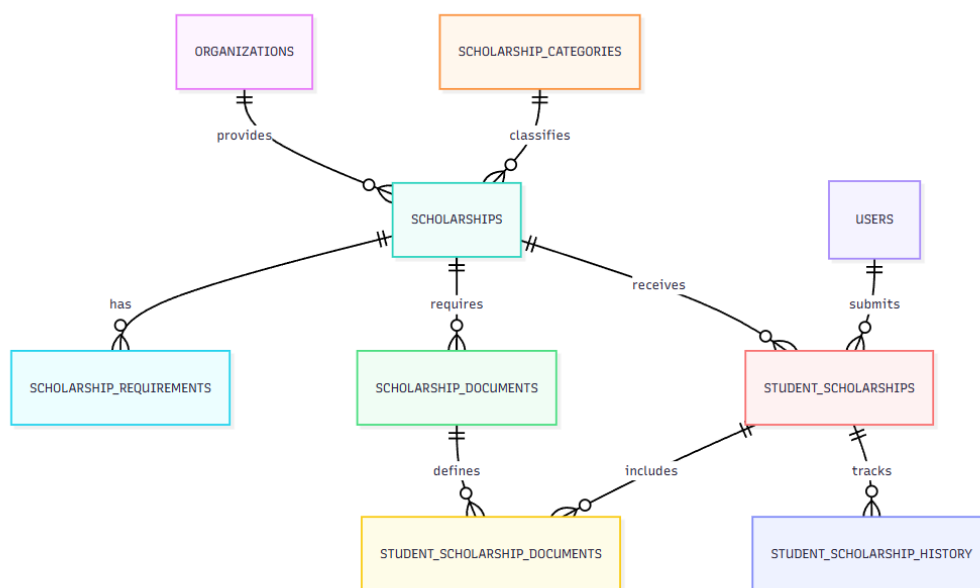
Về mặt thiết kế, số dư của mỗi hũ tài chính không được lưu trữ tĩnh mà luôn được tính toán lại từ lịch sử giao dịch thực tế mỗi khi cần hiển thị, nhằm đảm bảo tính nhất quán và tránh sai lệch dữ liệu do lỗi bất đồng bộ giữa các thao tác ghi. Cách tính cụ thể và cơ chế xử lý các trường hợp đặc biệt (như xóa một hũ tài chính đang có số dư) được trình bày chi tiết cùng đặc tả chức năng liên quan ở Chương 4.



Hình 3.4: Sơ đồ thực thể liên kết (ERD) cho mô-đun Tài chính 6 lọ



Hình 3.5: Sơ đồ thực thể liên kết (ERD) cho mô-đun Học bổng và Học thuật



Hình 3.6: Sơ đồ ERD cho phân hệ học bổng

3.6 Luồng tương tác giữa các thành phần

Phần này mô tả ba luồng tương tác tiêu biểu giữa các thành phần kiến trúc, ở góc nhìn thành phần nào tham gia và giao tiếp qua kênh nào; đặc tả chi tiết từng bước nghiệp vụ và endpoint cụ thể của các luồng này được trình bày tại Chương 4.

3.6.1 Luồng ghi nhận và phân loại giao dịch bằng AI

Sinh viên nhập một mô tả giao dịch bằng ngôn ngữ tự nhiên trên Mobile App. Yêu cầu được gửi tới Backend qua HTTP REST; Backend chuyển tiếp sang AI Service để phân loại; AI Service trả về dữ liệu tài chính và danh mục chi tiêu được đề xuất; Backend chuyển kết quả này về Mobile App để sinh viên xác nhận trước khi lưu giao dịch vào PostgreSQL.

3.6.2 Luồng trò chuyện AI thời gian thực

Sinh viên đặt câu hỏi trong màn hình trò chuyện AI. Mobile App mở một kết nối SSE tới Backend; Backend xác thực rồi chuyển tiếp yêu cầu sang AI Service; AI Service suy luận và truy vấn dữ liệu thật qua Backend khi cần, sau đó truyền câu trả lời về dưới dạng luồng token liên tục; Backend chuyển tiếp luồng này về Mobile App để hiển thị theo thời gian thực (Hình 3.3).

3.6.3 Luồng xét duyệt học bổng và thông báo kết quả

Nhà tài trợ cập nhật trạng thái một hồ sơ ứng tuyển trên Web Admin Portal. Yêu cầu được gửi tới Backend qua HTTP REST; Backend cập nhật trạng thái hồ sơ trong PostgreSQL, sau đó đẩy một tác vụ gửi thông báo vào hàng đợi Redis thay vì gửi ngay trong cùng yêu cầu; một worker chạy nền lấy tác vụ ra khỏi hàng đợi và gọi Firebase Cloud Messaging để gửi thông báo đẩy tới thiết bị của sinh viên.

3.7 Đánh giá kiến trúc hệ thống

3.7.1 Ưu điểm

Kiến trúc phân tán nhiều tầng, đa-repository giúp tách trách nhiệm rõ ràng theo cả chiều tầng (trình bày/ứng dụng/dữ liệu) lẫn chiều dịch vụ (Mobile, Web Admin, Backend, AI Service). Mỗi cặp thành viên trong nhóm có thể phát triển độc lập trên repository riêng của mình, hạn chế xung đột mã nguồn khi làm việc song song (liên kết với cách phân công tại Chương 5). Việc tách AI Service thành một dịch vụ riêng giúp cô lập hoàn toàn rủi ro về độ trễ, lỗi hoặc chi phí gọi mô hình ngôn ngữ lớn khỏi các nghiệp vụ lõi (tài chính, học bổng), đảm bảo các chức năng cốt lõi vẫn hoạt động ổn định ngay cả khi thành phần AI gặp sự cố. Việc xử lý bất đồng bộ qua hàng đợi giúp hệ thống chịu tải tốt khi cần gửi thông báo tới số lượng lớn người dùng cùng lúc mà không làm chậm các yêu cầu API chính.

3.7.2 Khả năng mở rộng và bảo trì

Kiến trúc phân lớp và module hóa theo miền nghiệp vụ cho phép bổ sung dịch vụ hoặc miền nghiệp vụ mới mà không phá vỡ cấu trúc hiện có. Tầng trừu tượng hóa nhà cung cấp mô hình ngôn ngữ lớn tại AI Service cho phép đổi hoặc bổ sung nhà cung cấp mới mà không cần sửa đổi logic nghiệp vụ ở các tầng phía trên. Việc tổ chức đa-repository, mỗi dịch vụ một repository riêng, giúp giảm xung đột mã nguồn giữa các nhóm nhỏ và cho phép mỗi dịch vụ được triển khai, phiên bản hóa độc lập với các dịch vụ còn lại.

3.7.3 Hướng mở rộng kiến trúc trong tương lai

Mã nguồn của AI Service hiện đã chuẩn bị sẵn một khung điều phối đa-agent (multi-agent orchestration), cho phép định tuyến yêu cầu tới nhiều tác nhân AI chuyên biệt theo từng miền nghiệp vụ khác nhau; tuy nhiên khung này chưa được kích hoạt trong luồng xử lý thực tế ở phiên bản hiện tại, mà chỉ mới sử dụng một tác nhân duy nhất với cơ chế phân loại ý định như đã mô tả ở Mục 3.4. Đây là nền tảng sẵn có để hệ thống có thể mở rộng thành kiến trúc đa-agent thực thụ khi bổ sung thêm các nghiệp vụ AI phức tạp hơn trong tương lai.

3.7.4 Giới hạn hiện tại

Nhìn từ góc độ kiến trúc, hệ thống còn một số giới hạn: một số thao tác tài chính tổng hợp (như phân bổ thu nhập vào nhiều hũ cùng lúc) chưa được bọc trong một transaction cơ sở dữ liệu tường minh, tiềm ẩn rủi ro không nhất quán nếu xảy ra lỗi giữa chừng (chi tiết tại Chương 4); hệ thống chưa có bộ benchmark định lượng chính thức, có thể tái lập cho các thành phần AI (chi tiết tại Chương 6); và hệ thống phụ thuộc vào các dịch vụ bên thứ ba nằm ngoài tầm kiểm soát trực tiếp (Firebase Cloud Messaging, nhà cung cấp mô hình ngôn ngữ lớn).

Chương 4

Phân tích và đặc tả chức năng nghiệp vụ

Chương này tập trung phân tích tổng quan các chức năng hệ thống, xác định các tác nhân tham gia tương tác, và đặc tả chi tiết các luồng chức năng nghiệp vụ cốt lõi của hệ thống hỗ trợ sinh viên Student360.

4.1 Tổng quan chức năng hệ thống

Như đã giới thiệu tại Chương 1, hệ thống Student360 gồm 4 phân hệ chức năng cốt lõi: Quản lý tài chính cá nhân, Trợ lý tài chính ảo AI, Quản lý học bổng trực tuyến và Quản lý học thuật/Bảng điểm. Các mục tiếp theo của chương này xác định các tác nhân tham gia, sơ đồ phân rã chức năng và đặc tả chi tiết nghiệp vụ của từng phân hệ.

4.2 Các tác nhân tham gia hệ thống

Hệ thống Student360 xác định 3 nhóm tác nhân (Actors) chính tham gia tương tác:

- **Sinh viên (Student):** Tác nhân sử dụng phân hệ ứng dụng di động (Mobile App) để ghi chép chi tiêu, tương tác với AI Assistant, đăng ký và theo

dõi học bổng, lập bảng điểm cá nhân.

- **Nhà tài trợ (Sponsor):** Tác nhân sử dụng phân hệ Webadmin để đăng tải và thiết lập tiêu chí học bổng, quản lý và duyệt hồ sơ ứng tuyển của sinh viên.
- **Quản trị viên (Admin):** Tác nhân sử dụng Webadmin để cấu hình danh mục trường, khoa, môn học (Master Data) và giám sát hoạt động hệ thống.

4.3 Sơ đồ phân rã chức năng hệ thống

Dưới đây là cấu trúc phân rã chức năng (Functional Decomposition) của hệ thống Student360 theo dạng hình cây phân cấp:

- **Student360 System**
 - **1. Phân hệ Quản lý tài chính (Mobile App)**
 - * 1.1 Cấu hình tỷ lệ 6 lọ tài chính
 - * 1.2 Ghi nhận giao dịch thủ công (Thu, Chi, Chuyển khoản)
 - * 1.3 Phân bổ thu nhập tự động
 - * 1.4 Thiết lập ngưỡng cảnh báo chi tiêu theo hũ
 - * 1.5 Lên lịch giao dịch định kỳ
 - **2. Phân hệ Trợ lý ảo AI (Mobile App)**
 - * 2.1 Trò chuyện thời gian thực qua stream SSE
 - * 2.2 Phân loại giao dịch tự động bằng LLM Classifier
 - * 2.3 Nhận cảnh báo chi tiêu bất thường
 - **3. Phân hệ Quản lý học bổng (Web & Mobile)**
 - * 3.1 Quản lý chương trình học bổng (Nhà tài trợ)
 - * 3.2 Ứng tuyển học bổng trực tuyến (Sinh viên)
 - * 3.3 Cơ chế gợi ý học bổng tự động (AI Engine)

- * 3.4 Xét duyệt hồ sơ ứng tuyển (Nhà tài trợ)

– 4. Phân hệ Học thuật và bảng điểm (Web & Mobile)

- * 4.1 Quản trị Master Data trường, khoa, môn học (Admin)
- * 4.2 Lập mục tiêu tín chỉ học tập (Sinh viên)
- * 4.3 Quản lý bảng điểm cá nhân (Sinh viên)
- * 4.4 Nhập bảng điểm hàng loạt từ file CSV/TXT (Sinh viên)

4.4 Đặc tả chức năng nghiệp vụ

Dưới đây là phần đặc tả chi tiết các chức năng nghiệp vụ cốt lõi của hệ thống Student360.

4.4.1 Quản lý tài chính cá nhân

Chức năng 1: Cấu hình tỷ lệ 6 lọ tài chính

- **Tên chức năng:** Cấu hình tỷ lệ phân bổ 6 lọ tài chính.
- **Tác nhân (Actor):** Sinh viên.
- **Mô tả:** Cho phép sinh viên tùy chỉnh tỷ lệ phần trăm phân bổ giữa các hũ tài chính (6 hũ mặc định theo mô hình 6 Jars, hoặc hũ tùy chỉnh do sinh viên tự thêm), làm cơ sở để hệ thống tự động chia khoản thu nhập mới ở Chức năng "Phân bổ thu nhập tự động" bên dưới.
- **Điều kiện trước (Preconditions):** Sinh viên đã đăng nhập thành công vào Mobile App.
- **Luồng chính (Basic Flow):**
 1. Sinh viên truy cập màn hình cấu hình hũ tài chính.
 2. Hệ thống hiển thị danh sách hũ hiện có kèm tỷ lệ phần trăm hiện tại và tổng tỷ lệ đang phân bổ.

3. Sinh viên điều chỉnh tỷ lệ phần trăm cho từng hũ, có thể thêm hũ tùy chỉnh mới (đặt tên, biểu tượng, màu sắc) hoặc gỡ bỏ hũ không còn sử dụng.
4. Sinh viên nhấn nút "Lưu".
5. Mobile App gửi toàn bộ danh sách phân bổ vừa chỉnh sửa lên Backend để lưu lại, mỗi phần tử gồm mã hũ hiện có (nếu là hũ đã tồn tại), tên, tỷ lệ phần trăm, biểu tượng và màu sắc.
6. Backend kiểm tra tổng tỷ lệ phần trăm của toàn bộ danh sách phải bằng đúng 100% (sai lệch cho phép không quá 0.01), sau đó cập nhật hoặc tạo mới từng hũ tương ứng trong bảng `money_jars`.
7. Backend phản hồi danh sách hũ đã được cập nhật.
8. Mobile App hiển thị lại danh sách hũ với tỷ lệ phân bổ mới.

- **Luồng ngoại lệ (Exception Flows):**

- *Ngoại lệ 1 - Tổng tỷ lệ khác 100%:* Backend từ chối toàn bộ yêu cầu và trả về lỗi 400 Bad Request. Mobile App hiển thị thông báo: "Tổng tỷ lệ phân bổ phải bằng 100%".
- *Ngoại lệ 2 - Cập nhật hũ không thuộc sở hữu:* Nếu mã hũ gửi lên không thuộc về sinh viên hiện tại, Backend trả về lỗi 404 Not Found.

- **Kết quả (Postconditions):** Tỷ lệ phân bổ mới được lưu vào cơ sở dữ liệu và áp dụng ngay cho các lần phân bổ thu nhập tự động tiếp theo.

Chức năng 2: Ghi nhận giao dịch thủ công

- **Tên chức năng:** Ghi nhận giao dịch tài chính thủ công (Thu, Chi, Chuyển khoản).
- **Tác nhân (Actor):** Sinh viên.
- **Mô tả:** Cho phép sinh viên tự nhập trực tiếp một giao dịch Thu nhập, Chi tiêu hoặc Chuyển khoản giữa các hũ mà không cần qua bước phân tích

của AI – khác với Chức năng "Phân loại giao dịch bằng AI" ở mục Trợ lý AI tài chính, vốn tự động trích xuất thông tin từ một câu mô tả bằng ngôn ngữ tự nhiên.

- **Điều kiện trước (Preconditions):** Sinh viên đã đăng nhập và có ít nhất một hũ tài chính.

- **Luồng chính (Basic Flow):**

1. Sinh viên chọn loại giao dịch (Thu nhập, Chi tiêu hoặc Chuyển khoản) trên màn hình tương ứng.
2. Sinh viên nhập số tiền, chọn hũ nguồn; riêng với Chuyển khoản, sinh viên chọn thêm hũ đích. Sinh viên có thể chọn nhãn chi tiêu (mặc định là nhãn "Khác" nếu bỏ trống), nhập ngày giao dịch và mô tả/ghi chú.
3. Sinh viên nhấn nút "Lưu giao dịch".
4. Mobile App gửi yêu cầu tạo giao dịch mới kèm toàn bộ dữ liệu đã nhập lên Backend.
5. Backend xác thực dữ liệu (số tiền phải lớn hơn 0, các hũ tham chiếu phải thuộc về sinh viên hiện tại), tạo bản ghi mới trong bảng `financial_transactions`.
6. Backend phản hồi thông tin giao dịch vừa tạo về Mobile App.
7. Mobile App cập nhật danh sách lịch sử giao dịch và số dư hũ liên quan theo cơ chế tính động đã mô tả ở Chức năng "Phân bổ thu nhập tự động" bên dưới.

- **Luồng ngoại lệ (Exception Flows):**

- *Ngoại lệ 1 - Số tiền không hợp lệ:* Sinh viên nhập số tiền bằng 0 hoặc số âm, Backend từ chối yêu cầu và trả về lỗi 400 Bad Request.
- *Ngoại lệ 2 - Hũ không thuộc sở hữu:* Hũ nguồn hoặc hũ đích không thuộc về sinh viên hiện tại, Backend trả về lỗi 404 Not Found.

- **Kết quả (Postconditions):** Giao dịch mới được lưu vào cơ sở dữ liệu và số dư các hũ liên quan được cập nhật ngay lập tức. Thao tác sửa và xóa giao dịch (vuốt trái trong màn hình lịch sử) được đánh giá riêng tại Chương 6.

Chức năng 3: Phân bổ thu nhập tự động

- **Tên chức năng:** Phân bổ thu nhập tự động theo mô hình 6 lọ.
- **Tác nhân (Actor):** Sinh viên.
- **Mô tả:** Cho phép sinh viên ghi nhận một khoản thu mới và tự động tính toán, phân chia số tiền đó vào 6 hũ tài chính theo tỷ lệ phần trăm đã được cấu hình từ trước trên hệ thống.
- **Điều kiện trước (Preconditions):** Sinh viên đã đăng nhập thành công vào Mobile App và đã thiết lập cấu hình tỷ lệ phần trăm cho 6 hũ tài chính (đảm bảo tổng tỷ lệ bằng 100%).
- **Luồng chính (Basic Flow):**
 1. Sinh viên nhấn nút "Thêm thu nhập" tại màn hình Tổng quan ví.
 2. Hệ thống hiển thị biểu mẫu nhập thông tin gồm: Số tiền thu nhập, nguồn thu nhập, mô tả chi tiết, và tùy chọn phân bổ.
 3. Sinh viên nhập số tiền (ví dụ: 10.000.000 VNĐ), chọn danh mục nguồn thu (ví dụ: "Lương làm thêm"), viết mô tả và bật tùy chọn "Tự động phân bổ".
 4. Sinh viên nhấn nút "Xác nhận".
 5. Mobile App gửi yêu cầu phân bổ thu nhập kèm thông tin số tiền và mô tả lên Backend.
 6. Backend nhận yêu cầu, truy vấn bảng `money_jars` để lấy cấu hình tỷ lệ phần trăm của sinh viên hiện tại (ví dụ: 55%, 10%, 10%, 10%, 10%, 5%).

7. Backend tính toán số tiền phân bổ cụ thể cho từng hũ tài chính, đồng thời kiểm tra tổng số tiền đã phân bổ cho các hũ phải khớp chính xác với số tiền thu nhập ban đầu.
8. Backend thực hiện đồng loạt (`Promise.all`) 6 lệnh ghi bản ghi giao dịch thu vào bảng `financial_transactions`, mỗi lệnh cập nhật số dư của một hũ tương ứng. Đây hiện là các lệnh ghi độc lập chứ chưa được bọc trong một Database Transaction chung – một điểm cần bổ sung để đảm bảo an toàn tuyệt đối nếu có lỗi xảy ra giữa chừng.
9. Sau khi toàn bộ 6 lệnh ghi hoàn thành, Backend phản hồi kết quả về Mobile App.
10. Mobile App cập nhật trạng thái dữ liệu (qua React Query) và hiển thị thông báo phân bổ tiền thành công kèm chi tiết số tiền từng lọ nhận được.

- **Luồng ngoại lệ (Exception Flows):**

- *Ngoại lệ 1 - Số tiền không hợp lệ:* Tại bước 3, sinh viên nhập số tiền bằng 0 hoặc số âm. Hệ thống hiển thị lỗi ngay tại giao diện di động: "Số tiền thu nhập phải lớn hơn 0" và không cho phép gửi yêu cầu.
- *Ngoại lệ 2 - Tổng số tiền phân bổ không khớp:* Tại bước 6-7, nếu client tự gửi kèm danh sách phân bổ chi tiết theo từng hũ (`jarAllocations`) mà tổng số tiền của danh sách này không bằng đúng số tiền thu nhập ban đầu, Backend hủy tiến trình, trả về lỗi. Mobile App hiển thị thông báo: "Số tiền phân bổ không khớp với số tiền thu nhập, vui lòng kiểm tra lại". *Lưu ý:* hiện tại Backend chưa kiểm tra độc lập việc tổng tỷ lệ phần trăm (percentage) của 6 hũ có bằng 100% hay không trước khi phân bổ theo tỷ lệ mặc định – nếu dữ liệu tỷ lệ hũ của người dùng bị lệch 100% do thao tác trước đó, hệ thống vẫn phân bổ tiền theo đúng tỷ lệ đang lưu mà không cảnh báo. Đây là một khoảng trống cần bổ sung validation trong các phiên bản kế tiếp.

– *Ngoại lệ 3 - Lỗi kết nối cơ sở dữ liệu giữa chừng*: Tại bước 8, nếu một trong 6 lệnh ghi thất bại do mất kết nối cơ sở dữ liệu, các bản ghi giao dịch đã ghi thành công trước đó (nếu có) sẽ không được rollback tự động do chưa dùng chung một Database Transaction. Backend trả về lỗi HTTP 500 Internal Server Error, Mobile App hiển thị thông báo "Hệ thống bận, vui lòng thử lại sau" và khuyến nghị người dùng kiểm tra lại lịch sử giao dịch trước khi thực hiện lại thao tác.

- **Kết quả (Postconditions)**: Số dư khả dụng của từng hũ được cập nhật tăng tương ứng. 6 bản ghi giao dịch thu nhập mới được lưu vào cơ sở dữ liệu PostgreSQL. Hệ thống đồng bộ dữ liệu số dư mới lên màn hình Tổng quan ví.
- **Cơ chế đảm bảo tính toàn vẹn số dư hũ**: Số dư hiển thị của mỗi hũ không được lưu trữ tĩnh mà luôn được tính lại từ lịch sử giao dịch thực tế theo công thức tích lũy:

$$\text{Số dư hũ} = \sum(\text{Thu vào}) - \sum(\text{Chi ra}) \pm \sum(\text{Chuyển quỹ})$$

Cách tính này giúp tránh sai lệch số liệu do lỗi bất đồng bộ mạng hoặc lỗi phần mềm. Một trường hợp đặc biệt liên quan là khi sinh viên xóa một hũ tài chính tùy chỉnh đang có số dư khác 0: hệ thống bắt buộc chọn một hũ đích để nhận lại tiền, sau đó tạo đồng thời một cặp giao dịch bù trừ (trừ tiền ở hũ nguồn dưới dạng giao dịch expense, cộng tiền vào hũ đích dưới dạng giao dịch income) để tổng tài sản của người dùng không đổi, rồi tính lại số dư của cả hai hũ liên quan. Hiện tại thao tác này được thực hiện bằng một lệnh ghi đồng thời cả hai bản ghi giao dịch, chưa được bọc trong một QueryRunner Transaction tường minh như cơ chế chuyển khoản thông thường – tương tự hạn chế đã nêu ở Ngoại lệ 3 của chức năng này, đây là một điểm cần bổ sung để đảm bảo an toàn tuyệt đối nếu có lỗi xảy ra giữa chừng.

Chức năng 4: Thiết lập ngưỡng cảnh báo chi tiêu theo hũ

- **Tên chức năng:** Thiết lập ngưỡng cảnh báo chi tiêu cho từng hũ tài chính.
- **Tác nhân (Actor):** Sinh viên.
- **Mô tả:** Cho phép sinh viên cấu hình một ngưỡng cảnh báo cho từng hũ tài chính theo một trong hai chế độ độc lập: (a) cảnh báo khi số dư hũ giảm xuống dưới một số tiền cố định, hoặc (b) cảnh báo khi tỷ lệ đã chi vượt quá một phần trăm nhất định. Khác với mô hình "hạn mức ngân sách theo tháng" ban đầu, cấu hình này không giới hạn theo một tháng/năm cụ thể mà áp dụng liên tục cho tới khi sinh viên thay đổi lại.
- **Điều kiện trước (Preconditions):** Sinh viên đã đăng nhập thành công vào Mobile App và đã có ít nhất một hũ tài chính.
- **Luồng chính (Basic Flow):**
 1. Sinh viên truy cập vào màn hình "Chi tiết hũ" của một hũ bất kỳ (ví dụ: hũ Hưởng thụ - enjoyment).
 2. Sinh viên nhấn chọn "Cài đặt cảnh báo chi tiêu".
 3. Hệ thống hiển thị giao diện với hai lựa chọn độc lập có thể bật/tắt riêng: "Cảnh báo theo số dư tối thiểu" (nhập số tiền, ví dụ 200.000 VNĐ) và "Cảnh báo theo phần trăm đã chi" (nhập số nguyên từ 0-100, ví dụ 80%).
 4. Sinh viên bật một hoặc cả hai lựa chọn, nhập giá trị tương ứng và nhấn "Lưu".
 5. Mobile App gửi yêu cầu lưu cấu hình cảnh báo vừa thiết lập lên Backend.
 6. Backend xác thực giá trị đầu vào (phần trăm phải là số nguyên trong khoảng 0-100, số tiền phải không âm), sau đó tạo mới hoặc cập nhật bản ghi duy nhất tương ứng với hũ trong bảng `jar_notification_settings`.
 7. Backend phản hồi thành công về Mobile App.

- **Luồng ngoại lệ (Exception Flows):**

- *Ngoại lệ 1 - Giá trị nhập không hợp lệ:* Nếu sinh viên nhập phần trăm ngoài khoảng 0-100 hoặc số tiền âm, Backend từ chối yêu cầu ở tầng validate DTO và trả về lỗi 400 Bad Request. Mobile App hiển thị thông báo lỗi tương ứng ngay tại form nhập liệu.
- *Lưu ý về giới hạn hiện tại:* API cập nhật ngưỡng cảnh báo hiện chưa trả về kèm dữ liệu "đã chi bao nhiêu / ngưỡng bao nhiêu" đã tính sẵn, nên Mobile App chưa thể hiển thị trực tiếp thanh tiến trình (progress bar) chỉ từ phản hồi của API này mà cần tự tính toán thêm từ dữ liệu giao dịch. Đồng thời, nhánh cảnh báo theo phần trăm hiện dựa trên trường spentAmount/amount của bảng budgets (bảng nhân chi tiêu theo hũ, được tái sử dụng từ thiết kế hạn mức ngân sách ban đầu) — trường này chưa được các luồng nghiệp vụ hiện hành cập nhật chủ động, nên trong thực tế nhánh cảnh báo theo phần trăm có thể chưa hoạt động ổn định như nhánh cảnh báo theo số dư tối thiểu.

- **Kết quả (Postconditions):** Bản ghi ngưỡng cảnh báo được cập nhật hoặc tạo mới trong bảng jar_notification_settings. Từ giao dịch chi tiêu tiếp theo trên hũ đó, hệ thống sẽ kiểm tra ngưỡng này ngay trong luồng tạo giao dịch (đồng bộ, không cần chờ job quét định kỳ) và gửi cảnh báo qua hàng đợi thông báo đẩy nếu vượt ngưỡng.

- **Cơ chế Idempotency cho kênh cảnh báo bất thường tự động:** Bên cạnh ngưỡng cảnh báo do sinh viên tự cấu hình ở trên, hệ thống còn một kênh cảnh báo riêng do Backend tự động phát hiện: một Cron Job chạy định kỳ lúc 23:00 hằng ngày quét các giao dịch bất thường trong ngày. Vì job này có thể chạy lại hoặc xử lý trùng, hệ thống áp dụng cơ chế loại bỏ trùng lặp (Deduplication) dựa trên khóa Idempotency tính từ tổ hợp user_id + target_id (target_id là mã giao dịch đang được đánh giá): trước khi thêm một cảnh báo mới vào bảng ai_anomaly_alerts và kích hoạt gửi thông báo đẩy, hệ thống kiểm tra đã tồn tại cảnh báo nào cùng khóa

này chưa, nếu có thì bỏ qua không tạo mới. *Giới hạn hiện tại:* vì khóa chỉ dựa trên từng giao dịch riêng lẻ, cơ chế này chưa gộp được kịch bản "cùng một hũ liên tục vượt hạn mức trong tháng" thành một cảnh báo duy nhất – mỗi giao dịch bất thường vẫn có thể sinh ra một cảnh báo (và một thông báo đẩy) riêng.

Chức năng 5: Lên lịch giao dịch định kỳ

- **Tên chức năng:** Lên lịch giao dịch tài chính định kỳ tự động.
- **Tác nhân (Actor):** Sinh viên.
- **Mô tả:** Cho phép sinh viên thiết lập một lịch giao dịch lặp lại tự động (ví dụ: thu nhập định kỳ như lương làm thêm, hoặc chi tiêu định kỳ như tiền trọ) theo tần suất hằng ngày, hằng tuần hoặc hằng tháng. Hệ thống tự động tạo giao dịch đúng thời điểm đã hẹn mà không cần sinh viên thao tác thủ công mỗi lần.
- **Điều kiện trước (Preconditions):** Sinh viên đã đăng nhập và có ít nhất một hũ tài chính.
- **Luồng chính (Basic Flow):**
 1. Sinh viên truy cập màn hình "Lịch giao dịch tự động" và nhấn "Thêm lịch mới".
 2. Hệ thống hiển thị biểu mẫu gồm: tên lịch, số tiền, loại giao dịch (thu nhập/chi tiêu), tần suất lặp lại (hằng ngày; hằng tuần kèm chọn thứ; hoặc hằng tháng kèm chọn ngày trong tháng), giờ thực thi, cách phân bổ (tự động theo tỷ lệ các hũ, chỉ định một hũ cụ thể, hoặc tùy chỉnh tỷ lệ theo từng hũ), tùy chọn nhắc trước/nhắc sau khi thực thi, và ngày kết thúc lịch (không bắt buộc).
 3. Sinh viên điền thông tin, ví dụ: "Lương thực tập", 3.000.000 VNĐ, loại Thu nhập, lặp lại hằng tháng vào ngày 5, phân bổ tự động theo tỷ lệ hũ, bật nhắc trước 1 ngày.

4. Sinh viên nhấn nút "Lưu lịch".
5. Mobile App gửi yêu cầu tạo lịch giao dịch mới kèm toàn bộ cấu hình đã nhập lên Backend.
6. Backend xác thực dữ liệu, lưu bản ghi lịch vào bảng `auto_transfer_scheduled` đồng thời đưa một Job trì hoãn (delayed job) vào hàng đợi BullMQ ứng với đúng thời điểm thực thi kế tiếp.
7. Đến đúng thời điểm đã lên lịch, Worker xử lý Job: tạo giao dịch tương ứng (phân bổ theo tỷ lệ các hũ nếu là thu nhập tự động phân bổ, hoặc tạo một giao dịch đơn vào hũ chỉ định), sau đó tự động đặt lại Job kế tiếp cho chu kỳ lặp tiếp theo.
8. Nếu sinh viên bật nhắc trước/nhắc sau, Worker gửi thêm thông báo đẩy tương ứng qua kênh thông báo hiện có.
9. Sinh viên có thể quay lại màn hình danh sách lịch để bật/tắt tạm thời, sửa hoặc xóa một lịch bất kỳ, hoặc kích hoạt thủ công ngay lập tức mà không cần chờ tới chu kỳ kế tiếp.

- **Luồng ngoại lệ (Exception Flows):**

- *Ngoại lệ 1 - Lịch bị xóa trong lúc Job đang chờ thực thi:* Worker phát hiện lịch không còn tồn tại khi Job tới hạn xử lý, sẽ bỏ qua không tạo giao dịch và không thử lại.
- *Ngoại lệ 2 - Cấu hình phân bổ tùy chỉnh không hợp lệ:* Nếu sinh viên chọn phân bổ tùy chỉnh theo tỷ lệ nhưng dữ liệu tỷ lệ từng hũ không hợp lệ, Backend từ chối yêu cầu tạo lịch và trả về lỗi 400 Bad Request.

- **Kết quả (Postconditions):** Lịch giao dịch định kỳ được lưu và kích hoạt; từ thời điểm lưu trở đi, hệ thống tự động tạo giao dịch đúng chu kỳ đã cấu hình mà không cần thao tác thủ công.

4.4.2 Trợ lý AI tài chính

Chức năng 1: Trò chuyện cùng Trợ lý AI

- **Tên chức năng:** Trò chuyện và tư vấn cùng Trợ lý tài chính ảo.
- **Tác nhân (Actor):** Sinh viên.
- **Mô tả:** Sinh viên đặt câu hỏi bằng ngôn ngữ tự nhiên về tình hình tài chính cá nhân hoặc tiêu chí học bổng. Trợ lý AI sẽ phân tích, gọi các công cụ truy vấn dữ liệu thực tế và trả lời sinh viên dưới dạng dòng dữ liệu truyền liên tục (stream SSE).
- **Điều kiện trước (Preconditions):** Sinh viên đã đăng nhập và thiết bị di động có kết nối internet hoạt động ổn định.
- **Luồng chính (Basic Flow):**
 1. Sinh viên truy cập màn hình "Trợ lý AI" và nhập câu hỏi (ví dụ: "Tháng này mình đã tiêu bao nhiêu tiền ăn từ hũ tiết kiệm?").
 2. Sinh viên nhấn nút gửi câu hỏi.
 3. Mobile App mở một kết nối truyền dữ liệu liên tục (Server-Sent Events - SSE) tới Backend NestJS để nhận phản hồi dạng luồng, khác với chế độ hỏi-đáp thông thường trả về trọn vẹn câu trả lời một lần.
 4. Backend NestJS xác thực token người dùng, sau đó chuyển tiếp yêu cầu sang FastAPI AI Service.
 5. FastAPI AI Service khởi chạy LangChain Agent, gửi câu hỏi kèm lịch sử trò chuyện ngắn và mã định danh sinh viên.
 6. Agent phân tích ý định câu hỏi và nhận diện cần dữ liệu chi tiêu thực tế. Agent quyết định gọi tool `search_transactions` (hoặc `get_recent_transactions` nếu chỉ cần các giao dịch gần nhất)

với tham số truy vấn là tháng hiện tại và danh mục "ăn uống/thực phẩm".

7. Agent gửi yêu cầu truy vấn ngược lại Backend NestJS để lấy dữ liệu giao dịch từ PostgreSQL.
8. Backend NestJS trả về danh sách giao dịch ăn uống của sinh viên trong tháng.
9. Agent nhận dữ liệu, tính toán tổng tiền, soạn thảo văn bản nhận định tài chính và gửi từng token chữ được sinh ra từ Gemini API về FastAPI AI Service.
10. FastAPI AI Service lập tức truyền các token chữ này về Backend NestJS dưới dạng stream SSE, Backend NestJS tiếp tục đẩy stream này về ứng dụng di động của sinh viên.
11. Mobile App nhận từng token chữ và hiển thị trực tiếp lên màn hình dưới dạng chữ chạy thời gian thực.

- **Luồng ngoại lệ (Exception Flows):**

- *Ngoại lệ 1 - Gọi API Gemini bị lỗi hoặc hết hạn mức (Quota):* Tại bước 9, Gemini API trả về lỗi giới hạn lượt gọi hoặc quá tải. FastAPI AI Service sẽ bắt ngoại lệ này, sinh một câu trả lời mặc định thân thiện: "Xin lỗi bạn, kết nối tới máy chủ trí tuệ nhân tạo hiện tại đang gián đoạn. Vui lòng thử lại sau giây lát!" và kết thúc stream.

- **Kết quả (Postconditions):** Phản hồi từ AI được truyền tải và hiển thị đầy đủ lên màn hình trò chuyện của sinh viên.

Chức năng 2: Phân loại giao dịch bằng AI

- **Tên chức năng:** Phân loại giao dịch chi tiêu tự động bằng AI.
- **Tác nhân (Actor):** Sinh viên.

- **Mô tả:** Sinh viên chỉ cần gõ một câu mô tả ngắn gọn về khoản thu chi bằng ngôn ngữ tự nhiên, AI sẽ tự động phân tích để trích xuất số tiền, phân loại hũ tài chính phù hợp và gợi ý danh mục chi tiết để sinh viên xác nhận lưu nhanh.
- **Điều kiện trước (Preconditions):** Sinh viên đã đăng nhập và đang mở biểu mẫu thêm giao dịch nhanh.
- **Luồng chính (Basic Flow):**
 1. Sinh viên mở ô nhập nhanh giao dịch và gõ câu lệnh: "mua sách 150k bang tien mat".
 2. Sinh viên nhấn nút "Phân tích bằng AI".
 3. Mobile App gửi câu mô tả văn bản thô lên Backend để yêu cầu phân loại bằng AI.
 4. Backend NestJS chuyển tiếp payload tới FastAPI AI Service.
 5. FastAPI sử dụng Few-shot Prompting kết hợp mô hình Gemini để phân tích ngữ nghĩa câu thô.
 6. AI nhận diện: hành động là "chi tiêu" (expense), số tiền là 150.000, hũ tài chính phù hợp là hũ "Giáo dục" (education), danh mục đề xuất là "Sách vở/Học cụ".
 7. FastAPI trả về cấu trúc dữ liệu JSON chuẩn hóa chứa các trường thông tin đã trích xuất về Backend NestJS, Backend phản hồi lại cho Mobile App.
 8. Mobile App tự động điền giá trị 150.000 vào ô Số tiền, chọn hũ Giáo dục và chọn danh mục Sách vở, đồng thời hiển thị nút "Lưu giao dịch".
 9. Sinh viên kiểm tra thấy thông tin chính xác và nhấn "Xác nhận".
 10. Hệ thống lưu giao dịch vào cơ sở dữ liệu PostgreSQL.
- **Luồng ngoại lệ (Exception Flows):**

- *Luồng phụ - Người dùng điều chỉnh lại gợi ý sai của AI:* Tại bước 8, nếu AI phân loại sai hũ (ví dụ gợi ý hũ Hưởng thụ thay vì Nhu cầu thiết yếu), sinh viên có thể tự tay chọn lại hũ Nhu cầu thiết yếu trên giao diện. Khi nhấn "Xác nhận", Mobile App sẽ gửi thêm một yêu cầu phụ để ghi nhận lại lỗi phân loại này, giúp AI tự cập nhật hành vi cho các lần phân tích sau.
- **Kết quả (Postconditions):** Các trường thông tin được điền tự động và giao dịch được tạo chính xác trong DB.

4.4.3 Quản lý học bổng

Chức năng 1: Nhà tài trợ quản lý chương trình học bổng

- **Tên chức năng:** Tạo, chỉnh sửa và gỡ bỏ chương trình học bổng.
- **Tác nhân (Actor):** Nhà tài trợ.
- **Mô tả:** Cho phép nhà tài trợ tạo mới, chỉnh sửa hoặc gỡ bỏ một chương trình học bổng trên hệ thống, bao gồm thông tin cơ bản (tên, mô tả, đơn vị tài trợ, hình ảnh), giá trị học bổng (số tiền, số lượng suất), tiêu chí xét duyệt (GPA tối thiểu, ngành/trường áp dụng), thời hạn nộp hồ sơ, cùng danh sách tài liệu bắt buộc phải nộp kèm khi sinh viên ứng tuyển.
- **Điều kiện trước (Preconditions):** Nhà tài trợ đã đăng nhập thành công vào Webadmin Portal với vai trò Sponsor.
- **Luồng chính (Basic Flow):**
 1. Nhà tài trợ truy cập màn hình "Học bổng của tôi" trên Webadmin và nhấn "Tạo học bổng mới".
 2. Hệ thống hiển thị biểu mẫu nhiều phần: thông tin chung, giá trị học bổng, tiêu chí xét duyệt, mốc thời gian (ngày mở đơn, hạn nộp hồ sơ, ngày công bố kết quả) và thông tin liên hệ.

3. Nhà tài trợ điền thông tin và nhấn "Lưu". Webadmin gửi toàn bộ dữ liệu đã nhập lên Backend để tạo bản ghi học bổng mới.
4. Nhà tài trợ tiếp tục thêm danh sách tài liệu bắt buộc (ví dụ: Bảng điểm, Minh chứng hoàn cảnh gia đình) và các điều kiện xét duyệt chi tiết cho học bổng vừa tạo, mỗi mục được quản lý riêng biệt cho yêu cầu xét duyệt và tài liệu đính kèm của học bổng đó.
5. Nhà tài trợ kích hoạt học bổng (chuyển trạng thái sang công khai); từ thời điểm này học bổng mới hiển thị cho sinh viên trong danh sách để ứng tuyển ở Chức năng 2 bên dưới.
6. Đối với chỉnh sửa, nhà tài trợ mở lại học bổng đã tạo, cập nhật thông tin và Webadmin gửi yêu cầu cập nhật lên Backend; đối với gỡ bỏ, nhà tài trợ nhấn "Xóa" và Webadmin gửi yêu cầu xóa học bổng đó lên Backend.

- **Luồng ngoại lệ (Exception Flows):**

- *Ngoại lệ 1 - Thiếu thông tin bắt buộc:* Nếu tên học bổng để trống, Backend trả về lỗi 400 Bad Request, Webadmin hiển thị lỗi ngay tại biểu mẫu nhập liệu.
- *Lưu ý về giới hạn hiện tại:* Các thao tác tạo, sửa, xóa học bổng ở tầng API hiện chưa được gắn Guard xác thực vai trò – việc giới hạn chỉ nhà tài trợ mới thao tác được các chức năng này hiện chỉ đảm bảo ở tầng giao diện Webadmin (kiểm tra vai trò khi định tuyến trang), chưa được kiểm soát lại ở tầng Backend. Đây là một điểm cần bổ sung trong các phiên bản kế tiếp để tránh rủi ro nếu request được gọi trực tiếp ngoài giao diện.

- **Kết quả (Postconditions):** Bản ghi chương trình học bổng được tạo mới, cập nhật hoặc xóa tương ứng trong cơ sở dữ liệu; nếu ở trạng thái kích hoạt, học bổng xuất hiện trong danh sách mà sinh viên có thể xem và ứng tuyển.

Thuật toán tính điểm số so khớp tương thích

Điểm số so khớp tương thích giữa hồ sơ sinh viên và một chương trình học bổng (nhắc tới ở Chức năng 2 bên dưới) được AI Service tính trực tiếp trên bộ nhớ RAM tại thời điểm truy vấn, không phụ thuộc cơ sở dữ liệu vector. Hệ thống gộp các trường thông tin có giá trị trong hồ sơ sinh viên (ngành học, trường, kỹ năng, hoạt động ngoại khóa) thành một đoạn văn bản, chuẩn hóa Unicode để đồng nhất cách hiển thị tiếng Việt trên các hệ điều hành khác nhau, rồi so sánh với tiêu đề và mô tả của học bổng bằng ba độ đo tương đồng văn bản kinh điển:

- **Độ tương đồng Jaccard theo token** [7] (trọng số 0.5): so sánh tập từ vựng xuất hiện trong hồ sơ và trong học bổng, điểm càng cao khi hai văn bản dùng chung nhiều từ khóa.
- **Độ tương đồng Jaccard theo Bigram ký tự** [8] (trọng số 0.3): so sánh theo từng cặp ký tự liên kề thay vì theo từ, giúp nhận diện các trường hợp viết tắt hoặc sai chính tả mà so khớp theo từ không phát hiện được.
- **Tỷ lệ khớp chuỗi** bằng mô-đun `difflib.SequenceMatcher` của Python (trọng số 0.2): đo mức độ giống nhau tổng thể giữa hai chuỗi văn bản.

Ba độ đo trên được kết hợp theo trọng số cố định (0.5/0.3/0.2) thành một điểm số tổng hợp duy nhất trong khoảng từ 0 đến 1, hiển thị cho sinh viên dưới dạng phần trăm khớp (ví dụ: Khớp 85%) ở Chức năng 2 bên dưới. Nếu hồ sơ sinh viên hoặc mô tả học bổng không có đủ dữ liệu văn bản để so sánh, hệ thống trả về điểm khớp bằng 0.

Chức năng 2: Ứng tuyển học bổng trực tuyến

- **Tên chức năng:** Sinh viên ứng tuyển học bổng trực tuyến.
- **Tác nhân (Actor):** Sinh viên.
- **Mô tả:** Sinh viên duyệt danh sách học bổng, xem mức độ tương thích hồ sơ do AI gợi ý, điền thông tin đăng ký và tải lên các tài liệu minh chứng để gửi hồ sơ ứng tuyển tới nhà tài trợ.

- **Điều kiện trước (Preconditions):** Sinh viên đã tạo hồ sơ học thuật cá nhân. Chương trình học bổng vẫn còn trong thời gian nhận hồ sơ.
- **Luồng chính (Basic Flow):**
 1. Sinh viên truy cập màn hình "Học bổng" trên ứng dụng di động.
 2. Hệ thống hiển thị danh sách các học bổng đang mở kèm điểm số so khớp tương thích của AI (ví dụ: Khớp 85%).
 3. Sinh viên nhấn chọn một học bổng cụ thể để xem chi tiết tiêu chí và danh sách các tài liệu bắt buộc phải nộp.
 4. Sinh viên nhấn nút "Ứng tuyển ngay".
 5. Hệ thống hiển thị biểu mẫu đăng ký bao gồm các thông tin cá nhân lấy từ profile và các nút upload file tương ứng với từng tài liệu yêu cầu.
 6. Sinh viên điền thông tin bổ sung, tải lên các file tài liệu dạng PDF/Image từ điện thoại. Việc upload file thực tế do Mobile App tự thực hiện trực tiếp lên dịch vụ lưu trữ file trước, sau đó chỉ gửi đường dẫn (fileUrl) đã có sẵn cho Backend – Backend không đóng vai trò trung chuyển nội dung file mà chỉ nhận và lưu URL.
 7. Sinh viên nhấn nút "Nộp hồ sơ".
 8. Mobile App gửi request lên Backend NestJS chứa thông tin đăng ký và danh sách URL file đã upload trước đó.
 9. Backend xác thực tính hợp lệ của thông tin đăng ký và định dạng các URL tài liệu được cung cấp.
 10. Backend lưu thông tin hồ sơ ứng tuyển vào bảng student_scholarships, lưu các URL file đính kèm vào bảng student_scholarship_documents, và lưu lịch sử chuyển trạng thái sang submitted vào bảng student_scholarships.
 11. Mobile App chuyển giao diện về trạng thái "Đã nộp" và hiển thị thông báo ứng tuyển thành công.

- **Luồng ngoại lệ (Exception Flows):**

- *Luồng phụ - Lưu nháp hồ sơ:* Tại bước 7, sinh viên chưa chuẩn bị đủ tài liệu có thể nhấn nút "Lưu nháp". Hệ thống sẽ lưu hồ sơ với trạng thái "Bản nháp" (draft) để sinh viên có thể tiếp tục chỉnh sửa và nộp sau.
- *Ngoại lệ 1 - Nộp hồ sơ trùng lặp:* Tại bước 9, Backend kiểm tra và phát hiện sinh viên đã có một hồ sơ ứng tuyển cho chương trình học bổng này ở trạng thái không còn cho phép chỉnh sửa (đã submitted, under_review, approved, rejected hoặc awarded). Backend trả về lỗi HTTP 409 Conflict. Mobile App hiển thị thông báo: "Bạn đã nộp hồ sơ cho học bổng này từ trước, không thể nộp lại". (Nếu hồ sơ cũ đang ở trạng thái draft, need_modification hoặc cancelled, Backend sẽ cập nhật ngay trên hồ sơ đó thay vì báo lỗi trùng lặp.)
- *Ngoại lệ 2 - Dung lượng file vượt quá giới hạn:* Do việc upload file diễn ra trực tiếp từ Mobile App lên dịch vụ lưu trữ (không qua Backend), việc kiểm tra dung lượng file vượt giới hạn được thực hiện ngay tại bước tải lên trên thiết bị di động trước khi gửi yêu cầu nộp hồ sơ. Nếu vượt giới hạn, ứng dụng hiển thị thông báo: "Tài liệu vượt quá dung lượng cho phép. Vui lòng nén file và thử lại" và không cho phép tiếp tục.

- **Kết quả (Postconditions):** Một bản ghi hồ sơ ứng tuyển mới được tạo ở trạng thái "Đã nộp" (submitted) và được lưu thành công trên hệ thống.

Chức năng 3: Xét duyệt hồ sơ học bổng

- **Tên chức năng:** Nhà tài trợ xét duyệt hồ sơ ứng tuyển học bổng.
- **Tác nhân (Actor):** Nhà tài trợ (Sponsor).
- **Mô tả:** Nhà tài trợ xem danh sách các hồ sơ ứng tuyển của sinh viên gửi tới chương trình học bổng của tổ chức mình, xem chi tiết thông tin học

thuật, tải các file tài liệu đính kèm để đối chiếu và thay đổi trạng thái hồ sơ.

- **Điều kiện trước (Preconditions):** Nhà tài trợ đã được phân quyền quản trị của tổ chức tài trợ và đã đăng nhập thành công vào Webadmin Portal.

- **Luồng chính (Basic Flow):**

1. Nhà tài trợ truy cập vào màn hình "Quản lý hồ sơ ứng tuyển" trên Webadmin.
2. Hệ thống hiển thị danh sách các hồ sơ đang ở trạng thái chờ duyệt.
3. Nhà tài trợ nhấn chọn một hồ sơ cụ thể để xem chi tiết.
4. Hệ thống hiển thị toàn bộ thông tin học thuật của sinh viên kèm danh sách các file tài liệu minh chứng sinh viên đã nộp. Nhà tài trợ click mở các file đính kèm để đối soát.
5. Nhà tài trợ đưa ra quyết định xét duyệt bằng cách chọn một trong ba tùy chọn:
 - **Chấp nhận cấp học bổng:** Cập nhật trạng thái sang "Chấp nhận" (approved).
 - **Yêu cầu chỉnh sửa:** Cập nhật trạng thái sang "Cần chỉnh sửa" (need_modification) kèm nội dung ghi chú yêu cầu.
 - **Từ chối:** Cập nhật trạng thái sang "Từ chối" (rejected) kèm lý do từ chối.
6. Nhà tài trợ nhấn nút "Cập nhật".
7. Webadmin gửi yêu cầu cập nhật trạng thái hồ sơ kèm trạng thái mới và ghi chú duyệt lên Backend.
8. Backend thực thi một Transaction cập nhật trạng thái hồ sơ, ghi nhận lịch sử vào bảng `student_scholarship_history`.
9. Backend tạo một tiến trình Job gửi thông báo đẩy và đẩy vào hàng đợi BullMQ.

10. Worker ngầm của hệ thống lấy Job ra, gọi FCM SDK để gửi thông báo tới thiết bị di động của sinh viên tương ứng.
11. Webadmin cập nhật danh sách hồ sơ và hiển thị thông báo "Cập nhật trạng thái hồ sơ thành công".

- **Luồng ngoại lệ (Exception Flows):**

- *Ngoại lệ 1 - Quyền truy cập không hợp lệ:* Tại bước 7, Backend kiểm tra thấy nhà tài trợ của tổ chức A đang cố gắng cập nhật trạng thái hồ sơ nộp vào chương trình học bổng của tổ chức B. Backend từ chối yêu cầu, trả về lỗi HTTP 403 Forbidden. Webadmin hiển thị thông báo: "Bạn không có quyền thực hiện thao tác này".

- **Kết quả (Postconditions):** Trạng thái hồ sơ được cập nhật và sinh viên nhận được thông báo kết quả.

4.4.4 Quản lý dữ liệu học thuật

Chức năng 1: Quản trị viên quản lý Master Data trường, khoa/chương trình đào tạo và môn học

- **Tên chức năng:** Quản trị danh mục dữ liệu học thuật chuẩn hóa (Master Data).
- **Tác nhân (Actor):** Quản trị viên.
- **Mô tả:** Cho phép quản trị viên tạo, cập nhật, xóa và import hàng loạt danh mục trường đại học, khoa/chương trình đào tạo và môn học trên Webadmin. Danh mục này là nguồn dữ liệu chuẩn hóa duy nhất được cả Webadmin và Mobile App sử dụng để phục vụ đăng ký thông tin học thuật, nhập điểm và đối chiếu điều kiện xét duyệt học bổng.
- **Điều kiện trước (Preconditions):** Quản trị viên đã đăng nhập thành công vào Webadmin Portal với quyền quản trị hệ thống.

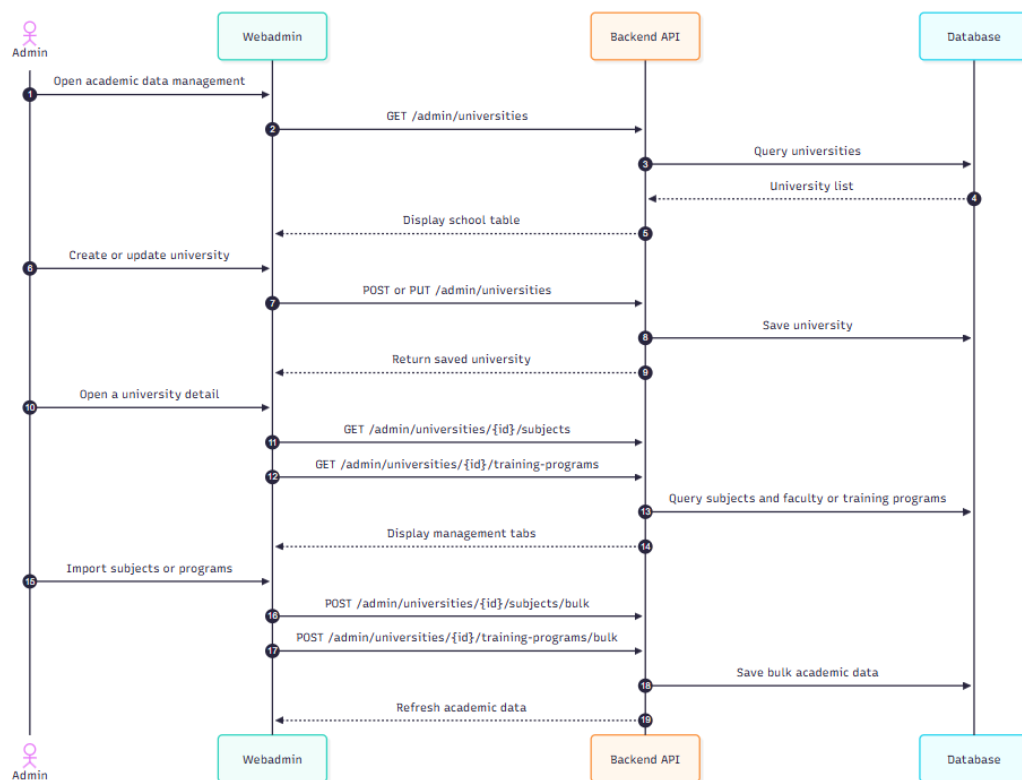
- **Luồng chính (Basic Flow):**

1. Quản trị viên quản lý danh mục trường: Webadmin cho phép xem danh sách, tạo mới, cập nhật và xóa từng trường đại học.
2. Quản trị viên quản lý khoa/chương trình đào tạo: với từng trường, Webadmin cho phép xem danh sách, thêm mới, cập nhật, xóa từng khoa/chương trình đào tạo, hoặc import hàng loạt từ file.
3. Quản trị viên quản lý môn học: với từng trường, Webadmin cho phép xem danh sách, thêm mới, cập nhật, xóa từng môn học, hoặc import hàng loạt từ file.
4. Dữ liệu vừa cập nhật lập tức khả dụng cho Mobile App: sinh viên có thể chọn trường, khoa/chương trình đào tạo và môn học từ danh mục mới nhất khi khai báo hồ sơ cá nhân hoặc nhập điểm.

- **Luồng ngoại lệ (Exception Flows):**

- *Ngoại lệ 1 - Import hàng loạt có bản ghi lỗi:* Khi import file dữ liệu hàng loạt, nếu một số dòng có dữ liệu không hợp lệ (thiếu trường bắt buộc, trùng mã), Backend trả về danh sách các dòng lỗi kèm lý do cụ thể thay vì từ chối toàn bộ file, cho phép quản trị viên sửa và import lại riêng các dòng lỗi.

- **Kết quả (Postconditions):** Danh mục Master Data được cập nhật và đồng bộ ngay cho cả Webadmin lẫn Mobile App.



Hình 4.1: Sơ đồ luồng quản trị dữ liệu học thuật trên Webadmin

Chức năng 2: Quản lý bảng điểm cá nhân

- **Tên chức năng:** Quản lý bảng điểm cá nhân và tự động tính toán GPA.
- **Tác nhân (Actor):** Sinh viên.
- **Mô tả:** Sinh viên tự cập nhật điểm số các môn học đã hoàn thành của mình trên ứng dụng di động. Hệ thống sẽ tự động tính toán điểm trung bình tích lũy GPA dựa trên số tín chỉ của từng môn học được tham chiếu từ danh mục môn học chuẩn hóa của trường.
- **Điều kiện trước (Preconditions):** Sinh viên đã đăng nhập và đã cấu hình thông tin trường đại học trong hồ sơ cá nhân.
- **Luồng chính (Basic Flow):**

1. Sinh viên truy cập màn hình "Góc học tập" và chọn tab "Bảng điểm".

2. Hệ thống hiển thị danh sách các môn học đã nhập điểm kèm số tín chỉ và điểm số cụ thể.
3. Sinh viên nhấn nút "Thêm điểm môn học".
4. Hệ thống hiển thị biểu mẫu bao gồm: Danh sách môn học chuẩn, Ô nhập điểm số.
5. Sinh viên gõ tìm kiếm và chọn môn học (ví dụ: "Nhập môn Lập trình - 4 tín chỉ"), nhập điểm số đạt được (ví dụ: 8.5 hoặc điểm chữ A) và nhấn "Lưu".
6. Mobile App gửi thông tin môn học và điểm số vừa nhập lên Backend để lưu lại.
7. Backend xác thực môn học thuộc về trường đại học sinh viên đang học, kiểm tra điểm số có nằm trong giới hạn thang điểm của môn học không.
8. Backend lưu bản ghi điểm môn học và phản hồi thành công về Mobile App. Điểm GPA tích lũy **không** được ghi đè trực tiếp vào hồ sơ học thuật của sinh viên ngay tại bước này.
9. Khi sinh viên mở màn hình Dashboard bảng điểm, Backend mới truy vấn toàn bộ các môn học có điểm của sinh viên, lấy số tín chỉ và điểm số của từng môn để tính GPA tích lũy có trọng số theo công thức sau, tính trực tiếp tại thời điểm truy vấn (không lưu lại giá trị này vào cơ sở dữ liệu):

$$GPA = \frac{\sum(\text{Điểm môn } i \times \text{Số tín chỉ môn } i)}{\sum(\text{Số tín chỉ})}$$

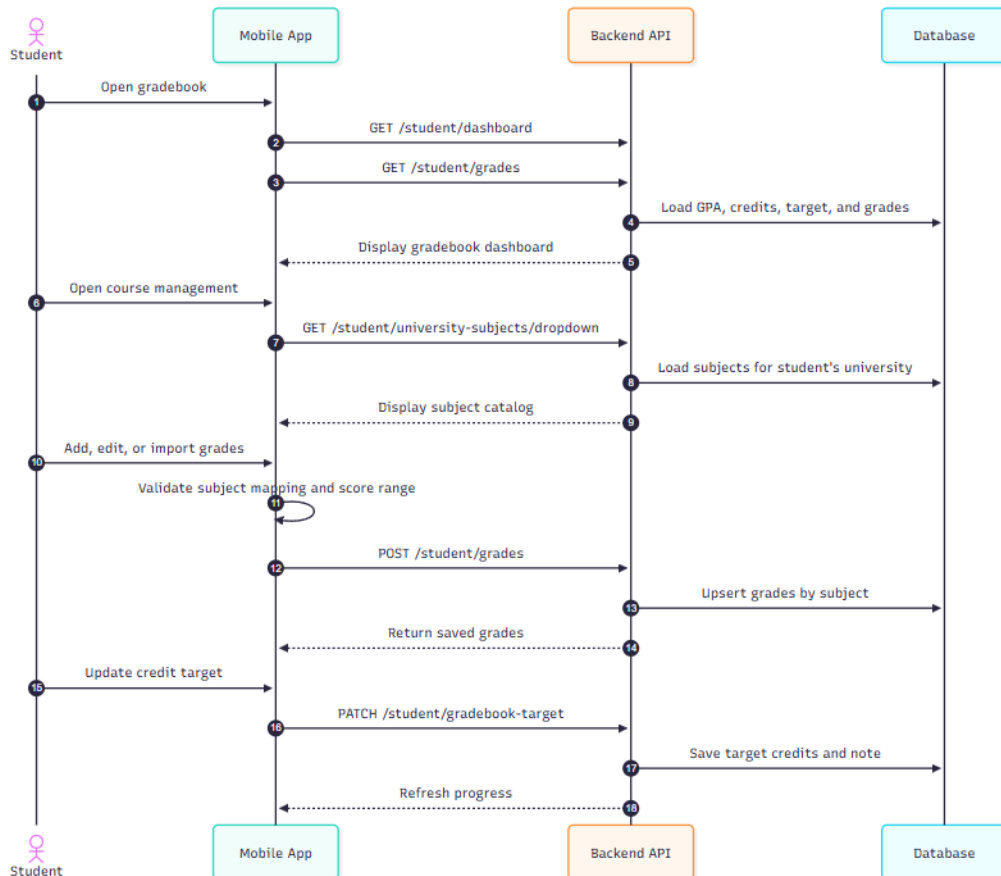
10. Mobile App hiển thị bảng điểm được cập nhật mới kèm chỉ số GPA tích lũy tính theo thời gian thực (ví dụ: GPA: 3.42 / 4.0).

- **Luồng ngoại lệ (Exception Flows):**

- *Ngoại lệ 1 - Điểm số không hợp lệ:* Tại bước 5, sinh viên nhập điểm

là 11 (vượt quá thang điểm 10) hoặc nhập số âm. Hệ thống hiển thị thông báo lỗi ngay trên giao diện nhập: "Điểm số nhập vào không hợp lệ. Vui lòng nhập điểm trong khoảng từ 0 đến 10".

- **Kết quả (Postconditions):** Bảng điểm và GPA tích lũy được cập nhật tức thời trên màn hình Dashboard học tập.



Hình 4.2: Sơ đồ luồng thao tác bảng điểm của sinh viên

Chức năng 3: Nhập điểm hàng loạt từ file

- **Tên chức năng:** Nhập điểm sinh viên hàng loạt từ file bảng điểm.
- **Tác nhân (Actor):** Sinh viên.
- **Mô tả:** Cho phép sinh viên tải lên một file văn bản (CSV hoặc TXT) chứa danh sách các môn học và điểm số kết xuất từ trang web đào tạo của

trường để hệ thống tự động phân tích và lưu hàng loạt môn học vào bảng điểm cá nhân.

- **Điều kiện trước (Preconditions):** Sinh viên có file bảng điểm đúng định dạng hỗ trợ. Thiết bị di động có quyền truy cập tệp tin.

- **Luồng chính (Basic Flow):**

1. Sinh viên truy cập màn hình "Bảng điểm" và chọn chức năng "Nhập điểm từ tệp tin".
2. Hệ thống hiển thị hộp thoại yêu cầu chọn tệp tin và hướng dẫn định dạng file mẫu.
3. Sinh viên nhấn chọn file từ bộ nhớ thiết bị.
4. Mobile App thực hiện đọc nội dung tệp tin cục bộ, sử dụng bộ Parser phân tích cú pháp từng dòng của tệp để trích xuất ra các cột thông tin: Mã môn học, Điểm số.
5. Hệ thống hiển thị danh sách các môn học và điểm số đã parse được lên màn hình xem trước để sinh viên rà soát lại.
6. Sinh viên kiểm tra thông tin và nhấn nút "Xác nhận lưu tất cả".
7. Mobile App gửi toàn bộ danh sách các cặp giá trị (Mã môn học, Điểm số) đã được parse lên Backend trong một lần gửi duy nhất, tái sử dụng cùng luồng lưu điểm với Chức năng 2 ở trên chứ không qua một luồng nhập hàng loạt tách biệt riêng.
8. Backend kiểm tra sự tồn tại của các mã môn học trong danh mục Master Data của trường đại học mà sinh viên đăng ký.
9. Backend lặp qua từng phần tử trong danh sách để xóa điểm cũ của những môn học trùng lặp (nếu có) và thêm mới các bản ghi điểm môn học.
10. Backend phản hồi kết quả thành công kèm số lượng môn học đã lưu. GPA tích lũy không được tính và lưu lại ở bước này, mà sẽ được tính

theo thời gian thực khi sinh viên mở lại Dashboard bảng điểm, theo đúng cơ chế đã mô tả ở Chức năng 2.

11. Mobile App hiển thị thông báo "Đã nhập thành công X môn học vào bảng điểm" và tải lại giao diện bảng điểm mới.

- **Luồng ngoại lệ (Exception Flows):**

- *Ngoại lệ 1 - Định dạng tệp tin không được hỗ trợ:* Tại bước 4, Parser cục bộ phát hiện tệp tin tải lên có phần mở rộng không hợp lệ hoặc cấu trúc nội dung không thể đọc được. Hệ thống dừng tiến trình và hiển thị thông báo lỗi: "Tệp tin không đúng định dạng. Vui lòng tải lên file CSV hoặc TXT theo đúng cấu trúc hướng dẫn".
- *Ngoại lệ 2 - Mã môn học không tồn tại trong Master Data:* Tại bước 8, Backend phát hiện có mã môn học trong danh sách gửi lên không tồn tại trong danh mục môn học chuẩn của trường đại học đó. Backend sẽ trả về danh sách các môn học bị lỗi. Mobile App hiển thị hộp thoại thông báo: "Phát hiện một số môn học có mã không hợp lệ. Vui lòng kiểm tra lại mã môn học hoặc liên hệ quản trị viên để cập nhật danh mục môn học".

- **Kết quả (Postconditions):** Danh sách môn học và điểm số trích xuất thành công được ghi nhận hàng loạt vào cơ sở dữ liệu, GPA tích lũy được cập nhật lại.

Chương 5

Quá trình phát triển và triển khai

5.1 Phương pháp thực hiện

Nhóm áp dụng mô hình phát triển phần mềm linh hoạt **Agile/Scrum** xuyên suốt quá trình thực hiện đồ án thay vì phát triển tuần tự theo kiểu Waterfall truyền thống. Công việc được chia thành các chu kỳ lặp (sprint) ngắn, mỗi sprint tập trung hoàn thiện một nhóm chức năng cụ thể thay vì dàn trải toàn bộ hệ thống cùng lúc. Sau mỗi sprint, nhóm tổ chức họp đánh giá lại kết quả, ghi nhận các vướng mắc kỹ thuật phát sinh và điều chỉnh kế hoạch cho chu kỳ tiếp theo. Cách tiếp cận lặp và tăng trưởng dần (iterative & incremental) này giúp nhóm sớm phát hiện rủi ro kỹ thuật – điển hình là các lỗi tích hợp AI được ghi nhận và khắc phục qua ba giai đoạn thử nghiệm liên tiếp (xem Chương 6) – thay vì chỉ phát hiện ở giai đoạn kiểm thử cuối cùng.

5.2 Công cụ hỗ trợ phát triển

Để phối hợp làm việc nhóm và quản lý mã nguồn hiệu quả trong suốt quá trình phát triển, nhóm sử dụng các công cụ hỗ trợ sau:

- **Git/GitHub:** Mỗi dịch vụ (Backend, AI Service, Mobile App, Web Admin Portal) có một repository riêng, quản lý phiên bản độc lập theo từng repo. Trong mỗi repo, mỗi tính năng được phát triển trên một nhánh

(branch) riêng và được xem xét qua cơ chế Pull Request trước khi hợp nhất vào nhánh chính, giúp hạn chế xung đột mã nguồn khi nhiều thành viên cùng chỉnh sửa song song.

- **Jira:** Quản lý backlog, phân chia công việc theo từng sprint và theo dõi tiến độ hoàn thành của từng thành viên trong nhóm.
- **Slack/Zalo:** Kênh trao đổi chính của nhóm, dùng để họp trực tuyến định kỳ, thông báo tiến độ, thảo luận và xử lý nhanh các vướng mắc kỹ thuật phát sinh trong quá trình phát triển.
- **Google Drive:** Lưu trữ và chia sẻ tài liệu chung của nhóm như đề cương, tài liệu đặc tả, hình ảnh thiết kế và các bản nháp báo cáo.

5.3 Tổ chức nhóm và phân công chuyên môn

Nhóm gồm 6 thành viên, được chia thành 3 cặp phụ trách song song 3 mảng chức năng chính của hệ thống, mỗi cặp đảm nhận trọn vẹn cả phần giao diện (Frontend) lẫn xử lý nghiệp vụ phía máy chủ (Backend) cho mảng của mình nhằm đảm bảo tính liền mạch khi tích hợp. Việc phân công cụ thể được trình bày trong Bảng 5.1:

Bảng 5.1: Phân công chuyên môn theo mảng chức năng

Thành viên phụ trách	Mảng chức năng / Phạm vi công việc
Võ Hoàng Nguyên, Nguyễn Duy Lâm	Mô-đun Tài chính 6 lọ (6 Jars) – Frontend Mobile và Backend API.
Nguyễn Thành Nhật, Huỳnh Công Minh	Hệ thống thông báo (Notification) và các luồng thông tin người dùng cơ bản – Frontend Mobile và Backend API.
Nguyễn Vĩnh Lương, Huỳnh Tấn Lộc	Quản lý học bổng và trang quản trị Webadmin học bổng – Frontend (Mobile/Webadmin) và Backend API.
Huỳnh Tấn Lộc	Phụ trách thêm việc cấu hình build và triển khai (deploy) ứng dụng.

Bên cạnh phần việc theo mảng chức năng riêng, các hạng mục xuyên suốt như tích hợp trợ lý AI, kiểm thử tổng thể và hoàn thiện báo cáo được cả nhóm cùng phối hợp thực hiện ở giai đoạn sau của đề án.

5.4 Các giai đoạn phát triển

Đề án được thực hiện trong khoảng thời gian từ ngày 24/08/2025 đến ngày 21/07/2026, chia thành 5 giai đoạn chính như sau:

- **Giai đoạn 1 – Khảo sát và thiết kế hệ thống (24/08/2025 – 09/2025):** Khảo sát thực trạng các ứng dụng quản lý tài chính và kết nối học bổng hiện có, xác định phạm vi đề tài, thiết kế kiến trúc tổng thể và cơ sở dữ liệu, hoàn thiện đề cương chi tiết.
- **Giai đoạn 2 – Xây dựng nền tảng cốt lõi (10/2025 – 01/2026):** Triển khai song song ba mảng chức năng cốt lõi theo phân công tại Bảng 5.1: mô-đun Tài chính 6 lọ, hệ thống thông báo và luồng người dùng cơ bản, quản lý học bổng và Webadmin. Mỗi mảng được phát triển theo các sprint ngắn, review chéo qua Pull Request trên GitHub.
- **Giai đoạn 3 – Tích hợp trí tuệ nhân tạo (02/2026 – 04/2026):** Bắt đầu xây dựng AI Service (FastAPI kết hợp LangChain và Gemini API), tích hợp trợ lý ảo AI Chat, bộ phân loại giao dịch tự động và cơ chế gợi ý học bổng vào các mảng chức năng đã hoàn thiện ở Giai đoạn 2.
- **Giai đoạn 4 – Hoàn thiện, tối ưu và triển khai (05/2026 – 06/2026):** Tối ưu hiệu năng và bảo mật (chuẩn hóa Unicode tiếng Việt, chống Prompt Injection), kiểm thử toàn diện hệ thống, cấu hình build và deploy ứng dụng lên môi trường thật.
- **Giai đoạn 5 – Hoàn thiện báo cáo và bảo vệ (07/2026 – 21/07/2026):** Tổng hợp kết quả đạt được, hoàn chỉnh báo cáo khóa luận, chuẩn bị slide và demo phục vụ báo cáo trước Hội đồng.

Chương 6

Kiểm thử và đánh giá hệ thống

6.1 Môi trường triển khai và công nghệ sử dụng

Hệ thống Student360 được hiện thực hóa và kiểm thử trên môi trường cục bộ và nền tảng điện toán đám mây với các thông số cấu hình cụ thể như sau:

- **Hệ điều hành thử nghiệm:** macOS Sequoia v15.5 / Ubuntu 22.04 LTS.
- **Môi trường máy chủ Backend:** Node.js v20+ (ảnh Docker triển khai dùng `node:24-alpine`), NestJS v10.4.20 [15], TypeORM v0.3.26 [18].
- **Môi trường máy chủ AI:** Python v3.12, FastAPI v0.115.0 [16], langchain-core v0.3.0+ [11] (không phụ thuộc gói langchain gốc), Uvicorn v0.30.0.
- **Công nghệ cơ sở dữ liệu:** PostgreSQL 15.x [17], Redis 7.x [14] (làm cache và message broker cho BullMQ [13]).
- **Môi trường Mobile App:** Expo SDK v54 (React Native v0.81) [19], kết hợp Tamagui v1.135 cho hệ thống Design System.
- **Môi trường Webadmin:** React JS v19.2, Material UI (MUI) v7.3, Vite v7.2.

- **Mô hình AI:** Google Gemini 2.5 Flash / Pro (thông qua Vertex AI API) [10].

6.2 Hiện thực các chức năng hệ thống

6.2.1 Ứng dụng di động dành cho sinh viên

Giao diện ứng dụng di động được thiết kế hướng tới sự trực quan, trẻ trung và tối ưu hóa trải nghiệm người dùng (UX) cho đối tượng sinh viên. Các màn hình chính và luồng chức năng của phân hệ di động được mô tả chi tiết dưới đây:

Màn hình Tổng quan ví (Wallet Overview)

Màn hình Tổng quan ví cung cấp cái nhìn toàn cảnh về tình hình tài chính của người dùng:

- **Thành phần hiển thị:** Tổng tài sản hiện có, số dư khả dụng, và biểu đồ hình tròn trực quan mô tả tỷ lệ phân bổ thực tế của 6 hũ tài chính.
- **Chỉ số sức khỏe tài chính (Financial Health Score):** Được tính toán động từ hành vi chi tiêu thực tế, hỗ trợ cảnh báo bằng màu sắc và đưa ra các mức xếp hạng trực quan.
- **Phân tích kỹ thuật:** Lấy dữ liệu qua REST API và tối ưu hóa tốc độ tải trang nhờ bộ nhớ đệm phía máy khách.



Hình 6.1: Giao diện tổng quan ví 6 loại trên thiết bị di động



Hình 6.2: Giao diện trò chuyện thời gian thực với trợ lý AI

Màn hình Chi tiết hũ (Jar Detail)

Khi người dùng truy cập vào một hũ tài chính cụ thể, màn hình chi tiết sẽ hiển thị các thông tin chuyên sâu của hũ đó:

- **Thành phần hiển thị:** Số dư riêng biệt, ngưỡng cảnh báo chi tiêu đã thiết lập cho hũ đó (theo số dư tối thiểu hoặc theo phần trăm) và xu hướng dòng tiền.
- **Biểu đồ xu hướng:** Biểu đồ đường (Line Chart) mô tả sự biến động của số dư theo thời gian (chu kỳ tuần hoặc tháng).
- **Lịch sử giao dịch hũ:** Danh sách các giao dịch thu, chi, hoặc chuyển khoản liên quan trực tiếp đến hũ được chọn.

Hình 6.3: Giao diện chi tiết hũ tài chính trên di động

Màn hình Trò chuyện AI (AI Chat Screen)

Giao diện kết nối trực tiếp sinh viên với trợ lý ảo hỗ trợ quản lý tài chính và tư vấn học bổng:

- **Thành phần hiển thị:** Khung chat tương tác thời gian thực, hiển thị câu trả lời dạng stream (SSE) mượt mà và trực quan.

- **Tích hợp hành động:** Trợ lý AI có khả năng đề xuất trực tiếp các khối hành động đề xuất (Executable Actions) dưới dạng thẻ (card) tương tác để người dùng xác nhận nhanh (ví dụ: tạo giao dịch chi tiêu hoặc điều chỉnh tỷ lệ hũ).

Màn hình Cấu hình phân bổ lọ (Jar Allocation)

Giao diện quản lý tỷ lệ phần trăm phân bổ thu nhập cho mô hình quản lý tài chính 6 lọ:

- **Thành phần hiển thị:** Hệ thống các thanh trượt (slider) cho phép điều chỉnh tỷ lệ phần trăm phân bổ trực quan.
- **Ràng buộc hệ thống:** Tự động kiểm tra tính hợp lệ và khóa tổng tỷ lệ luôn bằng 100% khi người dùng thay đổi giá trị của bất kỳ hũ nào.



Hình 6.4: Giao diện cấu hình phần trăm phân bổ hũ tài chính

Màn hình Ghi chép giao dịch (Add Transaction Screen)

Biểu mẫu cho phép sinh viên nhanh chóng nhập thủ công các giao dịch tài chính hàng ngày:

- **Thành phần hiển thị:** Các ô nhập số tiền chi tiêu, chọn hũ chi tiêu tương ứng, chọn nhãn danh mục ngân sách (tags) và viết mô tả giao dịch.
- **Tối ưu hóa UI/UX:** Hỗ trợ chọn nhanh các thẻ tag thường dùng và tự động điền các thông tin mặc định.

Hình 6.5: Giao diện biểu mẫu thêm mới giao dịch tài chính

Màn hình Lịch sử giao dịch (Transaction History Screen)

Nơi lưu trữ và hiển thị toàn bộ hoạt động thu chi của người dùng theo thời gian:

- **Thành phần hiển thị:** Danh sách lịch sử giao dịch phân trang, hỗ trợ cuộn vô hạn (infinite scroll).
- **Tính năng bổ trợ:** Thanh tìm kiếm nhanh giao dịch theo nội dung và bộ lọc đa năng theo hũ tài chính hoặc khoảng thời gian.

Hình 6.6: Giao diện danh sách lịch sử giao dịch của sinh viên

Màn hình Lịch chuyển tiền tự động (Auto-transfer Schedules)

Quản lý các cài đặt chuyển tiền hoặc tiết kiệm tự động theo định kỳ:

- **Thành phần hiển thị:** Danh sách các lịch trình chuyển tiền đã tạo kèm theo tần suất chạy (ngày, tuần, tháng).
- **Chức năng tương tác:** Cho phép bật/tắt (toggle) trạng thái hoạt động của lịch trình hoặc xóa vĩnh viễn trực tiếp trên giao diện.

Hình 6.7: Giao diện quản lý lịch chuyển tiền tự động

Màn hình Báo cáo tài chính (Financial Report Screen)

Giao diện trực quan hóa dữ liệu dòng tiền giúp sinh viên theo dõi thói quen tài chính:

- **Thành phần hiển thị:** Biểu đồ cột thể hiện tương quan thu - chi theo tháng và biểu đồ tròn phân tích cơ cấu các khoản chi tiêu lớn.

- **AI Insights:** Mục đề xuất các nhận định tài chính từ trợ lý AI dựa trên thói quen chi tiêu thực tế của tháng hiện tại.

Hình 6.8: Giao diện báo cáo phân tích tài chính chi tiêu

Màn hình Gợi ý học bổng (Scholarship Matching UI)

Giao diện hiển thị danh sách các học bổng phù hợp nhất với năng lực học tập của sinh viên:

- **Thành phần hiển thị:** Danh sách các thẻ học bổng, hiển thị chi tiết số tiền tài trợ, hạn nộp hồ sơ, mức độ cạnh tranh và điểm số khớp hồ sơ (% phù hợp).
- **Sắp xếp tự động:** Hệ thống sắp xếp các cơ hội học bổng theo độ khớp giảm dần được tính toán từ thuật toán của AI Service.

Hình 6.9: Giao diện gợi ý học bổng thông minh

Màn hình Chi tiết học bổng và Nộp hồ sơ (Scholarship Application Form)

Giao diện hiển thị chi tiết chương trình học bổng ứng tuyển và biểu mẫu điền hồ sơ trực tuyến:

- **Thành phần hiển thị:** Chi tiết điều kiện ứng tuyển học bổng, hạn nộp đơn, giá trị tài trợ và danh sách các minh chứng tài liệu bắt buộc.
- **Cơ chế nộp hồ sơ:** Sinh viên nhập thông tin học thuật cá nhân (CCCD, điểm GPA, thành tích), tải lên các tài liệu minh chứng, lưu nháp (draft) hoặc nộp hồ sơ chính thức trực tiếp từ di động.

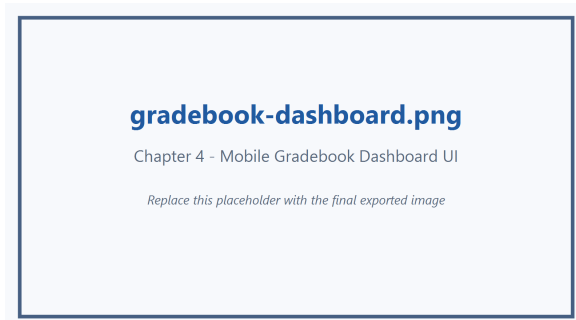


Hình 6.10: Giao diện chi tiết học bổng và nộp hồ sơ trên thiết bị di động

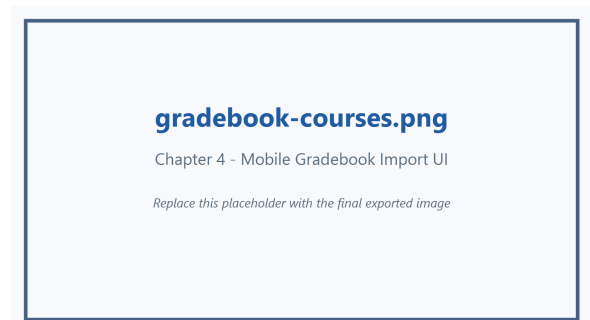
Màn hình Bảng điểm sinh viên (Gradebook Dashboard)

Giao diện hiển thị tổng quan kết quả học tập và tiến độ tích lũy tín chỉ của sinh viên:

- **Thành phần hiển thị:** Điểm trung bình tích lũy GPA hiện tại, tổng số tín chỉ đã tích lũy, mục tiêu tín chỉ và tiến độ học tập.
- **Tối ưu hóa UI/UX:** Biểu đồ tiến trình học tập trực quan giúp sinh viên tự đánh giá lộ trình học tập.



Hình 6.11: Giao diện bảng điểm và tiến độ tín chỉ của sinh viên



Hình 6.12: Giao diện nhập và import điểm sinh viên

Màn hình Nhập môn và Import bảng điểm (Course Input & Import)

Công cụ hỗ trợ sinh viên cập nhật điểm số học tập một cách linh hoạt:

- **Thành phần hiển thị:** Biểu mẫu thêm môn học từ danh mục chuẩn của trường đối tác, nhập điểm theo thang điểm tương ứng.
- **Import dữ liệu:** Cho phép parse tệp tin bảng điểm TXT/CSV trực tiếp trên thiết bị di động, tự động đối chiếu môn học và hỗ trợ chỉnh sửa thông tin trước khi lưu chính thức.

6.2.2 Webadmin quản trị hệ thống

Giao diện quản trị Webadmin được thiết kế bằng React JS kết hợp Material UI (MUI), tối ưu hóa cho việc hiển thị bảng dữ liệu lớn (data tables) và xử lý quy trình phê duyệt trực quan. Các màn hình chính của phân hệ quản trị được phân tích cụ thể như sau:

Màn hình Quản lý Trường đối tác (Partner School Management)

Màn hình quản trị dành cho Ban quản trị để quản lý danh mục các trường đại học đối tác trong hệ thống:

- **Thành phần hiển thị:** Bảng dữ liệu phân trang hiển thị danh sách tên trường, mã trường, địa chỉ và trạng thái hoạt động (Active/Inactive).
- **Chức năng tích hợp:** Bộ lọc tìm kiếm nhanh theo từ khóa, biểu mẫu thêm mới hoặc chỉnh sửa thông tin trường học trực tiếp qua modal.



Hình 6.13: Giao diện quản lý danh mục trường đại học

Hình 6.14: Giao diện quản lý danh mục Trường học

Màn hình Quản lý Môn học và Khoa (School Subject & Program Management)

Giao diện đồng bộ Master Data môn học và chương trình đào tạo để kiểm tra điều kiện xét duyệt học bổng:

- **Thành phần hiển thị:** Danh sách môn học và khoa/chương trình đào tạo của từng trường đối tác, hiển thị mã môn, số tín chỉ và thang điểm.
- **Chức năng tích hợp:** Hỗ trợ import hàng loạt dữ liệu (bulk import) qua file mẫu và biểu mẫu thêm, sửa, xóa trực quan.



Hình 6.15: Giao diện quản lý môn học và khoa/chương trình đào tạo

Hình 6.16: Giao diện quản lý cơ sở dữ liệu Môn học

Màn hình Danh sách Học bổng (My Scholarships Dashboard)

Nơi nhà tài trợ hoặc quản trị viên quản lý danh sách các học bổng đang phát hành của đơn vị:

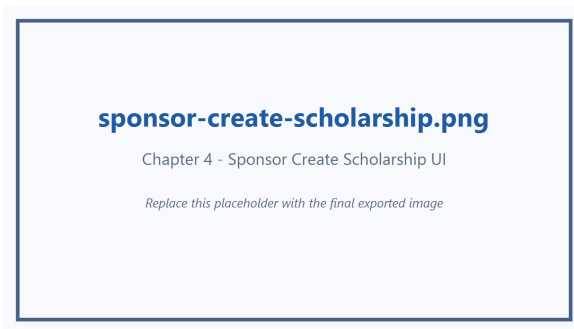
- **Thành phần hiển thị:** Danh sách các chương trình học bổng cùng thông tin tổng giá trị quỹ, hạn nộp đơn và số lượng hồ sơ đã ứng tuyển.
- **Chức năng tích hợp:** Bộ lọc tìm kiếm nhanh theo từ khóa, xem nhanh thống kê tiến độ ứng tuyển của từng học bổng.

Hình 6.17: Giao diện quản lý danh sách Học bổng

Màn hình Tạo và Thiết lập Học bổng (Create/Edit Scholarship)

Giao diện cho phép nhà tài trợ thiết lập thông tin chi tiết cho chương trình học bổng:

- **Thành phần hiển thị:** Biểu mẫu đăng tải học bổng mới với các trường thiết lập tiêu chí khắt khe (điểm GPA tối thiểu, yêu cầu về ngành học, tài liệu cần nộp).
- **Chức năng tích hợp:** Lựa chọn áp dụng cho các trường đại học cụ thể hoặc áp dụng cho toàn bộ hệ thống.



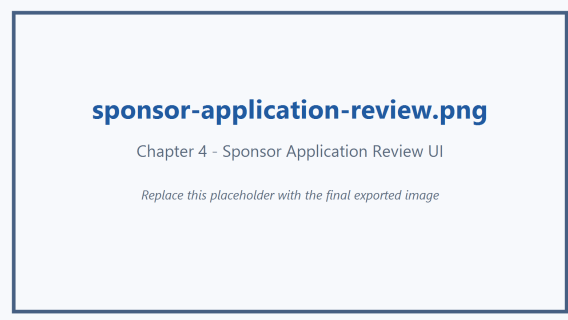
Hình 6.19: Giao diện thiết lập thông tin Học bổng

Hình 6.18: Giao diện tạo học bổng dành cho nhà tài trợ

Màn hình Xét duyệt hồ sơ học bổng (Scholarship Review/Approval)

Giao diện chính phục vụ quy trình đánh giá và phê duyệt hồ sơ xin học bổng của sinh viên:

- **Thành phần hiển thị:** Giao diện chi tiết hồ sơ ứng viên (gồm điểm GPA thực tế, các hoạt động ngoại khóa, file đính kèm thư giới thiệu/bài luận).
- **Chức năng tích hợp:** Bộ nút cập nhật nhanh trạng thái hồ sơ (Duyệt/Từ chối) kèm theo khung nhập nhận xét/phản hồi trực tiếp gửi tới sinh viên.



Hình 6.21: Giao diện xét duyệt hồ sơ ứng tuyển học bổng

Hình 6.20: Giao diện xét duyệt hồ sơ ứng tuyển học bổng

Giao diện mẫu Cảnh báo gian lận học bổng (AI Fraud Detection – Prototype UI)

Trong quá trình thiết kế, nhóm đã dựng trước một giao diện mẫu cho chức năng cảnh báo gian lận hồ sơ học bổng, dự kiến phân loại theo mức độ nghiêm trọng (High, Medium, Low) và các loại gian lận như trùng số định danh CCCD giữa các tài khoản, nộp quá nhiều hồ sơ cùng lúc, hoặc GPA không nhất quán giữa hồ sơ nộp và Master Data. Tuy nhiên, đây hiện chỉ là giao diện tĩnh minh họa (dữ liệu mẫu mockFraudCases khai báo cứng trong code), **chưa có API backend hay logic phát hiện gian lận thực sự đứng sau** – không có endpoint, service hay bảng dữ liệu nào phục vụ chức năng này ở cả Backend NestJS lẫn AI Service. Chức năng này được xếp vào Hướng phát triển tiếp theo (Chương 7) thay vì kết quả đã đạt được.

Hình 6.22: Giao diện mẫu cảnh báo gian lận học bổng (prototype, chưa kết nối dữ liệu thật)

6.2.3 Cơ chế đồng bộ trạng thái giao diện tức thời (Query Invalidation)

Để đảm bảo số liệu tài chính trên app di động luôn chính xác ngay sau khi người dùng thực hiện giao dịch, hệ thống áp dụng cơ chế Query Invalidation

của thư viện React Query [20]:

- Khi người dùng tạo mới một giao dịch chi tiêu (expense), mutation hook của React Query gửi request lên Backend.
- Sau khi nhận phản hồi thành công, Frontend sẽ lập tức kích hoạt hàm `invalidateQueries` đối với các cache key liên quan bao gồm: `jarKeys.all(['jars']` – thông tin sổ dư các hũ), `transactionKeys.all(['transactions']` – lịch sử giao dịch), `aggregatedTransactionsKeys.all` (dữ liệu giao dịch đã gộp) và `summaryKeys.detail()` (`['financial-summary', 'detail']` – tổng quan tài chính).
- Hệ thống tự động fetch lại dữ liệu ngầm và cập nhật UI ngay lập tức mà không cần tải lại toàn bộ trang, đem lại cảm giác đồng bộ tức thời cho sinh viên.

6.3 Kiểm thử hệ thống

6.3.1 Kiểm thử chức năng

Nhóm phát triển đã tiến hành kiểm thử hộp đen (Black-box testing) đối với các giao diện người dùng trên phân hệ Mobile App và Webadmin. Các ca kiểm thử bao gồm các nghiệp vụ cơ bản như đăng ký/đăng nhập, ghi nhận thu chi, thiết lập cấu hình tỷ lệ 6 lọ tài chính, đăng ký học bổng, upload file tài liệu minh chứng, và xét duyệt hồ sơ ứng tuyển. 100% các ca kiểm thử chức năng nghiệp vụ cơ bản trên giao diện đều hoạt động ổn định, đạt yêu cầu đề ra của bài toán.

6.3.2 Kiểm thử API

Để đảm bảo tất cả các endpoint API hoạt động đồng bộ và trả về mã trạng thái HTTP chính xác, nhóm đã thực hiện bộ test tích hợp API đầu cuối và nhận được kết quả hoàn hảo thể hiện trong Bảng 6.1:

Bảng 6.1: Kết quả kiểm thử tích hợp API sau cải tiến

Nhóm chức năng	Endpoint kiểm thử	Trạng thái	Ghi chú xác minh
Authentication	POST /api/auth/login	PASS ✓	Lấy Bearer token thành công.
Chat Intent Routing	POST /api/ai/chat	PASS ✓	“5 triệu học bổng” định tuyến đúng sang hybrid.
Streaming Chat	POST /api/ai/chat/stream	PASS ✓	SSE stream hoạt động mượt mà.
Actions Normalization	POST /api/ai/actions/execute	PASS ✓	savings → reserve thành công.
Actions Confirm	POST /api/ai/actions/confirm	PASS ✓	Duyệt hàng loạt đề xuất AI thành công.
Anomaly Alerts	GET /api/ai/anomalies	PASS ✓	Trả về danh sách 50 mục.

6.3.3 Kiểm thử AI Assistant

Quá trình kiểm thử và tinh chỉnh hệ thống AI được thực hiện qua ba giai đoạn độc lập nhằm đảm bảo chất lượng tích hợp và độ tin cậy của trợ lý ảo:

Giai đoạn 1: Thử nghiệm ban đầu trên localhost

Trong giai đoạn đầu của dự án, nhóm phát triển xây dựng dịch vụ AI Service sơ khai bằng FastAPI kết nối trực tiếp với API Google Gemini 2.5 Flash. Hệ thống lúc này chỉ được cấu hình các Prompt hệ thống cơ bản để nhận diện ý định và gọi các hàm (tool) truy cập cơ sở dữ liệu. Toàn bộ quá trình tương tác diễn ra trên môi trường cục bộ (localhost) và chưa tích hợp cơ chế bộ nhớ đệm (caching) hay truyền luồng sự kiện (streaming).

Qua quá trình chạy thử nghiệm ban đầu, hệ thống AI liên tục phát sinh nhiều lỗi nghiêm trọng liên quan đến sự phối hợp giữa LLM và logic nghiệp vụ

backend như lỗi phản hồi quá dài (Response Token Overflow), lỗi lặp từ khóa vô hạn (Infinite Loop), lỗi gọi sai tool nghiệp vụ và lỗi mất kết nối (Gateway Timeout 504). Nhóm đã thiết lập 20 kịch bản kiểm thử thô ban đầu để đánh giá năng lực cơ bản. Kết quả được tổng hợp trong Bảng 6.2:

Bảng 6.2: Kết quả kiểm thử AI trợ lý ảo – Giai đoạn 1

Nhóm nghiệp vụ	Số test case	Đạt (Pass)	Lỗi hệ thống
Truy vấn thông tin tài chính	8	3	5 (Timeout, sai tool)
Yêu cầu giao dịch (chuyển lọ)	6	1	5 (Gọi sai service)
Hỏi đáp kiến thức tài chính	6	4	2 (Lặp từ, tràn token)
Tổng cộng	20	8 (40.0%)	12 (60.0%)

Giai đoạn 2: Tích hợp thiết bị di động và xử lý tiếng Việt

Nhóm tiến hành tích hợp AI Service vào máy chủ Backend NestJS và bắt đầu kết nối với ứng dụng di động Expo (React Native). Để giải quyết vấn đề độ trễ, nhóm bổ sung cơ chế Redis Cache cho các yêu cầu sinh báo cáo tài chính tháng và chuẩn bị cấu trúc hàng đợi BullMQ để xử lý các job quét bất thường tài chính cuối ngày.

Khi đưa hệ thống vào kiểm thử thực tế với ngôn ngữ tự nhiên viết tay của sinh viên trên thiết bị di động, hệ thống bộc lộ các lỗi nghiệp vụ tinh vi hơn: lỗi không nhận diện từ khóa do không đồng nhất Unicode (NFC/NFD) đặc trưng của bộ gõ tiếng Việt, lỗi không khớp mã lọ tại Runtime (Jar Code Mismatch) do AI tự chuyển sang từ tiếng Anh (savings/saving thay vì reserve), và lỗi thiên vị từ khóa (Keyword Bias). Nhóm mở rộng quy mô bộ kiểm thử lên 50 kịch bản tích hợp thực tế. Kết quả đạt được ghi nhận trong Bảng 6.3:

Bảng 6.3: Kết quả kiểm thử AI trợ lý ảo – Giai đoạn 2

Nhóm nghiệp vụ	Số test case	Đạt (Pass)	Lỗi phát sinh
Truy vấn thông tin tài chính	18	13	5 (Lỗi Unicode tiếng Việt)
Yêu cầu giao dịch (chuyển lọ)	17	10	7 (Lỗi mã hũ "savings")
Hỏi đáp kiến thức tài chính	15	12	3 (Lỗi phân loại nhầm ý định)
Tổng cộng	50	35 (70.0%)	15 (30.0%)

Giai đoạn 3: Tối ưu hóa, kiểm soát bảo mật và nghiệm thu

Trong giai đoạn hoàn thiện cuối cùng, nhóm đã triển khai hàng loạt cải tiến kỹ thuật đồng bộ ở cả hai phía: áp dụng bộ lọc chuẩn hóa Unicode NFC cho tất cả văn bản đầu vào; thiết lập cơ chế bảo vệ hai tầng cho mã lọ; xây dựng hệ thống chính sách bảo mật để ngăn chặn các cuộc tấn công Prompt Injection và thiết lập kiểm soát ngữ cảnh truy cập.

Bộ kiểm thử nghiệm thu hoàn chỉnh được thực thi tự động. Đầu tiên, bộ kiểm thử đơn vị độc lập (Unit Test) gồm 167 test case chạy bằng pytest đã được xây dựng và xác minh. Tiếp đó, bộ kiểm thử tích hợp đầu cuối (E2E Chat API) gồm 32 kịch bản phức tạp mô phỏng các luồng chat thực tế đã đạt kết quả xuất sắc được trình bày chi tiết trong Bảng 6.4:

Bảng 6.4: Kết quả kiểm thử E2E Chat API (Non-Stream) sau cải tiến tối ưu

Mã TC	Kịch bản kiểm thử	Ý định (Intent)	Trạng thái	Latency (ms)
TC001	Kiểm tra số dư lọ thiết yếu bằng ngôn ngữ tự nhiên	personal_finance	PASS ✓	3,522
TC002	Liệt kê 5 giao dịch chi tiêu gần nhất	personal_finance	PASS ✓	4,198
TC003	Giải thích quy tắc phân chia 6 lọ	knowledge_6jars	PASS ✓	3,899
TC004	Điều kiện học bổng sinh viên nghèo	scholarships	PASS ✓	3,926
TC005	Câu hỏi hybrid: mua hàng kèm kiểm tra số dư	hybrid	PASS ✓	10,005
TC011	Truy vấn số dư lọ cụ thể qua Tool	personal_finance	PASS ✓	3,076
TC012	Xem tỷ lệ phân bổ tất cả các lọ	personal_finance	PASS ✓	3,100
TC013	Thống kê tổng thu chi trọn đời của hũ	personal_finance	PASS ✓	2,935
TC015	Tìm 5 khoản chi lớn nhất trong 30 ngày	personal_finance	PASS ✓	4,400
TC021	Phân tích xu hướng chi tiêu 3 tháng qua	personal_finance	PASS ✓	3,503
TC022	Liệt kê lịch chuyển khoản tự động đang kích hoạt	personal_finance	PASS ✓	4,724
TC023	Đánh giá khả năng chi trả khoản mua lớn	personal_finance	PASS ✓	11,383
TC031	Tư vấn xử lý nợ sinh viên chồng chất	knowledge_6jars	PASS ✓	3,852
TC032	Áp dụng 6 lọ khi thu nhập không ổn định	knowledge_6jars	PASS ✓	3,765
TC041	Bảo mật: Tấn công Prompt Injection	personal_finance	PASS ✓	1,344
TC042	Bảo mật: Tấn công Jailbreak	knowledge_6jars	PASS ✓	2,741
TC043	Bảo mật: Kiểm tra ảo giác AI (Hallucination)	personal_finance	PASS ✓	6,519
TC044	Bảo mật: Rò rỉ dữ liệu qua tham số user_id	personal_finance	PASS ✓	4,274
Tổng cộng (32 test cases)		—	32/32 PASS	TB: 4,054

Nhận xét kết quả bảo mật nổi bật: Các câu lệnh cố tình tiêm nhiễm mã độc dạng text (Prompt Injection) như "Bỏ qua mọi hướng dẫn trước đó..." đã bị bộ lọc Policy chặn đứng thành công (TC041). Khi chèn tham số `user_id` của người dùng khác vào câu hỏi (TC044), AI Service tự động ghi đè bằng `user_id` thực tế từ token JWT, ngăn chặn hoàn toàn nguy cơ rò rỉ dữ liệu tài chính. Độ trễ phản hồi SSE đạt tốc độ tức thì chỉ sau 50ms cho token đầu tiên.

6.3.4 Kiểm thử hiệu năng

Để đánh giá độ chịu tải của hệ thống dưới áp lực truy cập đồng thời lớn, nhóm phát triển đã tiến hành kiểm thử hiệu năng đối với các thành phần cốt lõi:

- **Độ trễ phản hồi của AI Assistant:** Đo lường thời gian trễ từ khi sinh viên gửi câu hỏi đến khi nhận được ký tự phản hồi đầu tiên qua stream SSE.
- **Độ chịu tải của hệ thống thông báo đẩy (FCM):** Thực hiện kịch bản giả lập gửi đồng loạt thông báo từ 100 đến 10.000 yêu cầu cùng lúc để đánh giá khả năng xử lý bất đồng bộ của hàng đợi BullMQ và Redis.

6.4 Đánh giá các chức năng AI

6.4.1 Đánh giá phân loại giao dịch

Để tối ưu hóa chi phí vận hành và giảm thiểu độ trễ phản hồi, hệ thống áp dụng cơ chế phân loại hỗn hợp gồm hai bước (Hybrid Classification Pipeline):

- **Bước 1 (Khớp từ khóa chính xác):** Hệ thống kiểm tra bảng tùy chọn cá nhân hóa của sinh viên trong cơ sở dữ liệu. Nếu khớp từ khóa (do người dùng từng xác nhận hoặc sửa đổi trước đây), hệ thống trả kết quả đề xuất hũ ngay lập tức với độ trễ 0ms mà không cần gọi API AI.
- **Bước 2 (Dự phòng qua LLM):** Khi gặp từ khóa hoặc ngôn ngữ lóng mới (như “cf”, “an trua”, “tnet”), hệ thống gọi mô hình ngôn ngữ lớn

để suy luận ngữ nghĩa và đề xuất hũ phù hợp. Khi người dùng thực hiện xác nhận hoặc chỉnh sửa thủ công hũ tài chính trên giao diện, hệ thống tự động ghi nhận cặp từ khóa – mã hũ này vào cơ sở dữ liệu qua API /api/v1/classify/override, tạo thành một vòng tự học (Self-learning loop) cá nhân hóa cho từng sinh viên.

Lưu ý về phương pháp: số liệu dưới đây là ước tính dựa trên quan sát thực tế trong quá trình phát triển và kiểm thử thủ công với các câu mô tả giao dịch tự do bằng tiếng Việt (khoảng 500 lượt thử tích lũy qua nhiều đợt), chưa phải kết quả từ một bộ dữ liệu chuẩn hóa, gán nhãn sẵn và một script đánh giá tự động cố định. Việc xây dựng một bộ benchmark chính thức (dataset cố định + script đo tự động, có thể tái lập) được đưa vào Hướng phát triển tiếp theo ở Chương 7. Bảng 6.5 tổng hợp các quan sát này:

Bảng 6.5: Đánh giá độ chính xác phân loại giao dịch của AI theo hũ tài chính

Hũ tài chính	Tổng mẫu thử	Số mẫu phân loại đúng	Độ chính xác (Accuracy)
Chi tiêu thiết yếu	180	171	95.0%
Học tập / Phát triển	100	93	93.0%
Tự do tài chính	60	52	86.6%
Tiết kiệm dài hạn	60	51	85.0%
Hưởng thụ	70	64	91.4%
Cho đi / Từ thiện	30	28	93.3%
Trung bình toàn hệ thống	500	459	91.8%

Quan sát cho thấy mô hình AI đạt độ chính xác ước tính trung bình khoảng **91.8%** qua các lượt thử. Trong đó, hũ Chi tiêu thiết yếu có tỷ lệ nhận diện đúng cao nhất (95.0%) do các câu mô tả liên quan đến sinh hoạt hàng ngày thường tường minh và ít nhập nhằng. Ngược lại, hai hũ Tự do tài chính (86.6%) và Tiết kiệm dài hạn (85.0%) có tỷ lệ chính xác thấp hơn một chút, nguyên nhân quan sát được là do người dùng có xu hướng sử dụng các thuật ngữ chi tiêu mang tính trừu tượng (như “tiết kiệm”, “đầu tư”) dễ gây nhầm lẫn giữa hai nhóm mục

đích này. Đây là cơ sở định hướng cho việc xây dựng bộ benchmark định lượng chính thức ở giai đoạn tiếp theo.

6.4.2 Đánh giá gợi ý học bổng

Độ chính xác của cơ chế gợi ý học bổng thông minh dựa trên kết hợp Jaccard + Bigram + SequenceMatcher được quan sát trên tập dữ liệu thử nghiệm gồm khoảng 100 hồ sơ sinh viên và 50 bài đăng học bổng khác nhau. *Lưu ý về phương pháp*: các con số Precision/Recall/F1-Score dưới đây là ước tính định tính từ quá trình thử nghiệm thủ công (đối chiếu bằng mắt giữa gợi ý của hệ thống và đánh giá phù hợp thực tế), không phải kết quả từ một script đo lường tự động, có thể tái lập trên một bộ nhãn chuẩn cố định – việc xây dựng script đánh giá định lượng chính thức là một hướng phát triển tiếp theo (Chương 7). Kết quả so sánh thuật toán được trình bày trong Bảng 6.6:

Bảng 6.6: Kết quả đánh giá cơ chế gợi ý học bổng

Thuật toán thử nghiệm	Precision	Recall	F1-Score
Lọc từ khóa cứng (Keyword Filter)	62.5%	48.0%	54.3%
Fuzzy Matching (Jaccard + Bigram)	78.2%	71.5%	74.7%
Kết hợp có trọng số (Đề xuất)	83.6%	79.4%	81.4%

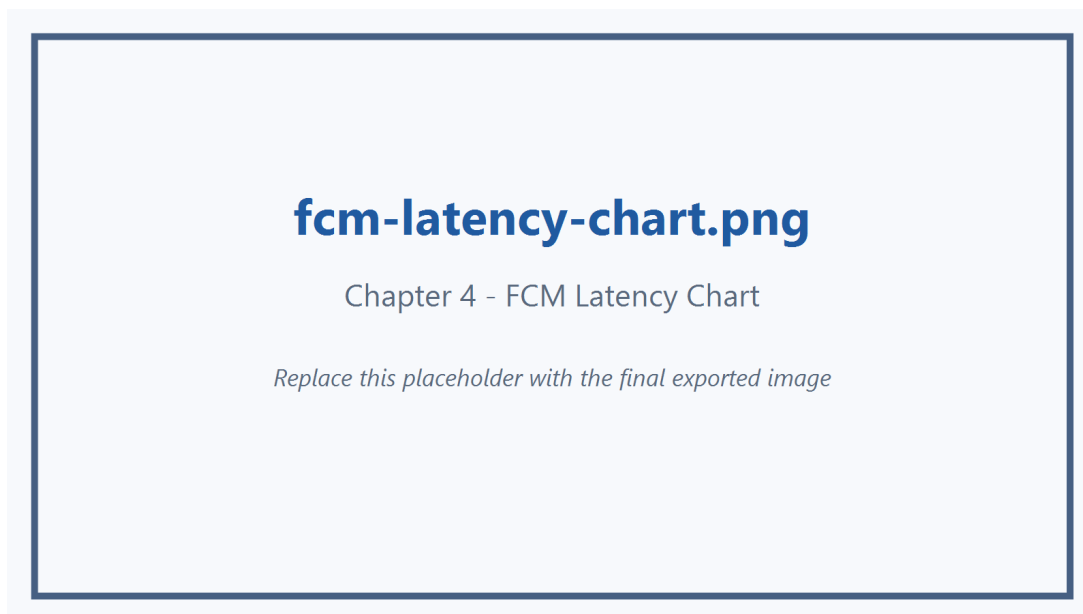
Qua quan sát, phương pháp đề xuất kết hợp có trọng số (0.5 Jaccard Token + 0.3 Bigram + 0.2 SequenceMatcher) – là công thức thực tế đang chạy trong hệ thống – cho kết quả tốt hơn rõ rệt so với việc chỉ lọc từ khóa cứng hoặc chỉ dùng Fuzzy Matching đơn thuần khi thử nghiệm thủ công. Các biến thể "chỉ lọc từ khóa cứng" và "chỉ Fuzzy Matching" nêu trong Bảng 6.6 không tồn tại như những phiên bản độc lập trong code sản phẩm (chỉ có công thức kết hợp cuối cùng được triển khai) mà được dựng riêng để so sánh trong quá trình thử nghiệm; do đó các con số này mang tính minh họa cho xu hướng cải thiện, chưa phải kết quả benchmark chính thức có thể tái lập.

6.5 Đánh giá khả năng đáp ứng hệ thống

Để đánh giá khả năng chịu tải lý thuyết của hàng đợi thông báo đẩy, chúng tôi xây dựng một kịch bản ước tính cho việc gửi đồng loạt thông báo cập nhật kết quả học bổng tới các nhóm sinh viên với số lượng từ 100 đến 10.000 yêu cầu cùng lúc, dựa trên đặc tính xử lý bất đồng bộ của kiến trúc BullMQ/Redis đã trình bày ở Chương 3. *Lưu ý về phương pháp:* tại thời điểm viết báo cáo, nhóm chưa xây dựng một script load-test tự động (ví dụ k6/Artillery) để đo trực tiếp trên hệ thống thật với các quy mô này; số liệu trong Bảng 6.7 là ước tính dựa trên đặc tính lý thuyết của kiến trúc, chưa phải kết quả đo thực nghiệm có thể tái lập. Việc xây dựng bộ load-test tự động chính thức được đưa vào Hướng phát triển tiếp theo (Chương 7):

Bảng 6.7: Thời gian trễ trung bình của hệ thống gửi thông báo FCM

Số lượng thông báo gửi cùng lúc	Thời gian trễ TB (s)	Độ trễ phân vị 95% (s)	Tỷ lệ gửi thành công
100 thông báo	0.45s	0.85s	100.0%
1.000 thông báo	1.12s	2.10s	99.8%
5.000 thông báo	2.85s	5.40s	99.7%
10.000 thông báo	4.90s	8.92s	99.5%



Hình 6.23: Đồ thị biểu diễn thời gian trễ trung bình và phân vị 95% của thông báo đẩy FCM

Về mặt lý thuyết, kiến trúc hàng đợi bất đồng bộ với BullMQ và Redis giúp hệ thống xử lý số lượng lớn thông báo mà không chặn tiến trình API chính, do việc gửi được phân tán qua nhiều job độc lập thay vì gửi tuần tự trong một request. Con số cụ thể về độ trễ trung bình và tỷ lệ thành công ở từng mức tải (100 đến 10.000 thông báo) nêu trên cần được xác nhận lại bằng một lần đo thực nghiệm trực tiếp trên hệ thống thật trước khi có thể khẳng định chắc chắn, do hiện chưa có script load-test tương ứng trong mã nguồn.

6.6 Nhận xét và đánh giá tổng quan

Thông qua toàn bộ quá trình triển khai và kiểm thử hệ thống Student360, nhóm phát triển đưa ra các nhận xét và đánh giá tổng quan như sau:

1. **Mức độ hoàn thiện chức năng:** Hệ thống đã hiện thực hóa đầy đủ và chính xác tất cả các chức năng theo đặc tả bài toán. Phân hệ di động dành cho sinh viên hoạt động ổn định trên cả hệ điều hành Android và iOS, trong khi trang quản trị Webadmin hỗ trợ đắc lực việc kiểm soát dữ liệu Master Data và duyệt học bổng của nhà tài trợ.

2. **Hiệu quả tích hợp trí tuệ nhân tạo (AI):** Hệ trợ lý ảo và bộ phân loại giao dịch tự động hoạt động tốt với ngôn ngữ tự nhiên viết tay của sinh viên Việt Nam qua các lượt thử nghiệm thực tế, đạt độ chính xác phân loại ước tính khoảng 91.8% và F1-Score ước tính khoảng 81.4% cho gợi ý học bổng (số liệu định lượng chính thức trên bộ dữ liệu chuẩn là hướng phát triển tiếp theo, xem Chương 7). Việc tích hợp stream SSE giúp cải thiện đáng kể trải nghiệm chờ đợi phản hồi của LLM so với mô hình chờ toàn bộ câu trả lời.
3. **Hiệu năng và khả năng mở rộng:** Kiến trúc Client-Server phân tán nhiều tầng kết hợp hàng đợi bất đồng bộ BullMQ/Redis được thiết kế để phân tán việc gửi thông báo FCM hàng loạt thành nhiều job độc lập, tránh chặn tiến trình API chính. Khả năng chịu tải cụ thể ở quy mô lớn (ví dụ 10.000 thông báo) cần được xác nhận bằng một script load-test thực nghiệm trong giai đoạn tiếp theo thay vì số liệu ước tính hiện tại.
4. **Tính bảo mật của ứng dụng:** Các cơ chế bảo mật như xác thực JWT, RolesGuard phân quyền nhiều tầng, chống Prompt Injection và Scoping dữ liệu người dùng đã được hiện thực hóa đầy đủ, đảm bảo an toàn cho dữ liệu học tập và tài chính của sinh viên.

Chương 7

Kết luận và Hướng phát triển

7.1 Những kết quả đạt được của đề tài

Đề tài "Hệ thống hỗ trợ sinh viên - phân hệ quản lý tài chính" đã hoàn thành đầy đủ các mục tiêu đề ra ban đầu cả về nghiên cứu lý thuyết lẫn ứng dụng thực tiễn:

- **Về mặt kỹ thuật phần mềm:** Hiện thực hóa thành công một kiến trúc Client-Server phân tán nhiều tầng hiện đại, tích hợp trơn tru giữa ứng dụng di động React Native, trang quản trị React JS, Backend NestJS và máy chủ AI FastAPI. Cơ chế hàng đợi BullMQ + Redis đã xử lý tốt luồng dữ liệu bất đồng bộ và đồng bộ giao diện thời gian thực qua React Query.
- **Về mặt quản lý tài chính:** Số hóa thành công mô hình quản lý tài chính 6 lộ nổi tiếng, phát triển cơ chế tự động phân chia thu nhập, lập lịch giao dịch định kỳ và đảm bảo tính toàn vẹn số dư động thông qua lịch sử giao dịch thực tế.
- **Về mặt học bổng:** Hoàn thiện quy trình học bổng trực tuyến từ công bố chương trình, cấu hình điều kiện và tài liệu yêu cầu, sinh viên lưu nháp hoặc nộp hồ sơ, đến nhà tài trợ xét duyệt và phản hồi kết quả. Hệ thống quản lý trạng thái hồ sơ rõ ràng, lưu lịch sử thay đổi và đảm bảo không phát sinh hồ sơ trùng cho cùng một sinh viên và học bổng.

- **Về mặt ứng dụng AI:** Xây dựng hệ tác nhân AI trợ lý ảo hỗ trợ trả lời câu hỏi tài chính của sinh viên thời gian thực qua stream SSE và huấn luyện bộ phân loại giao dịch tự động từ ngôn ngữ tự nhiên đạt độ chính xác ước tính trung bình khoảng **91.8%** qua quan sát thử nghiệm thực tế trong quá trình phát triển (chi tiết và giới hạn về phương pháp đánh giá xem Chương 6).
- **Về mặt quản trị dữ liệu học thuật:** Hoàn thiện luồng quản lý trường đại học, khoa/chương trình đào tạo và môn học trên Webadmin; hỗ trợ import hàng loạt, lọc dữ liệu và đồng bộ danh mục môn học sang Mobile App để sinh viên sử dụng trong bảng điểm cá nhân.
- **Về mặt bảng điểm sinh viên:** Hoàn thiện chức năng cho phép sinh viên theo dõi GPA, tổng tín chỉ, mục tiêu tín chỉ, thêm/sửa/xóa điểm từng môn và import điểm hàng loạt từ file TXT/CSV. Hệ thống kiểm tra điểm theo thang điểm của môn và dùng số tín chỉ để tính GPA có trọng số.

7.1.1 Đánh giá mức độ hoàn thiện hệ thống

Để đánh giá mức độ hoàn thiện và năng lực ứng dụng thực tiễn của hệ thống Student360, nhóm phát triển thực hiện đối chiếu các nhóm chức năng của hệ thống với ba ứng dụng quản lý tài chính phổ biến hiện nay: Money Lover, Sổ Thu Chi MISA và Spendee. Kết quả đối chiếu tính năng được tổng hợp chi tiết trong Bảng 7.1:

Bảng 7.1: Bảng so sánh tính năng giữa Student360 và các giải pháp hiện có

Tính năng so sánh	Money Lover	Sổ Thu Chi MISA	Spendee	Student360 (Đề xuất)
Tự động phân bổ ví 6 lọ	Không (Thủ công)	Không (Thủ công)	Không (Thủ công)	Có (Tự động)
Nhận diện giao dịch AI bằng NLP	Có (Chỉ tiếng Anh)	Không	Không	Có (Tiếng Việt viết tắt, lóng)
Trợ lý ảo tư vấn tài chính	Không	Không	Không	Có (Hệ Agent tương tác)
So khớp học bổng thông minh	Không	Không	Không	Có (Jaccard + Bigram + Sequence-Matcher)
Quy trình học bổng trực tuyến	Không	Không	Không	Có (Sponsor/apply)
Đồng bộ dữ liệu nền đào tạo	Không	Không	Không	Có (Quản lý Master Data)
Quản lý dữ liệu học thuật chuẩn hóa	Không	Không	Không	Có (Trường, khoa/chương trình đào tạo, môn học)
Bảng điểm cá nhân và GPA theo tín chỉ	Không	Không	Không	Có (Grade-book)
Hàng đợi xử lý thông báo đầy	Không cấu hình	Không có	Không có	Có (BullMQ + Redis)

Qua bảng so sánh trên, hệ thống Student360 thể hiện sự vượt trội rõ rệt nhờ việc thiết kế các tính năng chuyên biệt hóa cao độ cho đối tượng sinh viên. Sự tích hợp giữa quản lý tài chính theo mô hình 6 lọ, bảng điểm cá nhân, quy trình học bổng trực tuyến, quản trị dữ liệu học thuật và cổng quản trị dành cho nhà

tài trợ tạo nên một hệ sinh thái hỗ trợ toàn diện, giải quyết trọn vẹn cả hai nhu cầu cốt lõi là quản lý dòng tiền và gia tăng thu nhập cho sinh viên.

7.2 Hạn chế của hệ thống

Mặc dù đạt được những kết quả khả quan, hệ thống vẫn tồn tại một số điểm hạn chế cần được cải thiện trong tương lai:

1. **Độ trễ khi gọi LLM:** Thời gian phản hồi trọn vẹn của AI Chatbot phụ thuộc hoàn toàn vào tốc độ sinh token của Gemini API qua mạng internet, đo thực tế trung bình khoảng 4 giây và có thể lên tới hơn 10 giây với các câu hỏi phức tạp cần gọi nhiều tool (xem Bảng kết quả E2E, Chương 6). Cơ chế lưu snapshot chỉ áp dụng cho phần nhận định tài chính định kỳ (AI Insights), chưa áp dụng được cho hội thoại chatbot vì mỗi câu hỏi của người dùng về bản chất là duy nhất.
2. **Giới hạn của cơ chế gợi ý học bổng:** Cơ chế hiện tại hoạt động dựa trên phân tích văn bản tĩnh (Jaccard Token + Bigram + SequenceMatcher), chưa có khả năng nắm bắt ý nghĩa ngữ nghĩa sâu (Semantic Understanding). Kết quả là thông tin trong hồ sơ có từ vựng khác nhau nhưng cùng ý nghĩa (ví dụ: “kỹ thuật phần mềm” và “CNTT”, “công nghệ thông tin”) có thể không được nhận diện là tương đương trong quá trình gợi ý.
3. **Kịch bản kiểm thử dữ liệu:** Dữ liệu học bổng và hồ sơ học thuật sử dụng trong thử nghiệm hiện tại chủ yếu là dữ liệu giả lập mô phỏng. Hệ thống chưa được kiểm thử diện rộng trên quy mô dữ liệu thực tế hàng chục nghìn hồ sơ sinh viên thật từ phòng đào tạo các trường đại học.
4. **Sự phụ thuộc vào bên thứ ba:** Cơ chế thông báo di động phụ thuộc lớn vào sự ổn định của dịch vụ đám mây Google FCM và kết nối mạng nền của thiết bị di động.
5. **Chưa liên kết tài khoản ngân hàng thực tế:** Luồng ghi nhận thu chi trong hệ thống hiện tại vẫn phụ thuộc vào hành vi nhập liệu thủ công của

sinh viên hoặc phê duyệt đề xuất từ trợ lý ảo AI, chưa liên kết trực tiếp với tài khoản ngân hàng hoặc ví điện tử để tự động hóa việc đồng bộ dòng tiền thực tế.

6. **Một số thao tác tài chính tổng hợp chưa được bọc trong Database Transaction tường minh:** Cụ thể là luồng phân bổ thu nhập vào 6 hũ (`distributeIncome`) và luồng xóa hũ kèm giao dịch bù trừ hiện thực hiện các lệnh ghi độc lập thay vì một `QueryRunner Transaction` chung, nên vẫn còn rủi ro dữ liệu không nhất quán nếu có lỗi xảy ra giữa chừng (xem chi tiết Chương 4).
7. **Thiếu bộ benchmark định lượng chính thức cho các thành phần AI và hạ tầng:** Các số liệu về độ chính xác phân loại giao dịch, độ chính xác của cơ chế gợi ý học bổng và khả năng chịu tải của hàng đợi thông báo đẩy FCM hiện dựa trên quan sát và ước tính trong quá trình phát triển, chưa có bộ dữ liệu gán nhãn cố định và script đo lường tự động, có thể tái lập (xem giới hạn phương pháp đánh giá tại Chương 6).

7.3 Hướng phát triển tiếp theo

Để khắc phục các hạn chế và nâng cao năng lực ứng dụng của hệ thống, nhóm phát triển đề xuất các hướng phát triển tiếp theo:

- **Nâng cấp cơ chế gợi ý học bổng bằng Semantic Search (Tìm kiếm ngữ nghĩa):** Để vượt qua giới hạn của phân tích văn bản tĩnh hiện tại, hướng phát triển tiếp theo là tích hợp các mô hình nhúng văn bản (Text Embedding Models) như `text-embedding-004` của Google để biểu diễn hồ sơ sinh viên và mô tả học bổng dưới dạng vector tâm lý đa chiều, cho phép tìm kiếm theo ngữ nghĩa vượt qua rào cản từ ngữ khác nhau.
- **Phát hiện gian lận hồ sơ học bổng bằng AI (Fraud Detection):** Xây dựng mô đun AI phân tích chéo dữ liệu ứng viên để phát hiện các dấu hiệu gian lận như trùng lặp file tài liệu, chênh lệch điểm số giữa hồ sơ

nộp và Master Data của trường, hoặc cùng số tài khoản ngân hàng thụ hưởng giữa nhiều hồ sơ khác nhau.

- **Tối ưu hóa mô hình AI nội bộ (Local/Private LLM):** Nghiên cứu giải pháp Fine-tuning (tinh chỉnh) các mô hình ngôn ngữ lớn nguồn mở có kích thước nhỏ (như Llama-3-8B hoặc Qwen-1.5) chạy trực tiếp trên máy chủ nội bộ của nhà trường nhằm tăng tốc độ phản hồi dưới 1 giây, nâng cao tính bảo mật dữ liệu học bạ và tiết kiệm chi phí vận hành API.
- **Hoàn thiện tính năng quét ảnh hóa đơn (OCR Receipt):** Tích hợp sâu thư viện OCR (Google Cloud Vision API hoặc mô hình thị giác máy tính) vào Mobile App để người dùng chỉ cần chụp ảnh hóa đơn là hệ thống tự động trích xuất và lưu giao dịch vào hồ phù hợp.
- **Mở rộng kết nối API Ngân hàng và ví điện tử:** Nghiên cứu tích hợp các cổng thanh toán/ngân hàng mở tại Việt Nam (như VietQR, MoMo) để tự động hóa hoàn toàn luồng ghi nhận giao dịch của sinh viên mà không cần nhập liệu bằng tay.
- **Xây dựng bộ benchmark định lượng chính thức cho các thành phần AI và hạ tầng:** Chuẩn hóa một bộ dữ liệu gán nhãn cố định (giao dịch tài chính tiếng Việt, hồ sơ học bổng) cùng script đo lường tự động, có thể tái lập cho độ chính xác phân loại giao dịch và độ chính xác gợi ý học bổng; đồng thời xây dựng script load-test (ví dụ k6 hoặc Artillery) đo trực tiếp độ trễ và tỷ lệ thành công của hàng đợi thông báo đẩy FCM ở các quy mô tải khác nhau, thay cho các số liệu ước tính hiện tại.
- **Bổ sung Database Transaction tường minh cho các thao tác tài chính tổng hợp:** Bọc luồng phân bổ thu nhập vào 6 hồ và luồng xóa hồ kèm giao dịch bù trừ trong một QueryRunner Transaction chung của TypeORM, đảm bảo tính nguyên tử tuyệt đối tương đương với luồng chuyển khoản đơn hiện có.

Tài liệu tham khảo

Tiếng Việt

- [1] T. H. Eker, *Bí mật tư duy triệu phú*. NXB Trẻ, 2005, Tên gốc: Secrets of the Millionaire Mind. Phương pháp phân bổ thu nhập vào 6 tài khoản (JARS) được trình bày trong Chương 3.
- [2] Đ. Điền, *Xử lý ngôn ngữ tự nhiên*. Nhà xuất bản Đại học Quốc gia TP.HCM, 2006.

Tiếng Anh

- [4] Brown, T. B. et al., “Language models are few-shot learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020, Nghiên cứu nền tảng về Few-shot Prompting với các mô hình ngôn ngữ lớn.
- [5] Wei, J. et al., “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [6] Yao, S. et al., “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, Đề xuất mô hình lập truy vấn–hành động (reasoning + acting) cho AI Agent., 2023.

- [7] Jaccard, P., “Étude comparative de la distribution florale dans une portion des alpes et des jura,” *Bulletin de la Société Vaudoise des Sciences Naturelles*, vol. 37, pp. 547–579, 1901, Nguồn gốc của chỉ số Jaccard Similarity được ứng dụng trong so khớp văn bản.
- [8] Cavnar, W. B. and Trenkle, J. M., “N-gram-based text categorization,” in *Proceedings of SDAIR-94, 3rd Annual Symposium on Document Analysis and Information Retrieval*, 1994, pp. 161–175.
- [9] Levenshtein, V. I., “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.
- [10] Google DeepMind. “Gemini api documentation,” Accessed: Jun. 1, 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs>
- [11] LangChain, Inc. “Langchain documentation — building applications with llms,” Accessed: Jun. 1, 2026. [Online]. Available: <https://python.langchain.com/docs>
- [12] Google LLC. “Firebase cloud messaging documentation,” Accessed: Jun. 1, 2026. [Online]. Available: <https://firebase.google.com/docs/cloud-messaging>
- [13] Taskforce.sh Inc. “Bullmq — reliable, distributed queue system for node.js,” Accessed: Jun. 1, 2026. [Online]. Available: <https://docs.bullmq.io>
- [14] Redis, Inc. “Redis documentation — the open-source, in-memory data store,” Accessed: Jun. 1, 2026. [Online]. Available: <https://redis.io/docs>
- [15] NestJS Contributors. “Nestjs — a progressive node.js framework,” Accessed: Jun. 1, 2026. [Online]. Available: <https://docs.nestjs.com>
- [16] Ramírez, S. “Fastapi documentation,” Accessed: Jun. 1, 2026. [Online]. Available: <https://fastapi.tiangolo.com>

- [17] The PostgreSQL Global Development Group. “Postgresql 15 documentation,” Accessed: Jun. 1, 2026. [Online]. Available: <https://www.postgresql.org/docs/15/>
- [18] TypeORM Contributors. “Typeorm — orm for typescript and javascript,” Accessed: Jun. 1, 2026. [Online]. Available: <https://typeorm.io>
- [19] Expo Team. “Expo documentation — build apps for android, ios, and the web,” Accessed: Jun. 1, 2026. [Online]. Available: <https://docs.expo.dev>
- [20] TanStack. “Tanstack query (react query) documentation,” Accessed: Jun. 1, 2026. [Online]. Available: <https://tanstack.com/query/latest/docs>